



山东大学



计算思维

数据结构（8学时）

千载文脉凝风骨，根植山大盛芳华

主讲人：李琨

时间：2024年秋季学期



4.1 数、数据及数据结构

- 数、数据、数据结构的概念
- 数据的逻辑结构
 - 集合
 - 线性表
 - 树
 - 图
- 数据的存储结构
 - 顺序存储结构
 - 链式存储结构

知识要点

数，

数据，

数据类型，

简单数据类型，

构造数据类型，

数据结构，

抽象数据类型，

逻辑结构，

物理结构（存储结构），

顺序存储结构，

链式存储结构

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数 (Number)

- 自然数，表示物体个数的数 $0, 1, 2, 3, \dots$
- 整数，原始概念，是正整数、负整数，以及0这样的一个序列，如： $\{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$
- 分数，分数源于“分”数，是一个数学术语，其定义是把单位“1”平均分成若干份，表示这样的一份或几份的数叫分数。
- 有理数
 - 整数和分数的统称
 - 加法、减法、乘法、除法（除数不为零）4种运算

数 (Number) --2无理数

• 无理数

- 一个正方形的边长是1的时候，对角线的长 m 等于多少？
- 毕达哥拉斯学派弟子**希伯斯** (Hippasus, 公元前625年~公元前547年) 断言： m 既不是整数也不是分数，是当时人们还没有认识的新数，而这种数既不是整数也不是分数，不好理解，故称无理数 (irrational number)。数学史上将这一事件称为“第一次数学危机”。
- 无理数是指实数范围内不能表示成两个整数之比的数，无理数就是10进制下的无限不循环小数。
- 实数，有理数和无理数的统称，实实在在存在的数

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

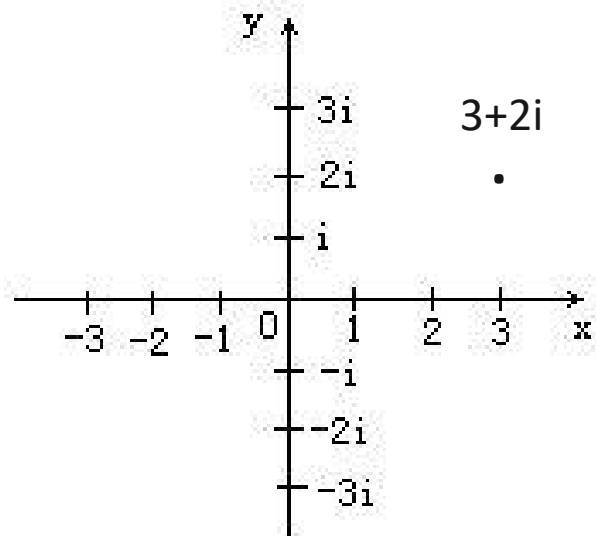
4.4 图结构

二、实例

4.5 应用举例

数 (Number) --3虚数

- 17世纪，著名数学家、哲学家笛卡尔创造了虚数 (imaginary number) 的概念。虚数就是其平方是负数的数， $i^2=-1$ 。
- 当时的观念认为这是真实不存在的数字。后来发现虚数可对应平面上的纵轴，与对应平面上横轴的实数同样真实。



数 (Number) --4复数

- 19世纪初，高斯系统地使用了 i 这个符号，并主张用数偶 (a, b) 来表示 $a+bi$ ，称为复数
 - i 表示虚数的单位
 - a, b 为实数， a, b 都不等于0时叫复数， a 等于0时叫纯虚数， b 等于0时就是实数
- 虚数闯进数的领域时，人们对它的实际用处一无所知，在实际生活中似乎没有用复数来表达的量
- 现在，复数一般用来表示向量（有方向的量），在水利学、地图学、航空学中的应用十分广泛，虚数越来越显示出其丰富的内容。

数据 (Data)

- 数据 (Data) 是对客观事物的符号表示，他是一组表示数量、行动和目标的非随机的可鉴别的符号。它可以是数字、字母等符号。
- 数据通常是指一切可以输入到计算机中并能被计算机程序处理的所有符号的总称。
 - 数字、字符等文本数据
 - 图形、图像数据
 - 音频数据
 - 动画、视频数据

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数据抽象与数据类型

- 数据类型，数据类型是具有同类性质的数据的抽象
 - 决定数据在计算机中所占的存储空间大小
 - 还决定数据的性质，如：数据的取值范围、操作运算等
- 简单类型
 - 整数，实数，字符，布尔
- 构造类型
 - 数组，结构

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数据类型的意义

- 数据存储空间大小
 - 例如：short x,y
- 决定数据解析规则
 - 两个字节
 - 0110 0001
 - 0110 0010
 - 字符：a,b
 - short：24934
- 决定数据操作运算
 - 程序编译

	7	6	5	4	3	2	1	0
x {	0	1	1	0	0	0	0	1
	0	1	1	0	0	0	1	0
y {								

图 4-1 数据、变量和数据类型



数据结构

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

• 数据结构

- 数据之间的关系称为数据结构 (Structure)
 - 逻辑结构:数据之间的逻辑关系称为数据的逻辑结构
 - 存储结构:数据结构在计算机中的表示称为数据的存储结构

• 抽象数据类型

- 数据
- 数据间的关系
- 在相应存储结构下对数据的操作运算, 即算法

【何谓数据结构, 算法又是什么? 】

https://www.bilibili.com/video/BV1aN411R7nd/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e



数据的逻辑结构

- 集合
- 线性结构
- 树形结构
- 图状结构

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

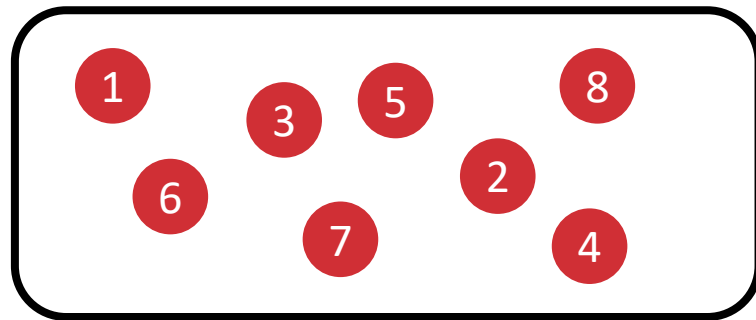
4.4 图结构

二、实例

4.5 应用举例

集合

- 集合的原始概念。有某种共性的东西，集中到一起，作为一个整体来看待时，就称为一个集合。
- 集合关系：数据元素的关系非常松散，只描述数据元素是否同在一个集合中



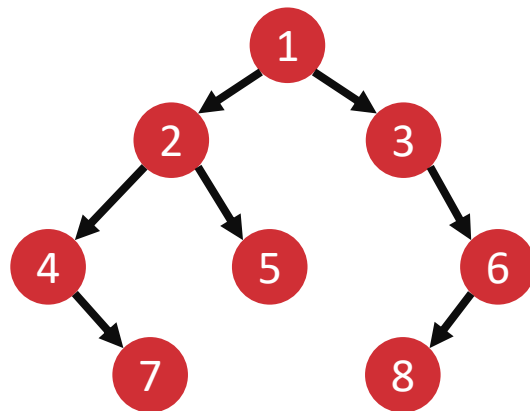
线性结构

- 数据元素之间存在线性关系，也称先后关系，每个元素都有一个唯一的前导元素和一个唯一的后继元素
- 第一个元素没有前导，最后一个元素没有后继。



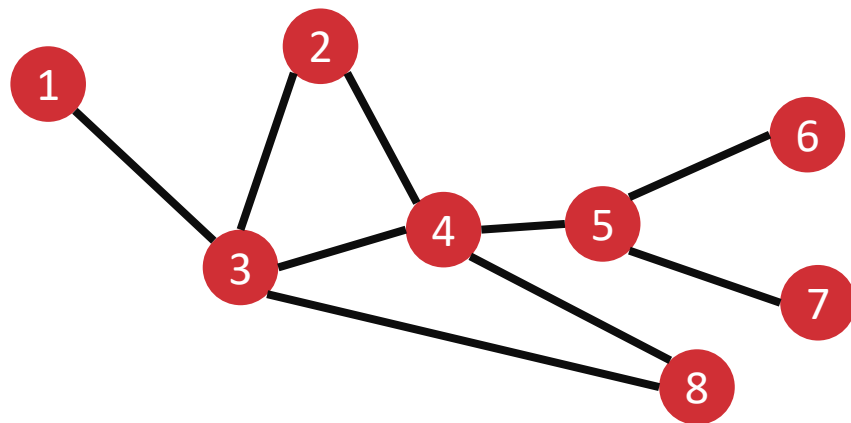
树形结构

- 数据元素之间呈现层次关系，在树形结构中，每一个元素通常称为一个节点，每个节点（根节点除外）有一个父节点，一个节点可以有多个子节点
- 例如：家族族谱, 抽象语法树



图状结构

- 元素呈多对多的关系，在图状结构中每个元素称为一个结点，图状结构又称网状结构。图结构可表达元素之间的任意关系。
- 例如，城市之间关系，可以有不同的交通线路连接





数据的存储结构

- 数据结构在计算机中的表示称为数据的存储结构，
又称物理结构
 - 存储数据
 - 存储关系
- 顺序存储
- 链式存储

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

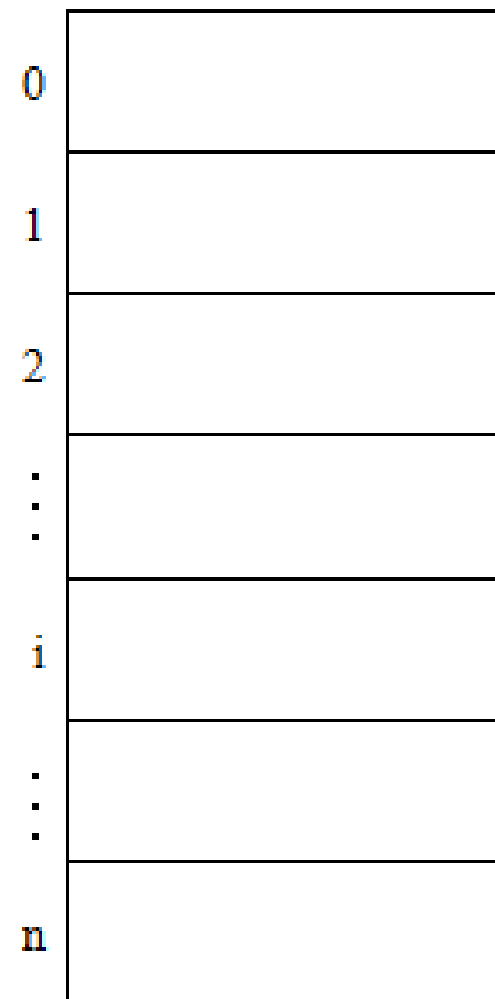
4.5 应用举例

顺序存储结构

- 用一组地址连续的存储单元依次存储数据集中的元素
- 借助于数据在存储器中的相对位置表示数据之间的关系

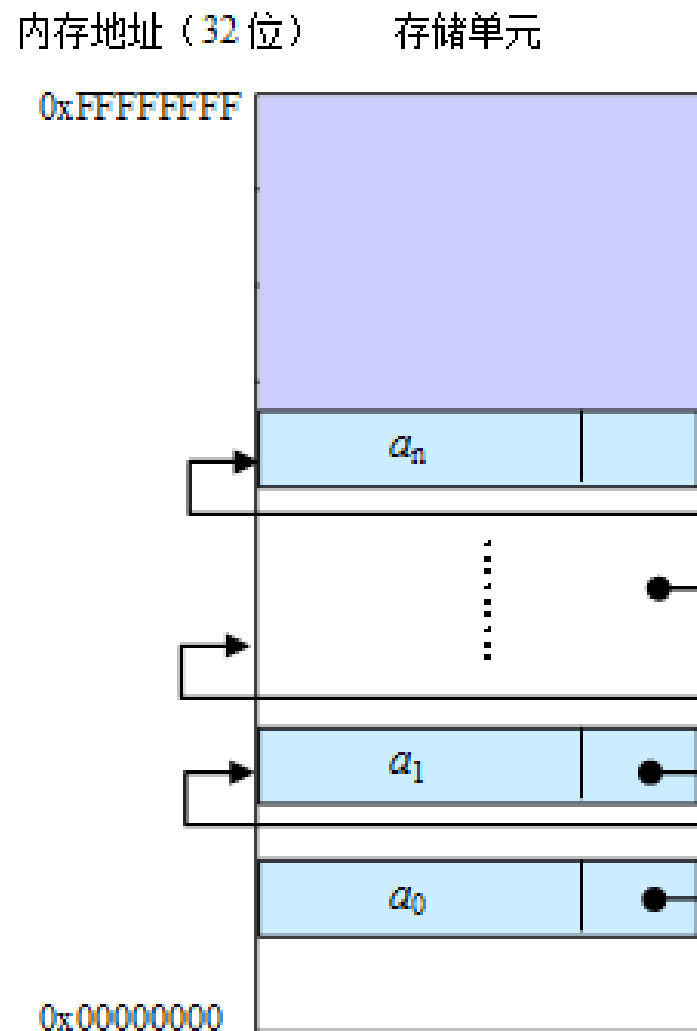
$$\text{LOC}(a_i) = \text{LOC}(a_1) + (i-1)$$

- 优缺点
 - 存储位置表示元素之间的关系，元素的位置可以直接计算得到，许多算法容易实现，并且效率较高，时间复杂性通常为一个常数。
 - 如果元素集合很大，顺序存储要求一块很大的连续的内存空间，这在操作系统的存储管理上可能无法满足



链式存储结构

- 每个数据元素存储为一个结点，每个结点有一个或多个指针，指向其他结点，指针表示了结点（数据元素）之间的逻辑关系。
- 优缺点
 - 无法用元素的存储位置表示元素之间的关系，它需要增加指针。指针增加了数据结构的存储空间要求
 - 对元素的许多操作算法，在实现上也变得较为麻烦，效率较低。
 - 链式存储增加了数据结构的灵活性，它无需事先指定元素的个数，



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

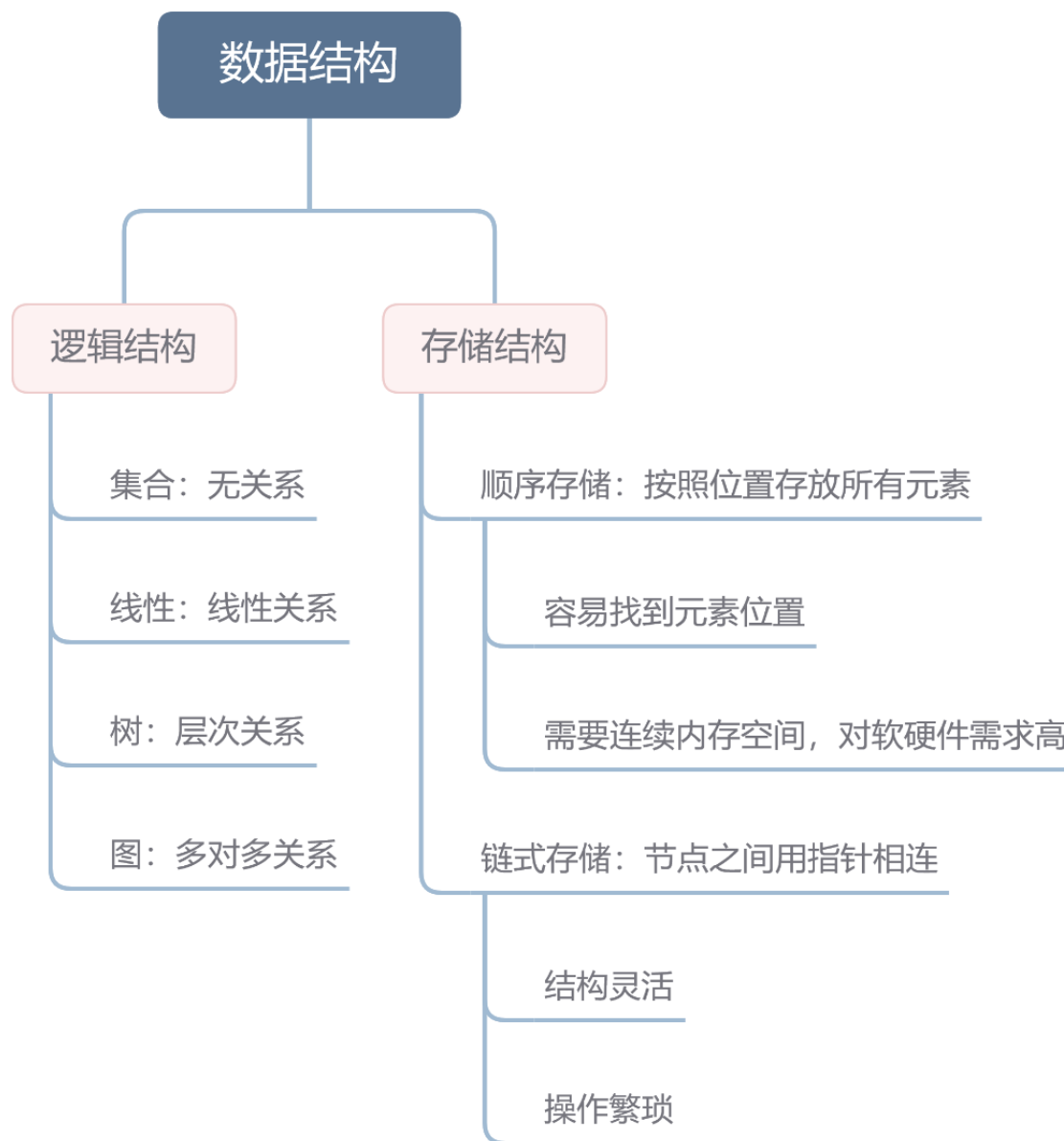
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

总结





4.2 线性结构

- 排队问题及其抽象
- 线性表
 - 线性表的概念
 - 线性表的基本操作
 - 线性表的存储结构
- 堆栈
 - 堆栈的概念
 - 栈的基本操作
 - 栈的表示和实现
- 队列
 - 队列的概念
 - 队列的基本操作
 - 队列的表示和实现

知识要点

先后关系，
线性结构，
线性表，
线性表基本操作，
堆栈，
队列。

排队问题及其抽象

- 生活中排队问题

- 车站购票、医院挂号等服务窗口的排队
- 银行业务
- 高速收费站、机场安检等
- 车辆调度

- 排队机

- 一个队列
- 保证了先到先服务



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

排队机

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

• 等待叫号



线性结构

- 线性结构是一种描述元素先后关系的数据结构
- 线性结构特点
 - (1) 存在一个唯一的称为“第一个”的数据元素，存在一个唯一的称为“最后一个”的数据元素；
 - (2) 除了“第一个”元素以外，每一个元素都有一个唯一的前导元素，第一个元素没有前导；除“最后一个”数据元素外，每一个元素都有一个唯一的后继元素，最后一个元素没有后继。
- 常见线性结构
 - 线性表
 - 堆栈
 - 队列

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

线性表

- 线性表 (Linear list) 是一个具有 n ($n \geq 0$) 个数据元素的有序序列。

- 线性表的形式定义

linear_list = (D,R)

其中, $D=\{a_i \mid a_i \in D_0, i=1,2,\dots,n, n \geq 0\}$; $R=\{N\}$, $N=\{<a_{i-1}, a_i> \mid a_{i-1}, a_i \in D_0, i=2,3,\dots,n, n \geq 0\}$ 。

D_0 为性质相同的数据元素的集合。

R 是有序偶的集合, 表示元素集中数据元素之间的相邻关系, 元素 a_{i-1} 是 a_i 的前导, a_i 是 a_{i-1} 的后继。

线性表中数据元素的个数 n 称为线性表的长度。 $n=0$ 时, 线性表称为空表。

$n>0$ 时, 线性标记作:

$(a_1, a_2, \dots, a_n)$

线性表的基本操作

- (1) $\text{Length}(L)$: 求线性表的长度, 返回线性表中数据元素的个数。
- (2) $\text{Get}(L,i)$: 取元素操作, 返回线性表 L 中的第 i 个元素
- (3) $\text{Locate}(L,x)$: 定位操作, 给定值 x , 判断线性表中是否有和 x 相同的元素。如果存在, 返回第一个和 x 相同的元素在线性表中的位序; 否则, 返回0。
- (4) $\text{Prior}(L,e)$: 前导函数, 返回线性表 L 中元素 e 的前导元素。如果 e 为第一个元素, 返回空。
- (5) $\text{Next}(L,e)$: 后继函数, 返回线性表 L 中元素 e 的后继元素。如果 e 为最后一个元素, 返回空。
- (6) $\text{Insert}(L,i,e)$: 插入操作, 在线性表 L 的第 i 个元素的前面插入一个元素 e 。
- (7) $\text{Delete}(L,i)$: 删除操作, 将线性表 L 的第 i 个元素删除。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

线性表的顺序存储结构

- 线性表的顺序存储结构可以用数组（向量）来表示

```
typedef struct l {
    ElementType elements[MAXSIZE];

    int length;
} SqList
```

`linear_list = c(1,2,3,4)`

Environment	History	Connections	Tutorial
   Import Dataset	 145 MiB		
R	Global Environment		
Values			
linear_list		num [1:4]	1 2 3 4

elements

0	a_1
1	a_2
$\vdots$	$\vdots$
i	a_n
$\vdots$	$\vdots$
MAXSIZE-1	

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

(1) Length(L)操作

- (1) Length(L)操作，返回线性表L的长度。

```
int Length(const SqList *l)
```

```
{
```

```
    return (*l).length;
```

```
}
```

```
linear_list = c(1,2,3,4)
l = length(linear_list)
```

SqList 数据类型

elements						length
a_1	a_2	...	a_n	...		

Environment		History		Connections		Tutorial			
				 Import Dataset ▾		 198 MiB ▾			
R ▾				Global Environment ▾					
Values									
1					4L				
linear_list					num [1:4] 1 2 3 4				

(2) Get(L,i)操作

- (2) Get(L,i)操作，返回线性表L的第i个元素。

ElementType Get(const SqList *l,int i) SqList 数据类型

```
{
    if (i<(*l).length)
        return (*l).elements[i-1];
    else
        printf("Error");
}
```

```
linear_list = c(1,2,3,4)
l = length(linear_list)
i = 2
l_i = linear_list[i]
```

elements						length
a_1	a_2	...	a_n	...		

Environment	History	Connections	Tutorial	
   Import Dataset  198 MiB 				
R Global Environment				
Values				
i	2			
l	4L			
l_i	2			
linear_list	num [1:4] 1 2 3 4			

(3) Locate(L,x), 定位操作

- 判断线性表L中是否有和x相同的元素。
- // 如果存在, 返回第一个和x相等的元素
在线性表中的位序; 否则, 返回0。

SqList 数据类型

elements						length
a_1	a_2	...	a_n	...		

- int Locate(const SqList *l,ElementType x)
- {
- int i=0;
- for (i=0;i<(*l).length;i++)
- if (x==(*l).elements[i])
- return i+1;
- return 0;
- }

```
linear_list = c(1,2,3,4)
x = 3
i = 0
for (e in linear_list){
  i = i+1
  if (e == x){
    print(i)
    break
  }
}
```

Environment	History	Connections	Tutorial
   Import Dataset	 200 MiB		
R	Global Environment		
Values			
e	3		
i	3		
l	4L		
l_i	2		
linear_list	num [1:4] 1 2 3 4		
x	3		

(4) Insert(L,i,e), 插入元素操作

- Insert(L,i,e), 插入元素操作, 在线性表L的第i个元素前插入一个元素e。
- void Insert(SqList *l,int i,ElementType e)
- {
- //当前列表为空时, 插入到首个元素
- if ((*l).length==0) {
- (*l).elements[0]=e;
- (*l).length = 1;
- }
- else { //否则, 依次向后移动, 直到找到第i个元素, 并在其前面插入
- for (int j=(*l).length;j>=i;j--)
- (*l).elements[j]=(*l).elements[j-1];
- (*l).elements[i-1] = e;
- (*l).length ++;
- }
- }

SqList 数据类型

elements						length
a_1	a_2	...	a_n	...		

(4) Insert(L,i,e), 插入元素操作

```
linear_list = c(1,2,3,4)
linear_list = c(linear_list[1:2],10,linear_list[3:4])
```

```
linear_list = c(1,2,3,4)
x = 10
index = 3
l = length(linear_list)
if (l == 0){
  linear_list = c(x)
} else{
  for (i in l:index){
    linear_list[i+1] = linear_list[i]
  }
  linear_list[index] = x
}
```

Environment	History	Connections	Tutorial
   Import Dataset	 220 MiB		
R	Global Environment		
Values			
i	3L		
index	3		
l	4L		
linear_list	num [1:5] 1 2 10 3 4		
x	10		

(5) Delete(L,i), 删除操作

- Delete(L,i), 删除操作。删除线性表L中的第i个元素。

```
void Insert(SqList &l,int i)
```

```
{
```

```
    if (i<=l.length) {
```

```
    //从第i+1个元素开始, 依次向前覆盖
```

```
        for (int j=i+1;j<=l.length;j++)
```

```
            l.elements[j-2]=l.elements[j-1];
```

```
        l.length --;
```

```
    }
```

```
}
```

SqList 数据类型

elements						length
a_1	a_2	...	a_n	...		

(5) Delete(L,i), 删除操作

```
linear_list = c(1, 2, 10, 3, 4)
linear_list = linear_list[-3]
```

```
linear_list = c(1, 2, 10, 3, 4)
l = length(linear_list)
index = 3
for (i in index:(l-1)){
  linear_list[i] = linear_list[i+1]
}
linear_list = linear_list[1:l-1]
```

Environment	History	Connections	Tutorial
   Import Dataset ▾	 222 MiB ▾		
R ▾	 Global Environment ▾		
Values			
i	4L		
index	3		
l	5L		
linear_list	num [1:4] 1 2 3 4		

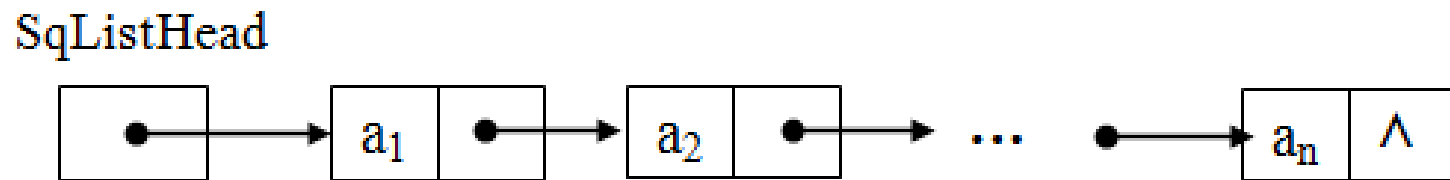
线性表的链式存储结构

• 链式存储结构数据类型定义

```
typedef struct Node{
    ElementType data;
    struct Node *next;
} Node;
```

Node *SqListHead;

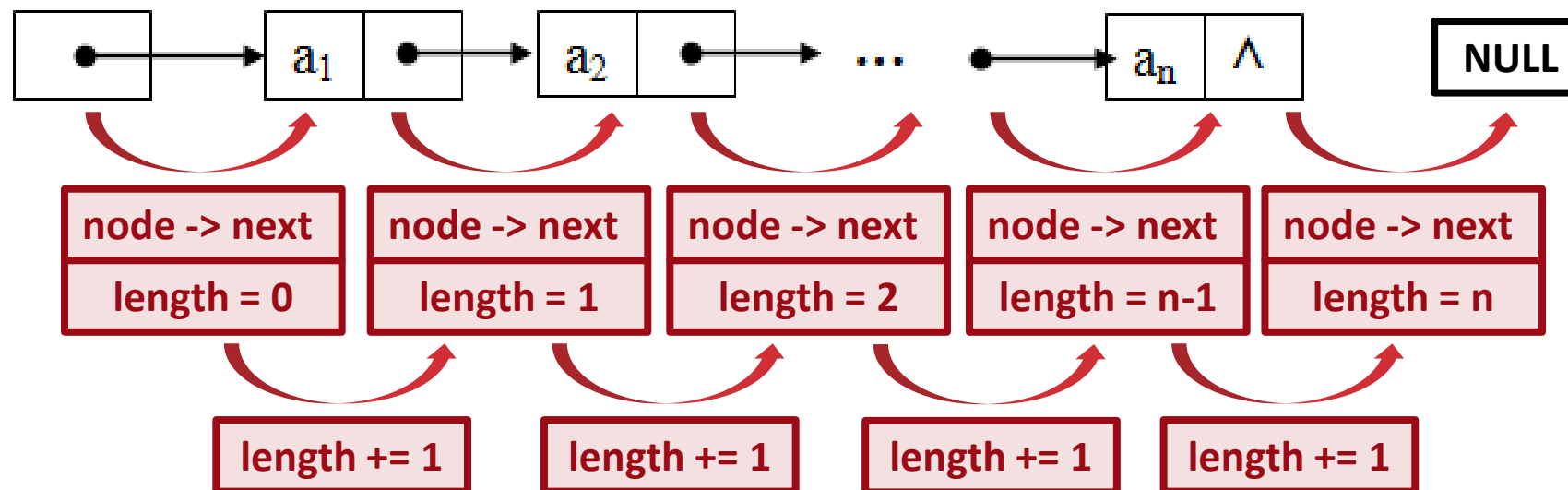
其中，SqListHead 为线性表的头指针，内存结构如下：



- 不再能通过下标索引找对应位置的元素，只能查看当前节点的元素
 - 可以通过 **新节点=当前节点 -> next** 的方式访问当前节点所指向的节点
- 例如：a2 = a1 -> next

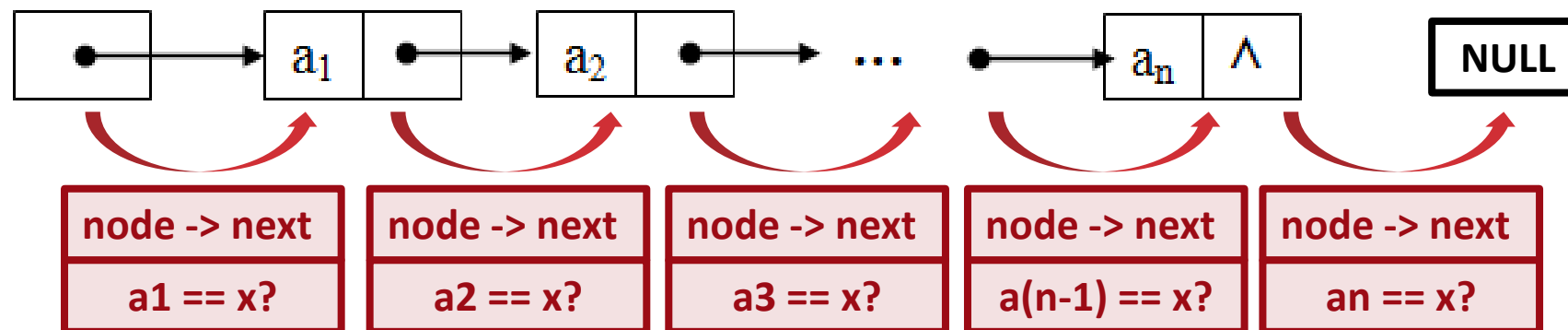
例4-2：链式存储结构中线性表的Length操作算法

SqListHead



链式存储结构中线性表的查找操作算法

SqListHead



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

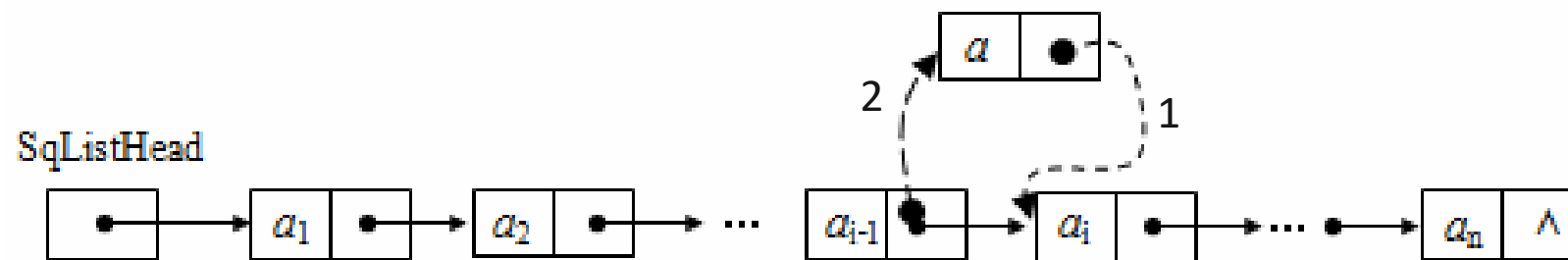
二、实例

4.5 应用举例

链式存储结构中线性表的Insert操作

插入一个新元素

- 第一个元素前
 - 空表
 - 非空表
- 中间元素
- 最后一个元素



Insert操作算法

在第一个位置插入：

是空表：

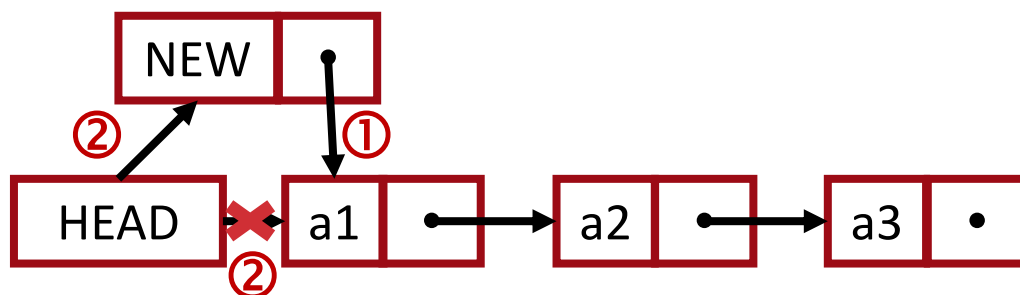
表头指针指向要插入的元素



不是空表：

1. 新插入的元素指向原本的第一个元素
2. 表头指针指向要插入的元素

能交换吗

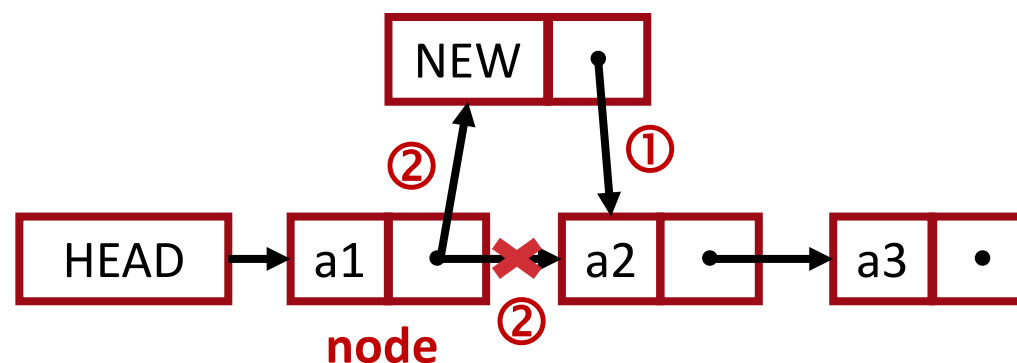


Insert操作算法

在中间位置插入：

1. 从表头开始通过 $node = node \rightarrow next$ 的方式找到要插入位置的前一个节点，此时该节点用 $node$ 表示
2. 新插入元素指向 $node \rightarrow next$
3. 当前节点指向新插入元素

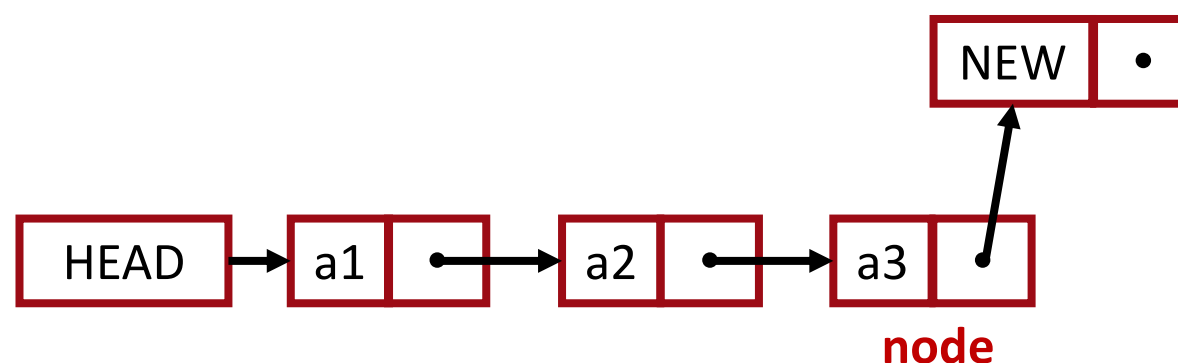
能交换吗



Insert操作算法

在最后一个位置插入：

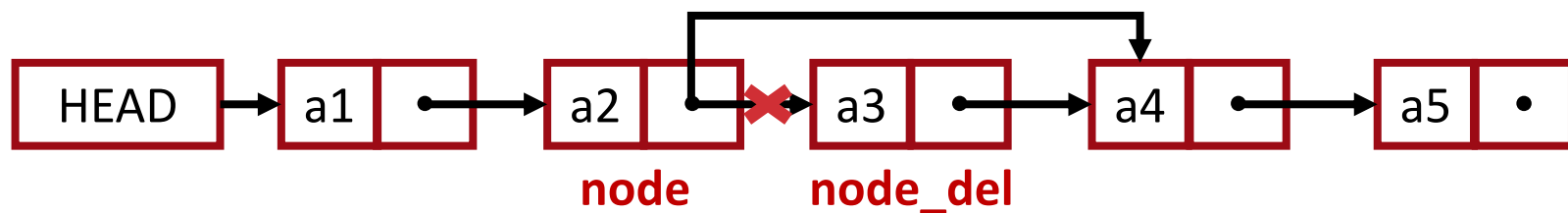
1. 从表头开始通过 $node = node \rightarrow next$ 的方式找到最后一个节点，此时该节点用 $node$ 表示
2. $node \rightarrow next = new$



链式存储结构中线性表的删除操作

删除第*i*个元素：待删除节点的前导指向后继

1. 通过`node = node -> next`的方式找到第*i*-1个节点，此时该节点用`node`表示
2. `node_del = node -> next` //用`node_del`表示要删除的节点
3. `node -> next = node_del -> next` //`node`指向`node_del`的后继节点



数据存储结构和算法的关系

- 同样的操作，在不同的存储结构中，其算法的实现也不相同，其算法的效率可能悬殊较大。
- 求线性表的长度操作
 - 顺序存储，时间复杂度为 $O(1)$,
 - 链式存储，却需要遍历整个单链表，复杂度为 $O(n)$ 。
- 对于插入操作
 - 顺序存储，需要大量的元素搬移
 - 链式存储，无需搬移操作。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

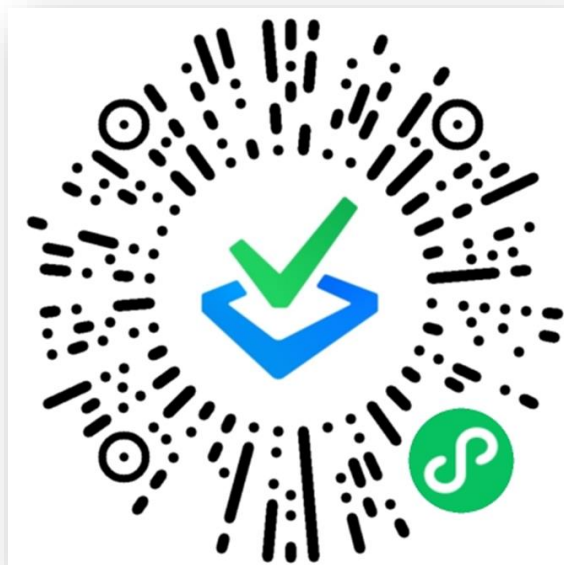
4.4 图结构

二、实例

4.5 应用举例

线性结构练习题

1. 采用顺序存储结构来存放1000个元素，现在需要在第2个位置插入新元素。请描述完成这个操作需要移动多少个元素？如果是在链式存储结构中执行相同的操作，需要移动多少个元素？
2. 小明需要存储100个整数，他犹豫是使用顺序存储结构还是链式存储结构。请分析在以下场景下哪种数据结构更合适：
 - a) 经常需要随机访问第k个数
 - b) 经常需要在中间位置插入新数字
3. 在一个长度为n的数组中找到最后一个元素,与在单链表中找到最后一个元素相比：
 - A. 数组更快
 - B. 链表更快
 - C. 两者一样快
 - D. 依赖于具体实现



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

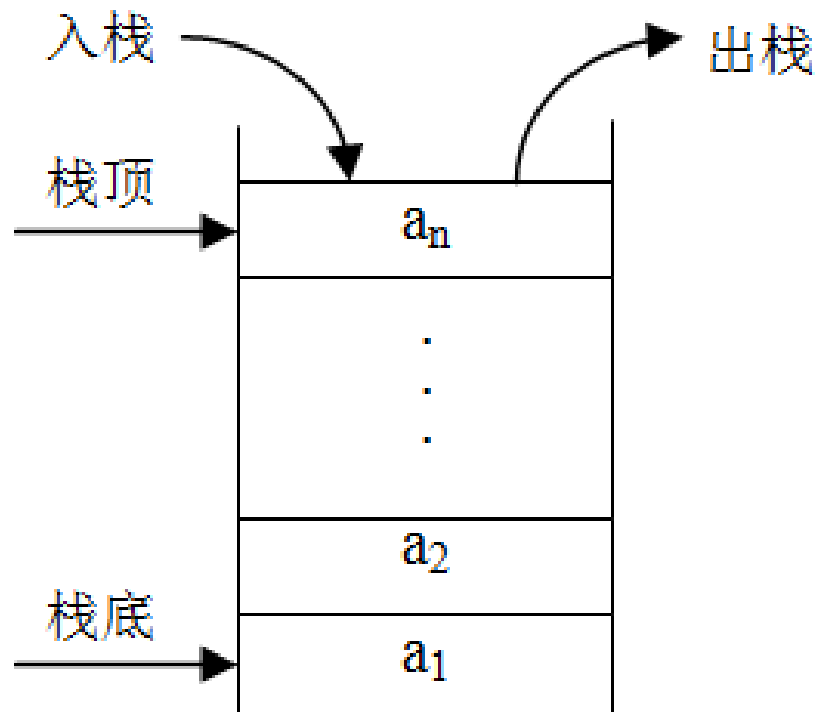
4.4 图结构

二、实例

4.5 应用举例

栈结构

- 栈 (Stack) 是操作受限的线性表, 只在表尾进行插入和删除操作
- 能够进行插入和删除操作的一端称为“栈顶”, 另一端称为“栈底”
- 栈是一种后进先出后进先出 (Last In First Out, 简称LIFO) 表。在递归算法、函数调用、面向对象程序设计等许多方面有着重要应用。



栈的基本操作

- (1) InitStack(S): 初始化操作, 将栈S设为空。
- (2) Push(S,x): 入栈操作, 将元素x压入栈S, 成为新的栈顶元素。
- (3) Pop(S): 出栈操作, S为堆栈, 若S不为空, 返回S的栈顶元素, 且从栈中删除该栈顶元素(退栈)。如果堆栈为空, 则函数返回值为NULL。
- (4) Gettop(S): 取栈顶元素操作, S为堆栈, 若S不为空, 返回S的栈顶元素。如果堆栈为空, 则返回值NULL。
- (5) Empty(S): 判断堆栈S是否为空, 若栈S为空, 返回True, 否则返回False。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

栈的示例

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

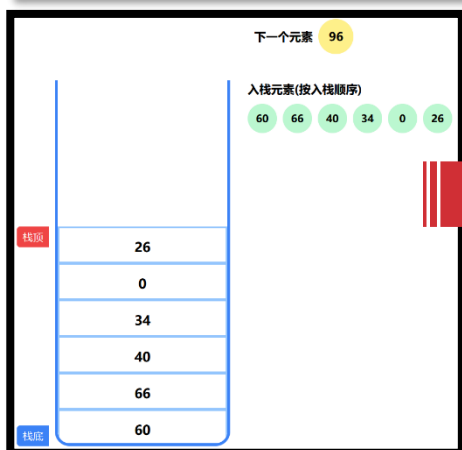
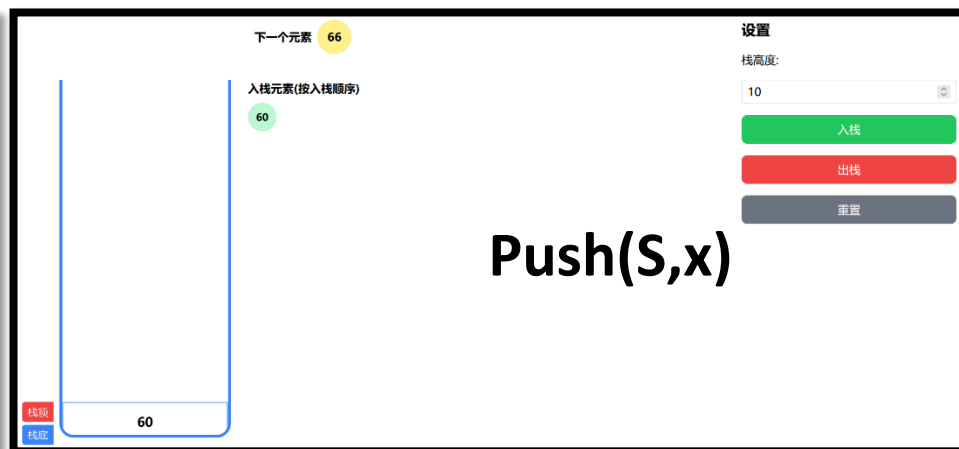
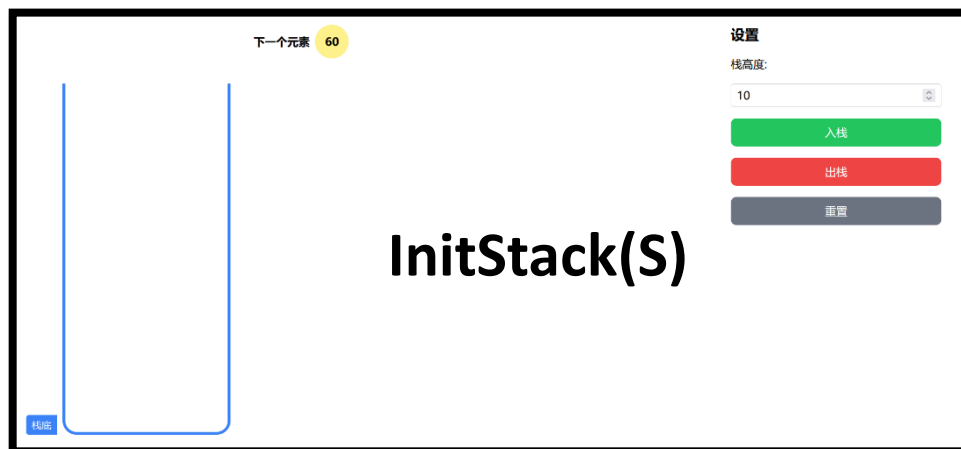
4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例



<https://gallery.selfboot.cn/zh/algorithms/stack>

<https://visualgo.net/zh/list>

<https://www.totuma.cn/algorithms/stack-queue/stack-sequence.html>

栈的应用示例：括号匹配

- 栈被用来验证括号的正确匹配：通过将左括号压入栈中，然后在遇到右括号时进行匹配，可以有效地检测括号是否正确闭合。
- 算法思路：正常来说，左右括号是一一匹配的，有一个(就会有一个)，{}和[]这两类也同理。
 1. 忽略字符串中除了括号以外的内容；
 2. 开始遍历所有括号，遇到左括号执行入栈操作，遇到右括号则看是否能与栈顶的元素匹配；
 3. 能匹配，则执行出栈，继续后续的遍历，直到循环结束；
 4. 不能匹配，则说明表达式中的括号是不匹配的，退出循环。

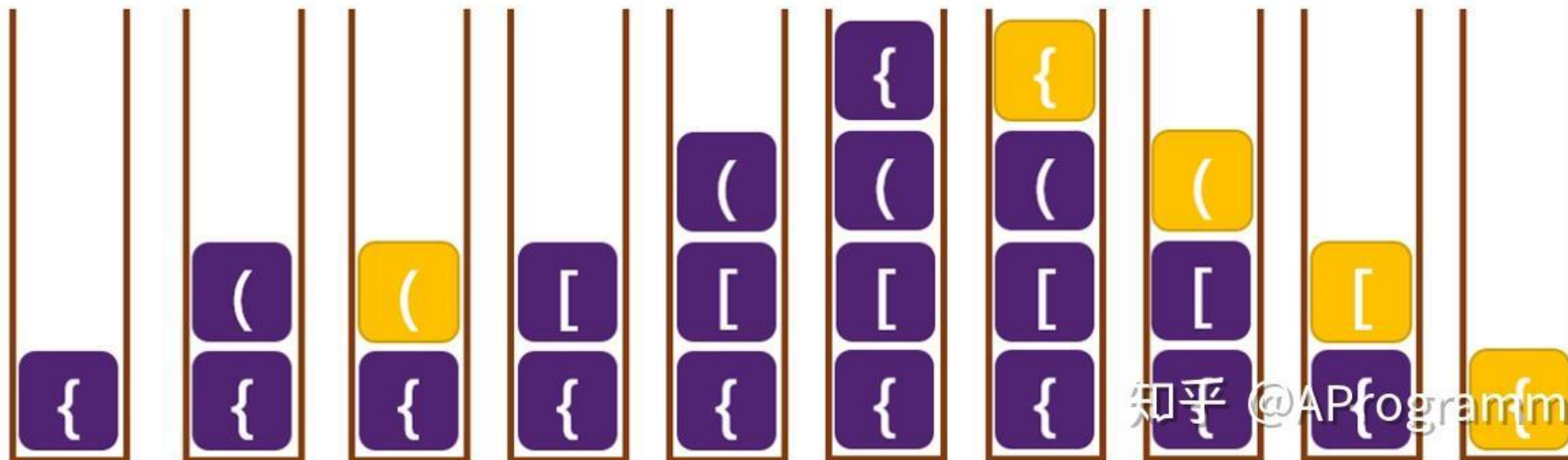
栈的应用示例：括号匹配

• 例：

序列一：{ () [({ })] } // 匹配

序列二：{ () [] ({ })] } // 失配

{ () [({ })] }



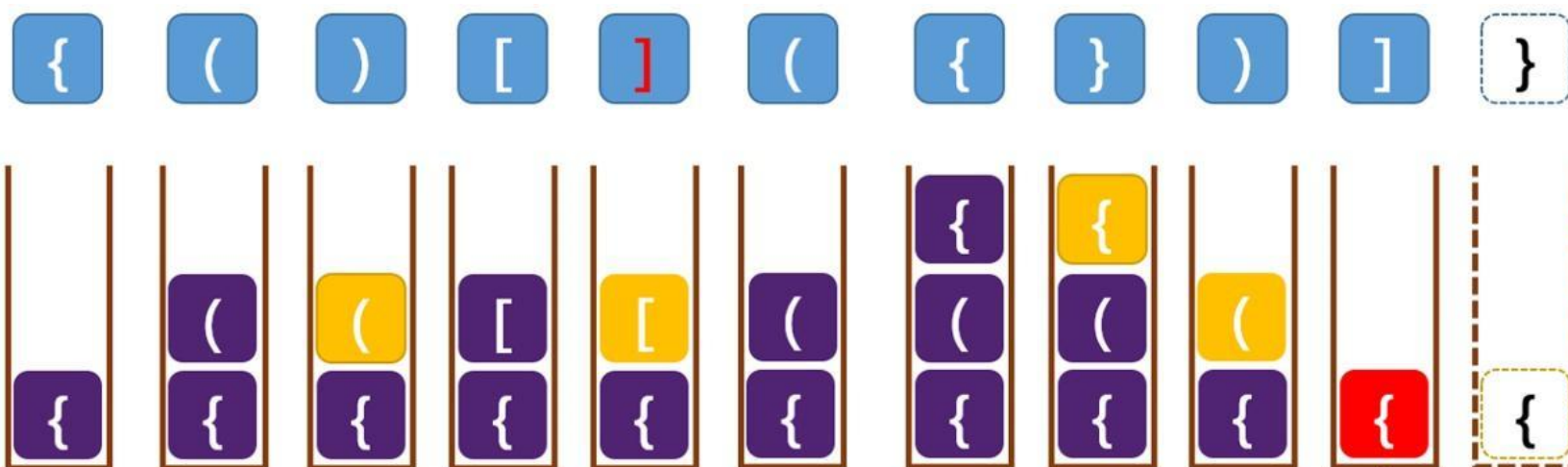
知乎 @APprogrammer

栈的应用示例：括号匹配

• 例：

序列一：{ () [({ })] } // 匹配

序列二：{ () [] ({ })] } // 失配



这一步方括号与栈顶的花括号不匹配，程序会退出，不再检查后面的字符

栈的应用示例：括号匹配

总结括号匹配的过程如下：

初始化栈InitStack(S)

从第一个到最后一个遍历所有括号

当前括号是左括号or右括号？

if 左括号：进栈Push(S,x)

else 右括号：当前括号与栈顶括号是否匹配？Gettop(S)

if 匹配：栈顶元素出栈Pop(S)

else 不匹配：括号序列失配，退出程序

栈是否为空？Empty(S)

if 栈空：括号序列完全匹配，退出程序

else 栈非空：括号序列失配，退出程序

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

栈的应用示例：括号匹配

```
# 输入：字符串 str (包含待检查的括号序列)
# 输出：逻辑值, TRUE 表示括号匹配, FALSE 表示不匹配
# (1) 初始化一个空栈 s
initstack(s)
# (2) 遍历字符串中的每一个字符 char
for (char in chars) {
  # 情况一：遇到左括号，直接入栈
  if (char == "(" || char == "[" || char == "{") {
    push(s, char) # 将左括号压入栈中
  }
  # 情况二：遇到右括号
  else if (char == ")" || char == "]" || char == "}") {
    if (empty(s)) {
      return(FALSE) # 直接返回不匹配
    }
    top_char <- pop(s) # 弹出栈顶元素
    # 检查弹出的左括号是否与当前右括号匹配
    if (!((top_char == "(" && char == ")") ||
        (top_char == "[" && char == "]") ||
        (top_char == "{" && char == "}"))) {
      return(FALSE) # 类型不匹配，返回不匹配
    }
    # 如果匹配，继续循环
  }
}
# 注意：非括号字符在此版本中被忽略，循环会自动进入下一次迭代
}
```

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例



栈的应用示例：括号匹配

```
# 输入：字符串 str (包含待检查的括号序列)
# 输出：逻辑值, TRUE 表示括号匹配, FALSE 表示不匹配
# (3) 字符串遍历完毕, 检查栈是否为空
if (empty(s)) {
    return(TRUE)    # 所有括号都匹配了
} else {
    return(FALSE)   # 还有多余的左括号未匹配
}
```

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

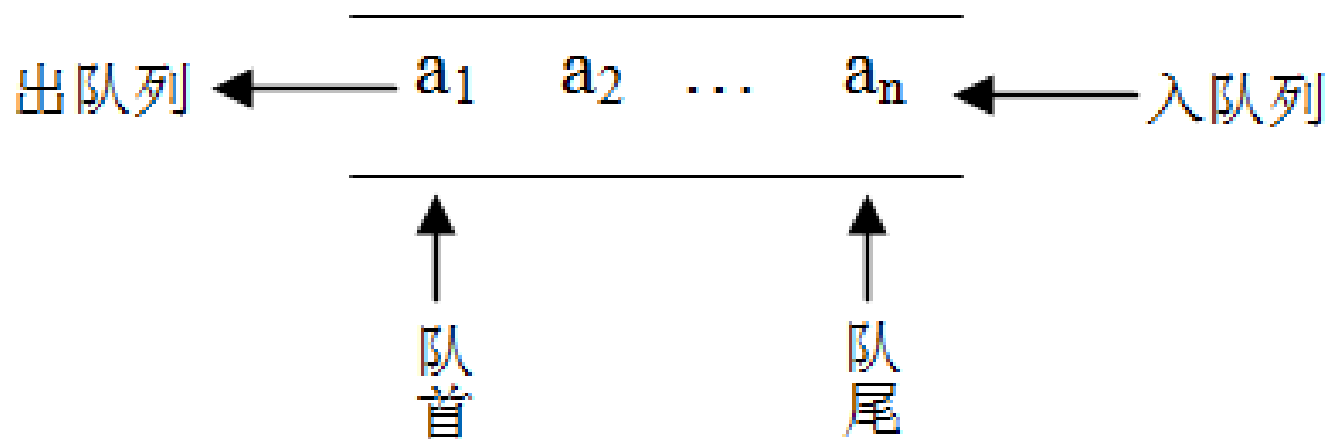
4.4 图结构

二、实例

4.5 应用举例

队列

- 队列 (Queue) 是一种先进先出 (First In Last Out, FILO) 的线性表。只能在表的一端进行插入操作, 在另一端进行删除操作。
- 允许插入的一端称为队尾 (Rear), 允许删除的一端称为队首 (Front)。





队列的基本操作

- (1) InitQueue(Q): 初始化操作, 设置一个空的队列Q。
- (2) EntQueue(Q,x): 入队操作, 将元素x插入到队列Q的尾部, 成为队列新的队尾元素。
- (3) DelQueue(Q): 出队列函数, 若队列Q不为空, 返回Q的队首元素, 且将队首元素从队列中删除。如果队列Q为空, 则函数返回值为NULL。
- (4) GetHead(Q): 取队首元素, 若队列Q不为空, 返回Q的队首元素。如果队列为空, 则函数返回值为NULL。
- (5) CurrentSize(Q): 队列大小函数, 返回队列Q当前所包含的元素个数。

https://youtu.be/HdTcc4err_I?si=IDWHTvLcOYrT6q62

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例



4.3 树形结构

- 层析关系
- 树形结构
- 二叉树及其性质
- 常用操作及算法

知识要点

层次关系，
树形结构，
树，树根，子树，
结点的度，
叶子（终端结点），
分支结点，
树的度，
孩子，
双亲，
兄弟，
祖先，
子孙，
树的深度，
二叉树，满二叉树，完全二叉树，
二叉树的遍历。

层次关系

- 在我们的实际生活中，有许多数据之间的关系表现为层次关系，例如，人类社会的家族族谱、各种社会组织机构的设置、一本书的章节目录，问题决策、博弈等等，
- 数据之间的关系都是一种层次关系，可以用树形结构来描述和建模。树形结构是一种用于描述层次关系的数据结构。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例



博弈与决策问题

- 博弈
 - 狭义的讲，博弈即下棋，对弈。
 - 现代意义，泛指在一定的游戏规则约束下，基于直接相互作用的环境条件，各参与者依靠所掌握的信息，选择各自策略（行动），以实现利益最大化和风险成本最小化的过程。
- 只要有涉及人群的互动，就有博弈。人与人之间的博弈，组织之间的博弈，国家之间的博弈。
- 博弈和决策是两类常见的问题，对这类问题的求解过程可以用图形来描述，这种图形就像一棵自然界中的树，称为博弈树和决策树。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

博弈树 (Game Tree)

- 博弈的过程通常可以用一颗树来描述，即博弈树 (Game Tree)，它描述了博弈的过程和可能的结果。
- 井字游戏
 - 两个玩家，一个打圈O，一个打叉×，轮流在 3×3 的格上打自己的符号，
 - 最先以横、直、斜连成一线为胜。

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

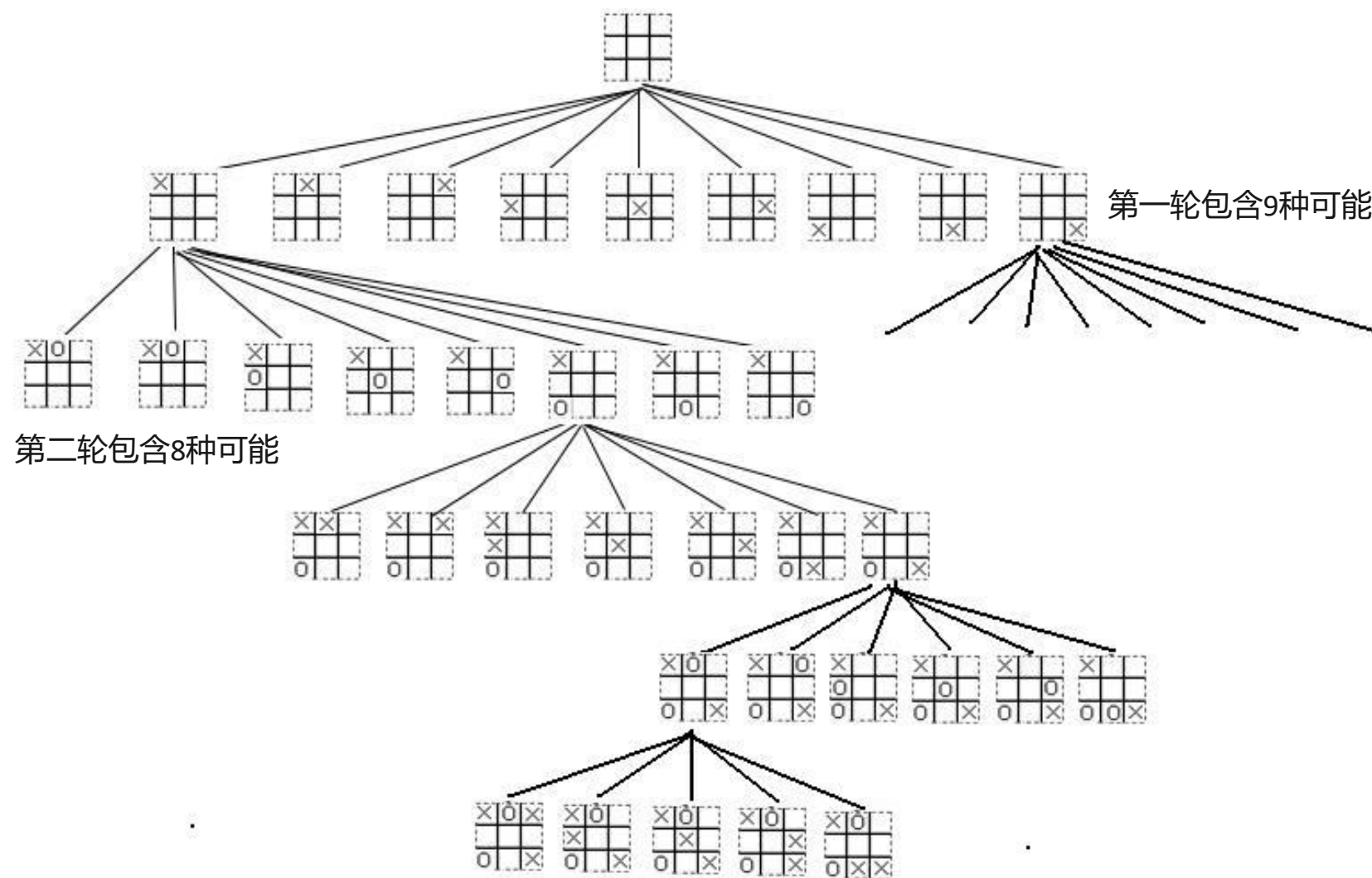
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

“井字棋” 博弈树示例



“井字棋” 博弈结果分析

- 井字棋，有9个格，树的高度最高为9层，第一层有9种选择，第二层有 $9 \times 8 = 72$ 中选择，第三层有 $9 \times 8 \times 7 = 504$ 种选择，依次类推，则第八层有 $9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 = 362\,880$ 种走法。最后一步，只剩一个格，由先手 $\times$ 占据，可见最终结果是362880种状态。
- 在全部362880种状态中，有大量的状态是中间就已经确定了胜负，无需进一步往下走，即对应的后续走法可以“剪”枝。这样，当获胜导致游戏结束时，则只剩下255168种可能的棋局。若 $\times$ 是先手，则131184种棋局为 $\times$ 获胜，77904种棋局为O获胜，46080种棋局为平局。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

博弈问题的发展

• 人与机器的博弈

- 1951年, A.S. Douglas在EDSAC计算机上开发出OXO (井字游戏) , 开启了人与机器对弈的时代。
- 1996年2月10日, 美国IBM的深蓝计算机首次挑战国际象棋世界冠军卡斯巴罗夫, 但以2-4落败, 比赛在2月17日结束。
- 1997年5月再度挑战卡斯巴罗夫, 比赛在5月11日结束, 最终深蓝电脑以3.5-2.5击败卡斯巴罗夫

• 博弈论 (Game Theory)

- 研究在多决策主体之间行为具有相互作用时, 各主体根据所掌握信息及对自身能力的认知, 做出有利于自己的决策的一种行为理论。
- 在经济学、管理科学、国际关系、政治学、军事战略、计算机科学和其他很多学科都有广泛的应用。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

决策问题与决策树

• 决策问题

- 在处理问题时，常常会面临几种可能出现的自然情况，同时又存在着几种可供选择的行动方案。此时，需要决策者根据当前状态和已知信息作决策，即选择出一种行动方案
- 决策问题通常还涉及到各种情况发生的概率，在此基础上进行计算和推演。
- 决策问题的求解过程可以很自然的用树状图形来描述，这就是决策树（Decision Tree）。它是一种直观的运用概率，进行可行性决策分析的图解方法，常用于项目风险评价等领域。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

决策树举例

某企业准备投资一个新的项目，市场预测表明：产品销路好的概率为0.7；

销路差的概率为0.3。有三种可选方案：

- (1) 建设大型工厂，需要投资1000万元，可使用10年。若产品销路好，每年可赢利200万元；若销路不好，每年将亏损50万元。
- (2) 建设小型工厂，需投资300万元。如销路好，每年可赢利70万元；如销路不好，每年也会赢利50万元。
- (3) 先建设小型工厂，若销路好，3年后扩建，扩建需投资500万元，可使用7年，扩建后每年会赢利180万元。

针对上述三种可选方案，企业如何选择？

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

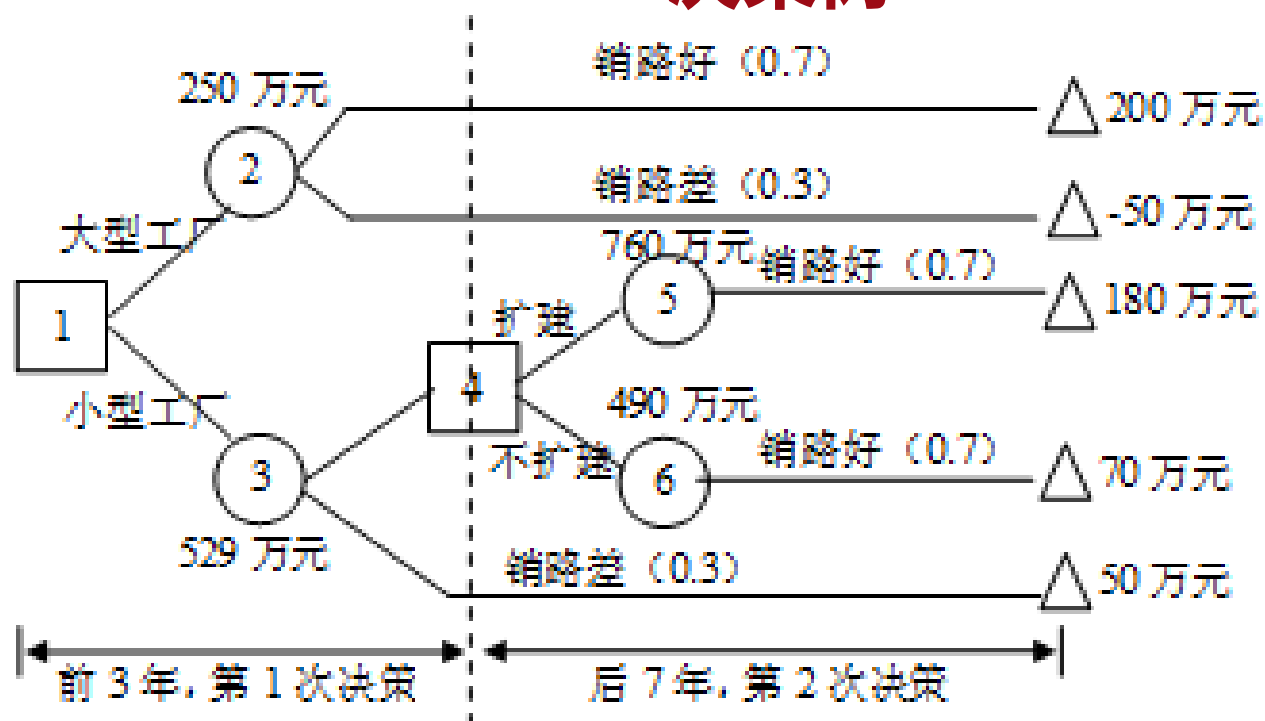
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

决策树



- 计算各状态点的利润期望值
- 点②: $0.7 \times 200 \times 10 + 0.3 \times (-50) \times 10 - 1000$ (大厂投资) = 250 (万元)
- 点⑤: $1.0 \times 180 \times 7 - 500$ (扩建投资) = 760 (万元)
- 点⑥: $1.0 \times 70 \times 7 = 490$ (万元)

树形结构

- 树 (tree) 是 n ($n \geq 0$) 个结点的有限集。在一棵非空树中：
 - (1) 有且仅有一个特殊的结点，称为树根 (Root)；
 - (2) 当 $n > 1$ 时，其余结点可分为 m ($m > 0$) 个互不相交的有限集 $T_1, T_2, \dots, T_m$ ，且其中的每一个集合也是一棵树。 $T_1, T_2, \dots, T_m$ 称为根的子树。
- 层次关系的抽象，用于表达数据间的层次关系，是一类重要的非线性数据结构。也可表示从属关系、并列关系。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

树的表示--1树形结构

• 树形结构

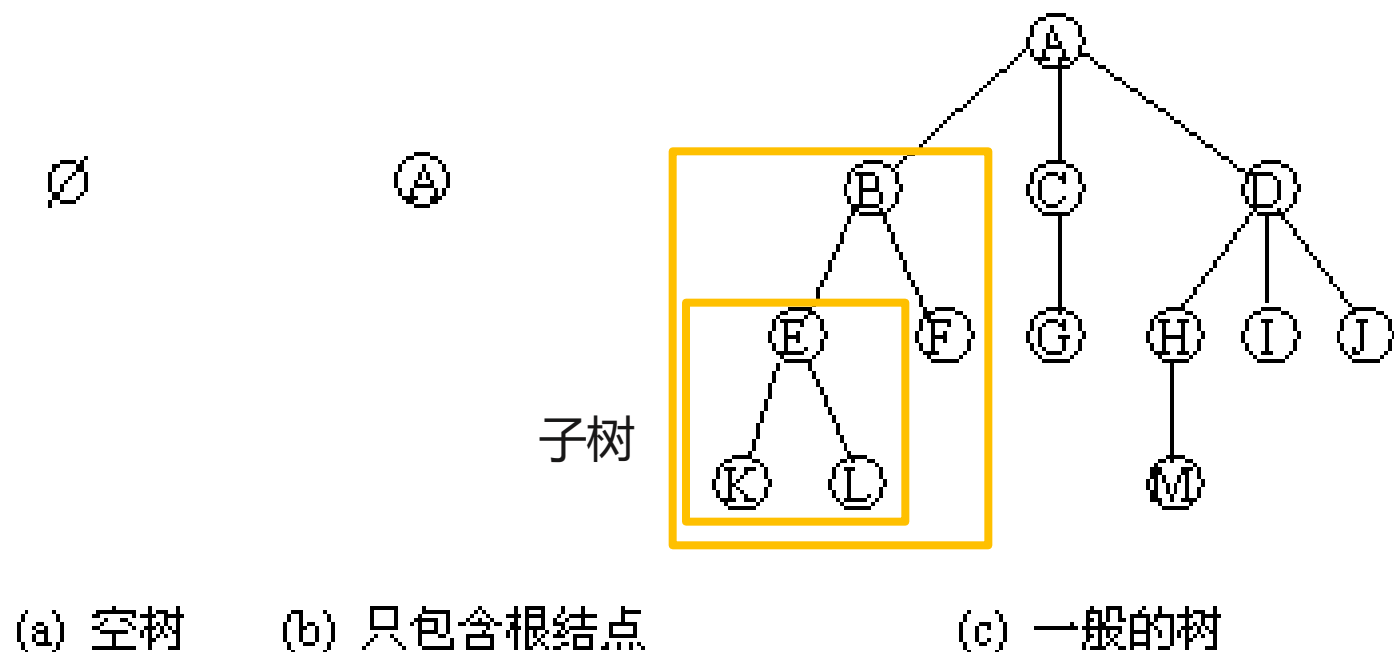
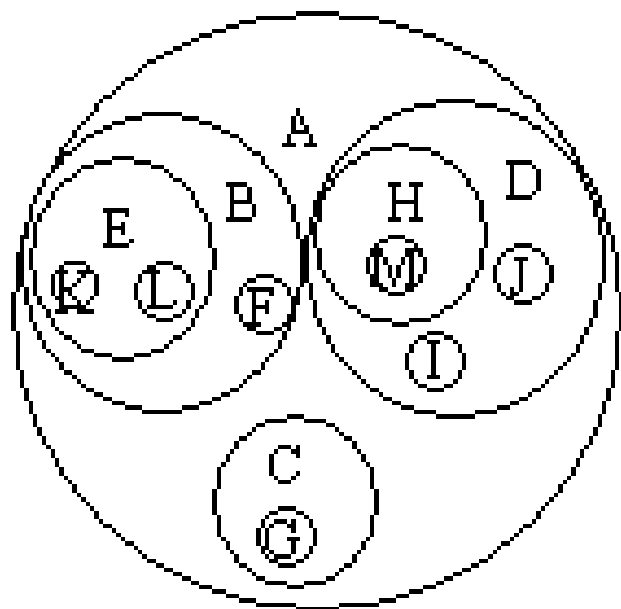


图 4-5 一般的树形结构

树的表示—2

- 嵌套集合表示法
- 凹入表示法



```

1 A
  2 B
    3 E
      4 K
      4 L
    3 F
  2 C
    3 G
  2 D
    3 H
      4 M
    3 I
    3 J
  
```

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

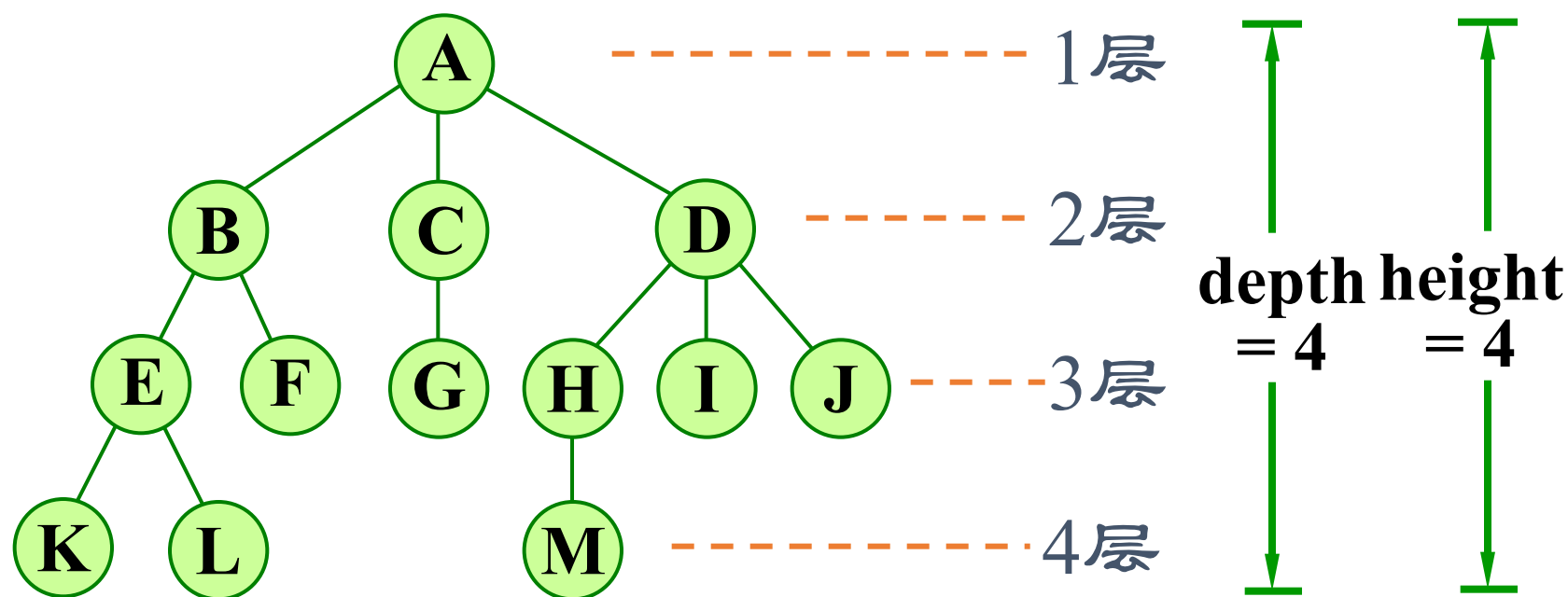
4.4 图结构

二、实例

4.5 应用举例

树的基本术语

- 子女：若结点的子树非空，结点子树的根即为该结点的子女。
- 双亲：若结点有子女，该结点是子女双亲。



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

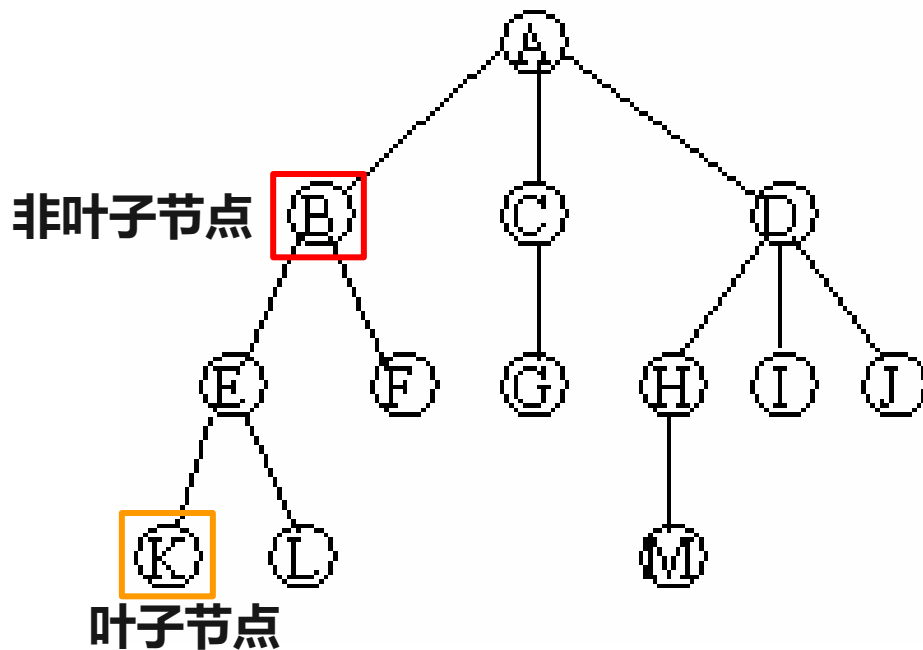
4.4 图结构

二、实例

4.5 应用举例

树的基本术语

- 兄弟：同一结点的子女互称为兄弟。
- 度：结点的子女个数即为该结点的度；树中各个结点的度的最大值称为树的度。
- 分支结点：度不为0的结点即为分支结点，亦称为非终端结点。
- 叶结点：度为0的结点即为叶结点，亦称为终端结点。
- 祖先：某结点到根结点的路径上的各个结点都是该结点的祖先。
- 子孙：某结点的所有下属结点，都是该结点的子孙。



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

树的基本术语

结点的层次：规定根结点在第一层，其子女结点的层次等于它的层次加一。以下类推。

深度：结点的深度即为结点的层次；离根最远结点的层次即为树的深度。

高度：规定叶结点的高度为1，其双亲结点的高度等于它的高度加一。

树的高度：等于根结点的高度，即根结点所有子女高度的最大值加一。

有序树：树中结点的各棵子树 $T_0, T_1, \dots$ 是有次序的，即为有序树。

无序树：树中结点的各棵子树之间的次序是不重要的，可以互相交换位置。

森林：森林是 m ($m \geq 0$) 棵树的集合。

【树 - 定义, 术语, 性质】

https://www.bilibili.com/video/BV1xeV5zYEuZ/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

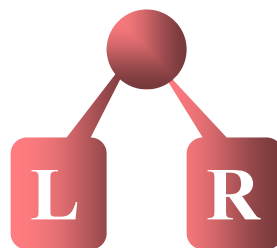
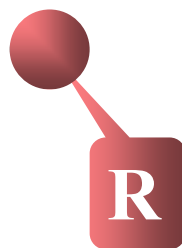
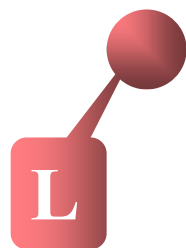
4.4 图结构

二、实例

4.5 应用举例

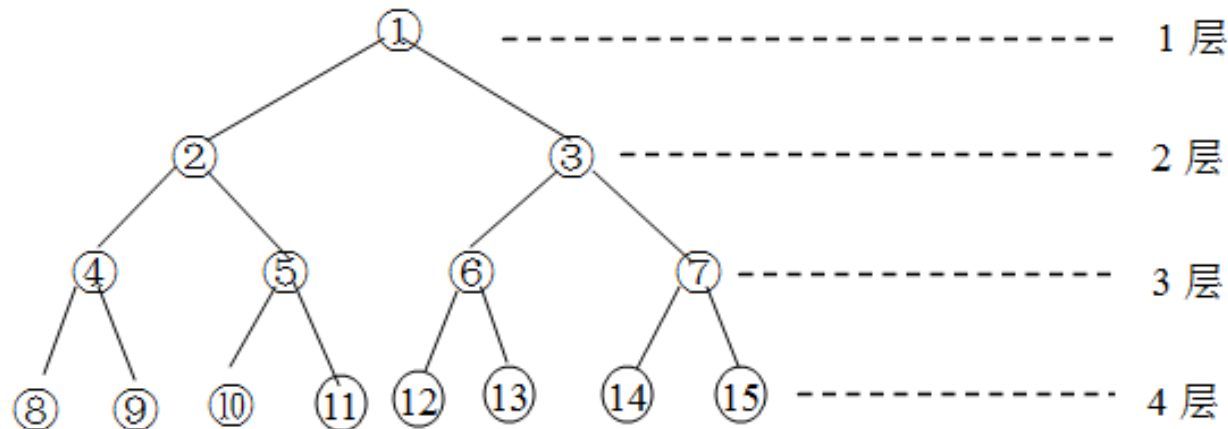
二叉树

- 二叉树 (Binary tree) 是 n ($n \geq 0$) 个结点的有限集合, 或者是空集, 或者是由一个根和称为左右子树的两个不相交的二叉树构成。
- 二叉树和树是两个不同的概念
 - 树的子树没有顺序, 二叉树的两棵子树有左右之分。
 - 树中结点的度是任意的, 二叉树中每个结点的度不能大于2。
- 二叉树共有5种不同的形态



二叉树的特性

- 性质1** 若二叉树结点的层次从1开始,则在二叉树的第*i*层最多有 2^{i-1} 个结点。 ($i \geq 1$)



- 证明 (数学归纳法)**
 - 当*i*=1时, 只有一个根结点。显然, $2^{i-1}=2^{1-1}=1$, 命题成立。
 - 假定对所有的 $1 \leq j < i$ 命题成立, 即在*j*层的结点数最多为 2^{j-1} 。
 - 由归纳假设知, 第*i*-1层上结点的最大数目为 2^{i-2} 。由于二叉树中每个结点的度最大为2, 因此, 第*i*层上结点的最大数目为第*i*-1层上结点数目的2倍, 即为 $2 \times 2^{i-2} = 2^{i-1}$ 。

二叉树的特性

- **性质2** 深度为 k 的二叉树最少有 k 个结点，最多有 2^k-1 个结点。 ($k \geq 1$)

- 证明

因为每一层最少要有1个结点，因此，最少结点数为 k 。最多结点个数借助性质1：用求等比级数前 k 项和的公式

$$2^0 + 2^1 + 2^2 + \dots + 2^{k-1} = 2^k - 1$$

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

二叉树的特性

· **性质3** 对任何一棵二叉树，如果其叶结点有 n_0 个，度为 2 的非叶结点有 n_2 个，则有

$$n_0 = n_2 + 1$$

设二叉树 T 中度为 1 的结点数为 n_1 ，由于二叉树中所有结点的度均小于等于 2，因此二叉树中总的结点个数为：

$$n = n_0 + n_1 + n_2 \quad (4.1)$$

在二叉树中，每一个结点引出的分支数目为该结点的度，因此总的分枝数为：

$$B = n_1 + 2 n_2 \quad (4.2)$$

另外，在一棵二叉树中，除了根结点以外，每一个结点都是由一个分支引出，因此有

$$B = n - 1 \quad (4.3)$$

根据 (4.2) 和 (4.3)， $n_1 + 2 n_2 = n - 1$ ， $n = n_1 + 2 n_2 + 1$ 。

代入 (4.1) 得

$n_1 + 2 n_2 + 1 = n_0 + n_1 + n_2$ ，得 $n_0 = n_2 + 1$ 。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

二叉树的特性

· **性质3** 对任何一棵二叉树，如果其叶结点有 n_0 个，度为 2 的非叶结点有 n_2 个，则有

$$n_0 = n_2 + 1$$

例题：某二叉树共有 399 个结点，其中有 199 个度为 2 的结点，则该二叉树中的叶子结点数为()

A、不存在这样的二叉树

B、200

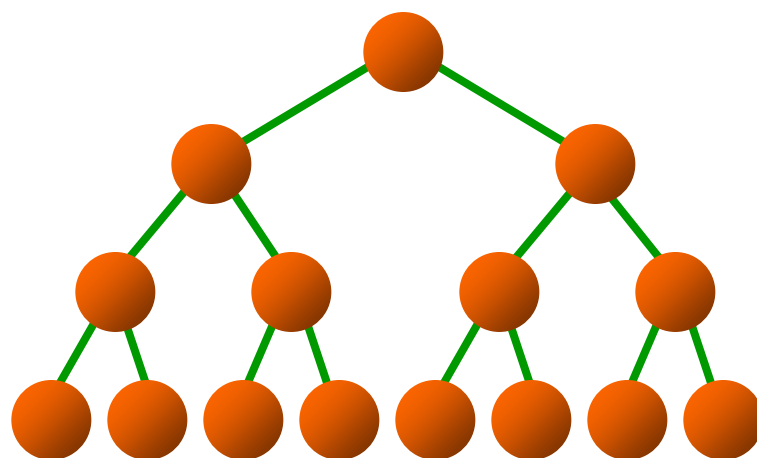
C、198

D、199

$n_2 = 199, n_0 = n_2 + 1 = 200$ ，因此选B

满二叉树

- 满二叉树：深度为 k 、且结点个数恰好为 2^k-1 的二叉树



深度为4的满二叉树

完全二叉树

- 完全二叉树，若设二叉树的深度为 k ，则共有 k 层。除第 k 层外，其它各层 ($1 \sim k-1$) 的结点数都达到最大个数，第 k 层从右向左连续缺若干结点，这就是完全二叉树。

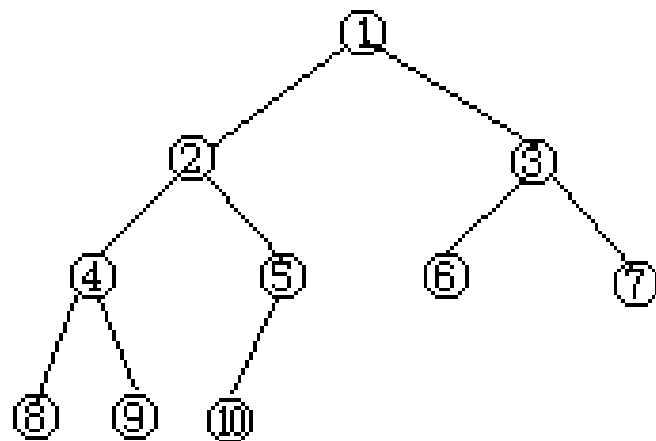


图 4-9 深度为 4 的完全二叉树

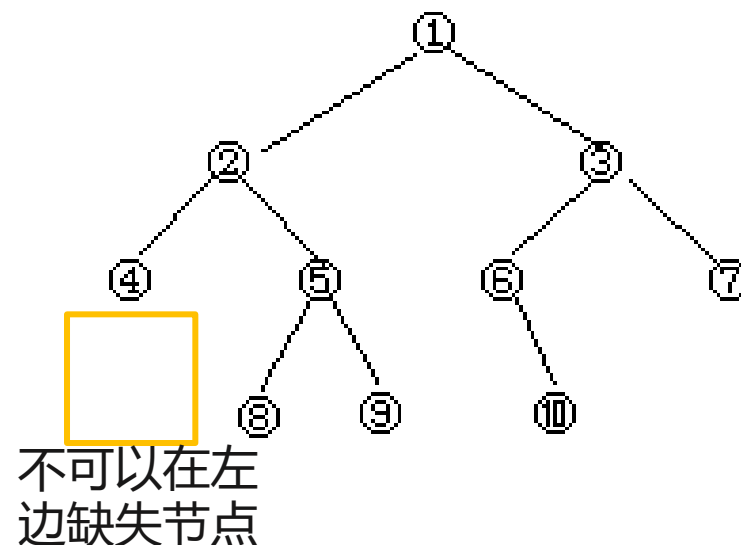
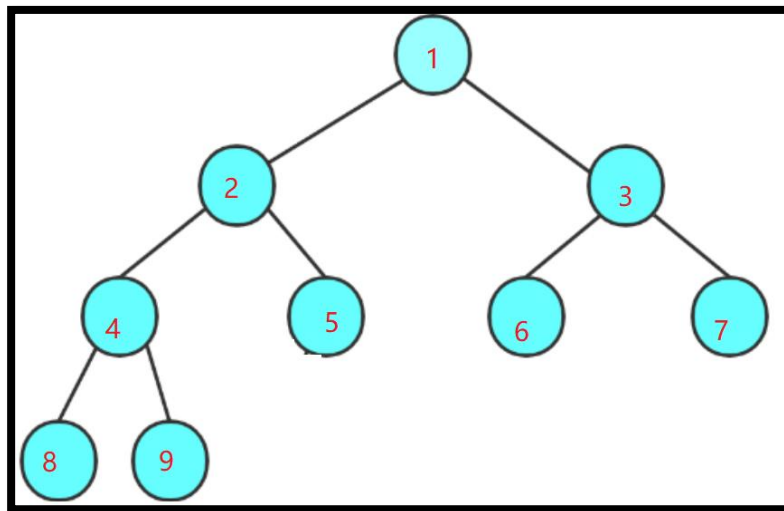
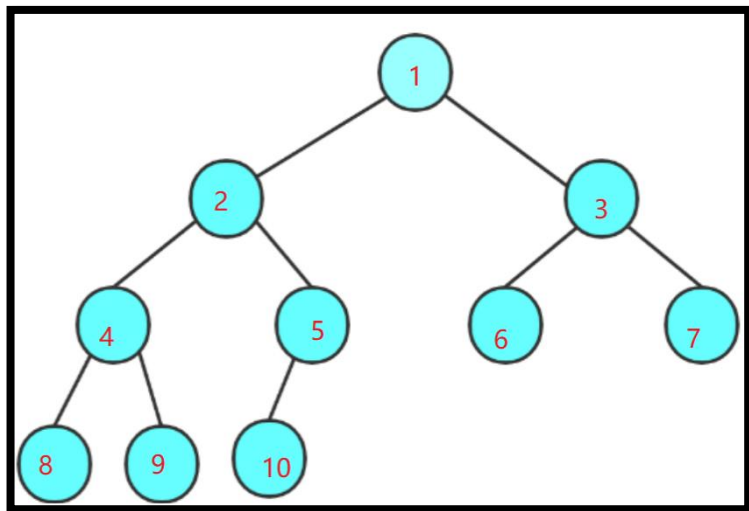


图 4-10 深度为 4 的非完全二叉树

完全二叉树

例题：在具有 $2n$ 个结点的完全二叉树中，叶子结点个数
为0。

- A、 n
- B、 $n+1$
- C、 $n-1$
- D、 $n/2$



已知总节点数 $2n$ ，完全二叉树的节点数是偶数，说明
最后一层有1个度为1的叶节点，总节点数
 $2n = n_0 + n_1 + n_2$ ， $n_1 = 1$ ， $n_0 = n_2 + 1$ ，因此
 $2n = n_0 + 1 + n_0 - 1$ ，可得 $n_0 = n$ ，因此选A

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

完全二叉树

例题：一个具有767个节点的完全二叉树，其叶子节点个数为（ ）

A、383

B、384

C、385

D、386

与上题类似，总节点数是奇数，说明没有度为1的叶节点，总节点数 $767=n_0+n_2$ ， $n_0=n_2+1$ ，因此 $767=n_0+n_0-1$ ，可得 $n_0=768/2=384$ ，因此选B

二叉树的特性

· **性质4** 具有 n ($n \geq 0$) 个结点的完全二叉树的深度为 $\lceil \log_2(n+1) \rceil$

设完全二叉树的深度为 k , 则有

$$\underbrace{2^{k-1}-1} < n \leq \underbrace{2^k-1}$$

上面 $k-1$ 层结点数 包括第 k 层的最大结点数

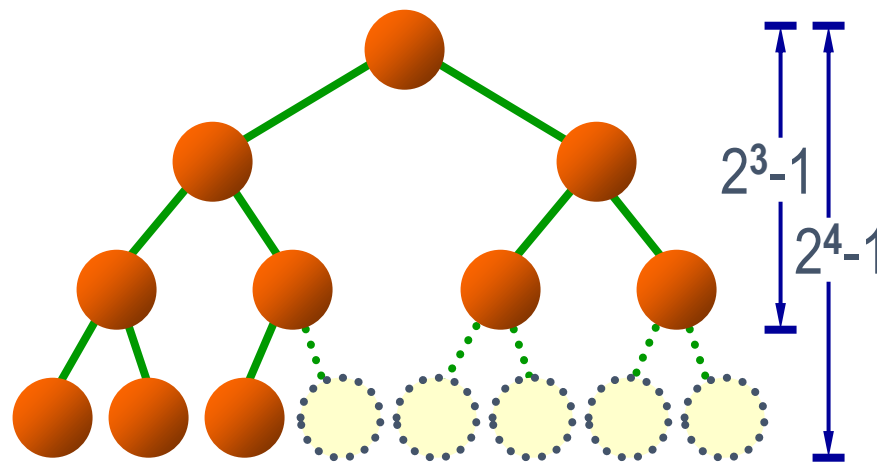
变形 $2^{k-1} < n+1 \leq 2^k$

取对数

$$k-1 < \log_2(n+1) \leq k$$

有

$$\lceil \log_2(n+1) \rceil = k$$



二叉树的特性

- **性质4** 具有 n ($n \geq 0$) 个结点的完全二叉树的深度为 $\lceil \log_2(n+1) \rceil$

例题：一个具有531个节点的完全二叉树高度是？

A、 11

B、 10

C、 8

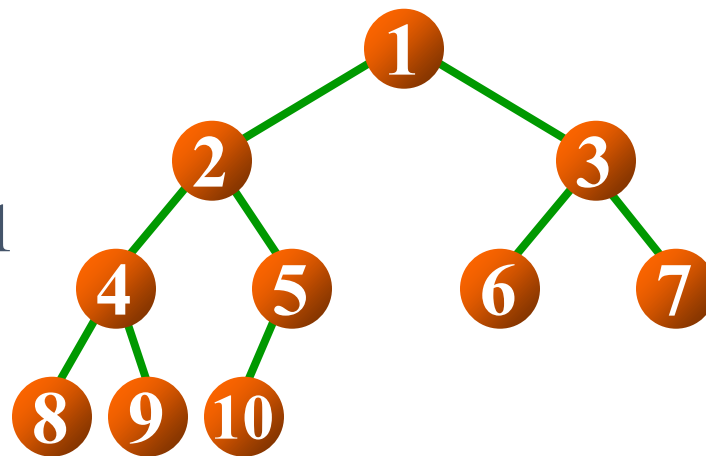
D、 12

代入公式 $\log_2 532 = 9.0552$ ，向上取整得到10，选B

二叉树的特性

性质5 如将一棵有 n 个结点的完全二叉树自顶向下，同一层自左向右连续给结点编号 $1, 2, \dots, n$ ，则有以下关系：

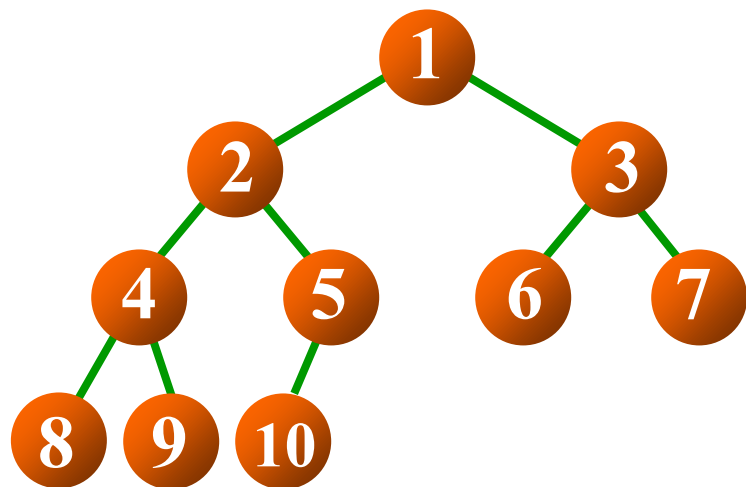
- ✓ 若 $i = 1$ ，则 i 无双亲
- ✓ 若 $i > 1$ ，则 i 的双亲为 $\lfloor i / 2 \rfloor$
- ✓ 若 $2*i \leq n$ ，则 i 的左子女为 $2*i$ ，
若 $2*i+1 \leq n$ ，则 i 的右子女为 $2*i+1$
- ✓ 若 i 为奇数，且 $i \neq 1$ ，
则其左兄弟为 $i-1$ ，若
- ✓ 若 i 为偶数，且 $i \neq n$ ，
则其右兄弟为 $i+1$



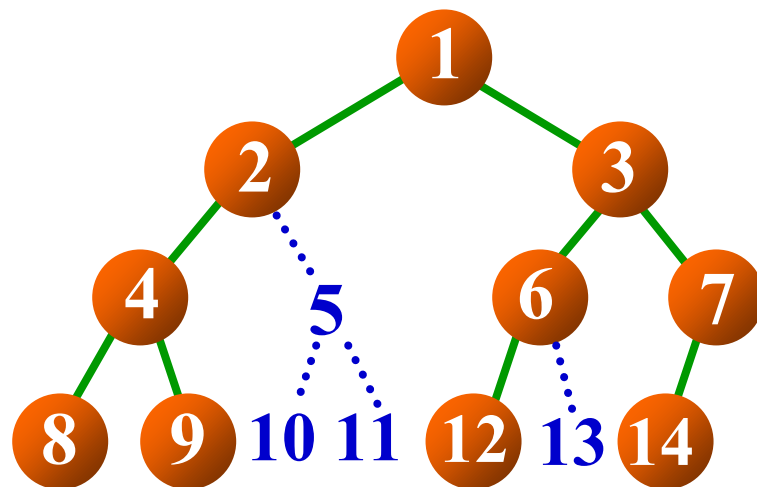
【二叉树&完全二叉树 - 定义, 性质】

https://www.bilibili.com/video/BV1eQ3RzxEoS/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

二叉树的顺序表示



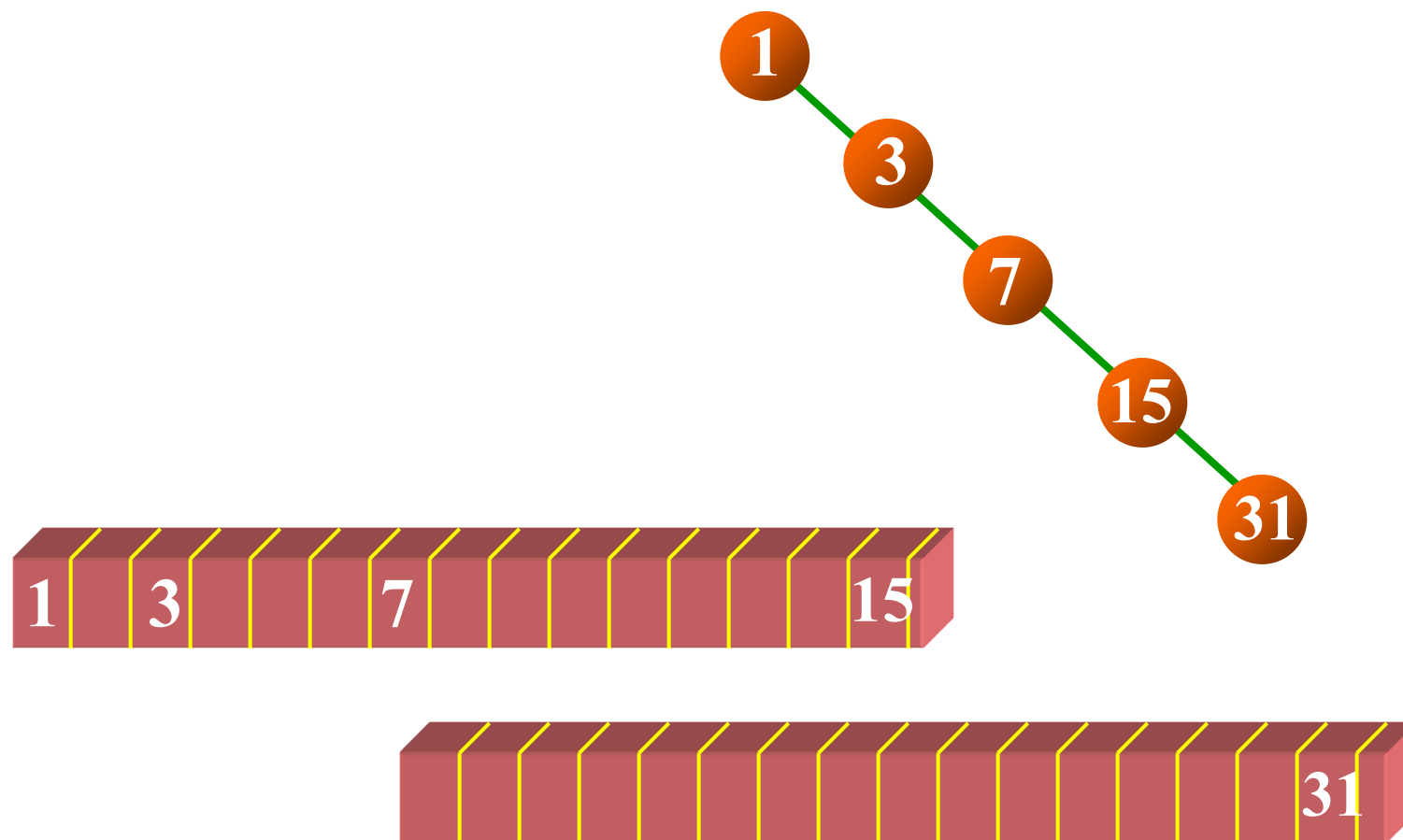
完全二叉树
的顺序表示



一般二叉树
的顺序表示

二叉树的顺序表示

极端情形：只有右单支的二叉树



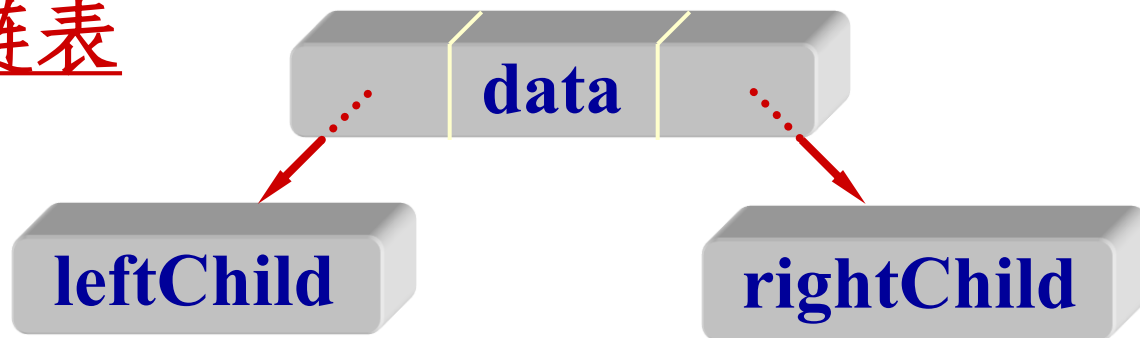
顺序存储在极端情况极浪费空间！

二叉树的链式存储

- 二叉树结点定义：每个结点有3个数据成员，**data**域存储结点数据，**leftChild**和**rightChild**分别存放指向左子女和右子女的指针。



二叉链表

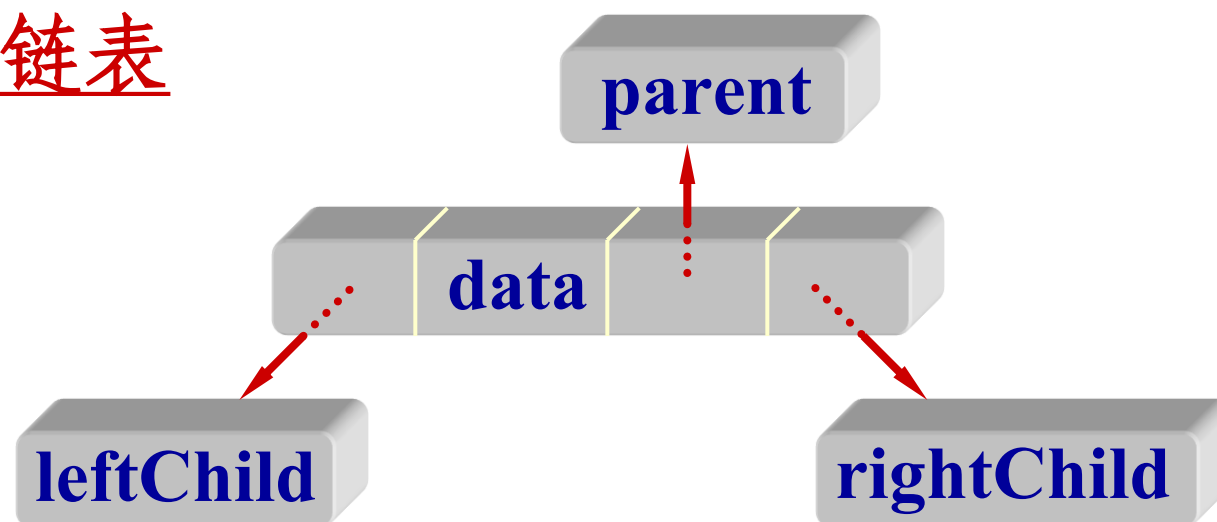


二叉树的链式存储

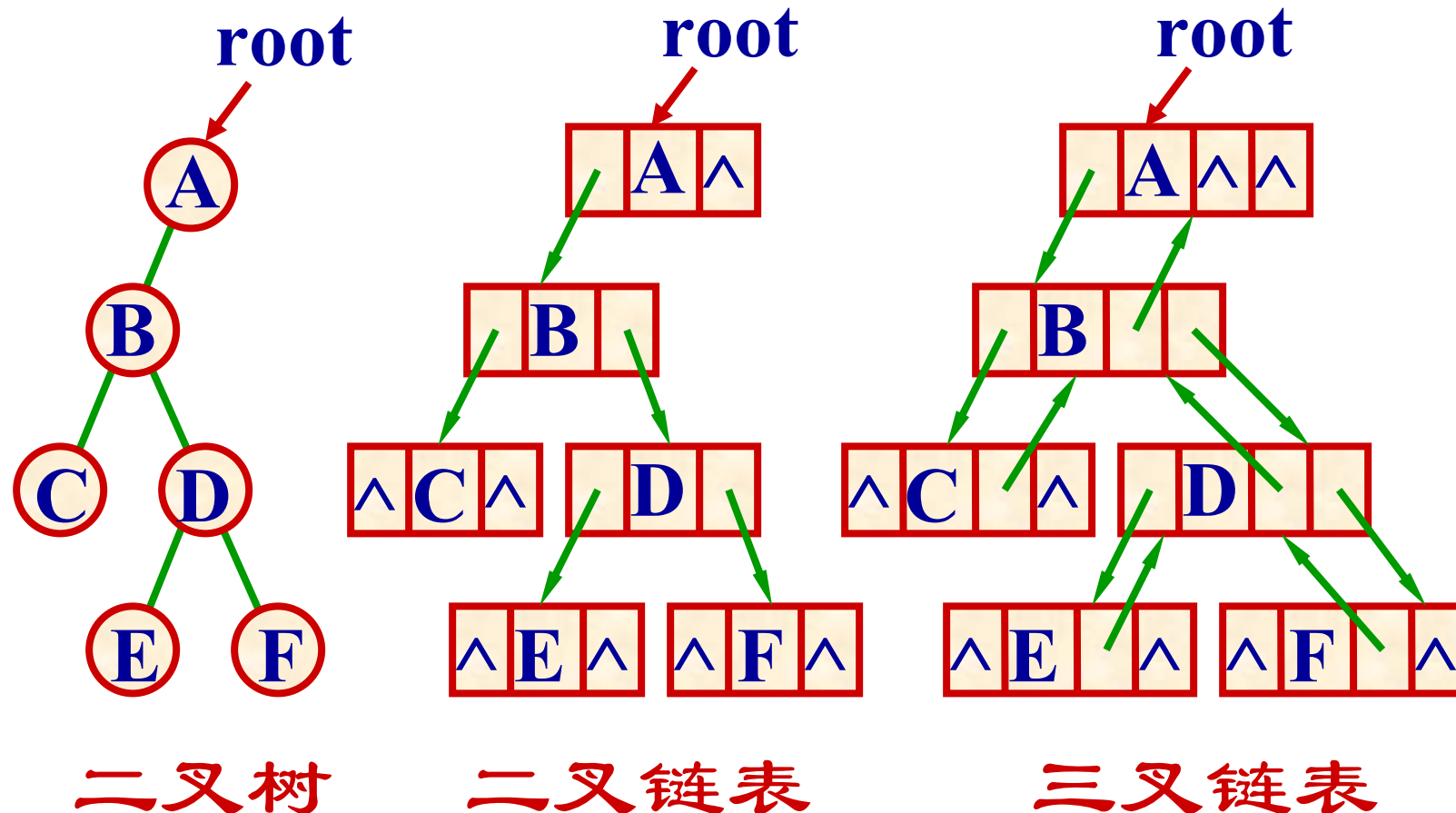
- 每个结点增加一个指向双亲的指针 **parent**, 使得查找双亲也很方便。



三叉链表



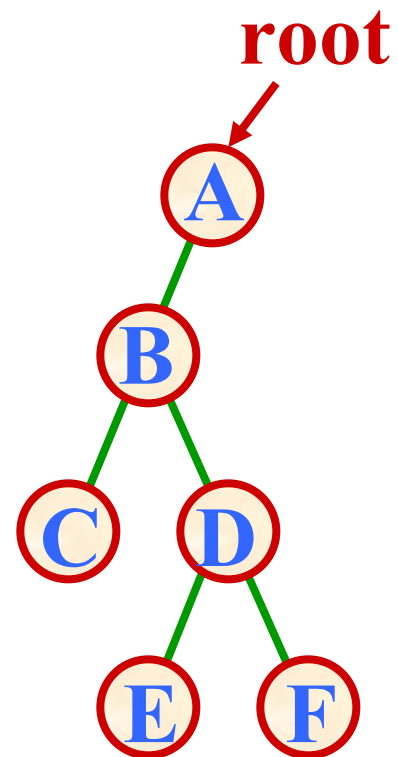
二叉树的链式存储



二叉树链表表示的示例

二叉树的链式存储

data parent leftChild rightChild



0	A	-1	1	-1
1	B	0	2	3
2	C	1	-1	-1
3	D	1	4	5
4	E	3	-1	-1
5	F	3	-1	-1

二叉链表的静态结构

二叉树的遍历

- 二叉树的遍历就是按某种次序访问树中的结点，要求每个结点访问一次且仅访问一次。

- 设访问根结点记作 V

遍历根的左子树记作 L

遍历根的右子树记作 R

- 则可能的遍历次序有

前序	VLR	镜像	VRL
----	-----	----	-----

中序	LVR	镜像	RVL
----	-----	----	-----

后序	LRV	镜像	RLV
----	-----	----	-----

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

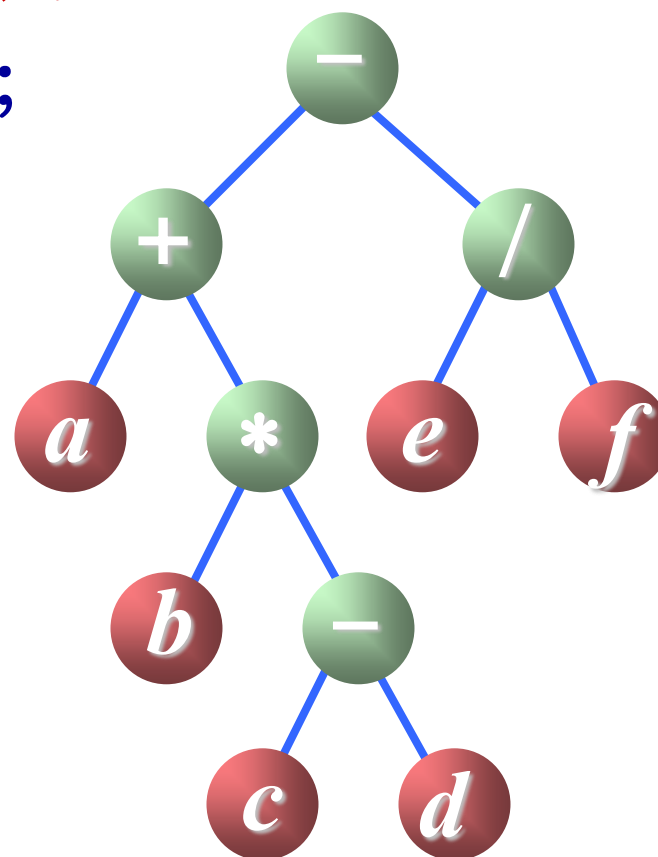
二叉树的中序遍历 (Inorder Traversal)

中序遍历二叉树算法的框架是：

- 若二叉树为空，则空操作；
- 否则
 - ◆ 中序遍历左子树 (L)；
 - ◆ 访问根结点 (V)；
 - ◆ 中序遍历右子树 (R)。

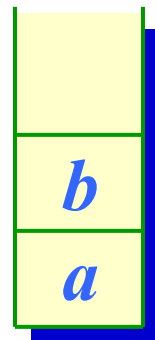
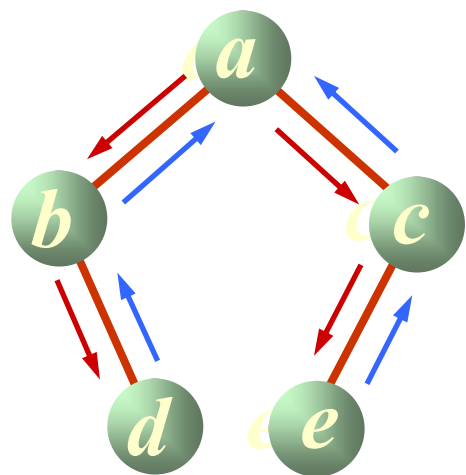
遍历结果

$$a + b * c - d - e / f$$

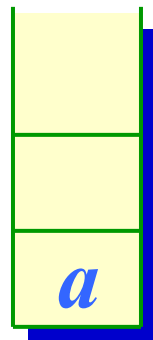


二叉树的中序遍历 (Inorder Traversal)

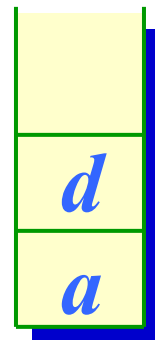
利用栈的中序遍历



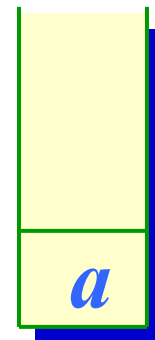
左空



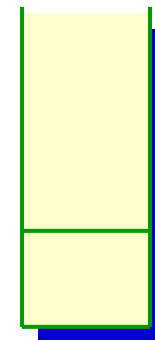
退栈
访问



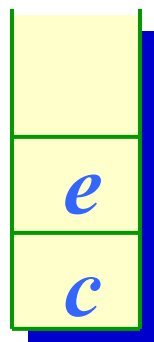
左空



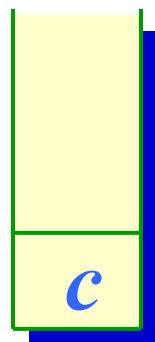
退栈
访问



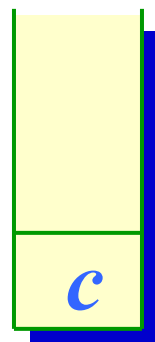
退栈
访问



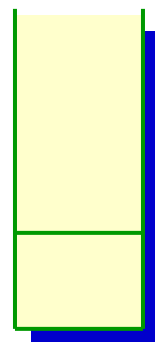
左空



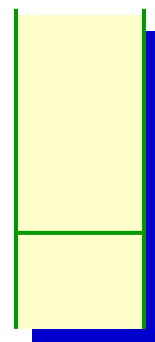
退栈访问



右空



退栈访问



栈空结束

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

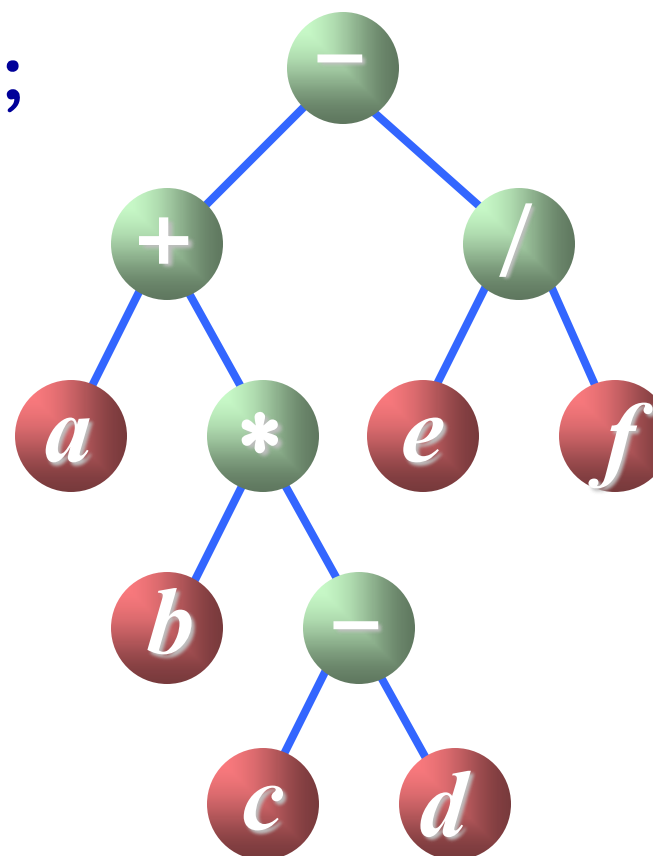
二叉树的前序遍历 (Preorder Traversal)

前序遍历二叉树算法的框架是：

- 若二叉树为空，则空操作；
- 否则
 - ◆ 访问根结点 (V)；
 - ◆ 前序遍历左子树 (L)；
 - ◆ 前序遍历右子树 (R)。

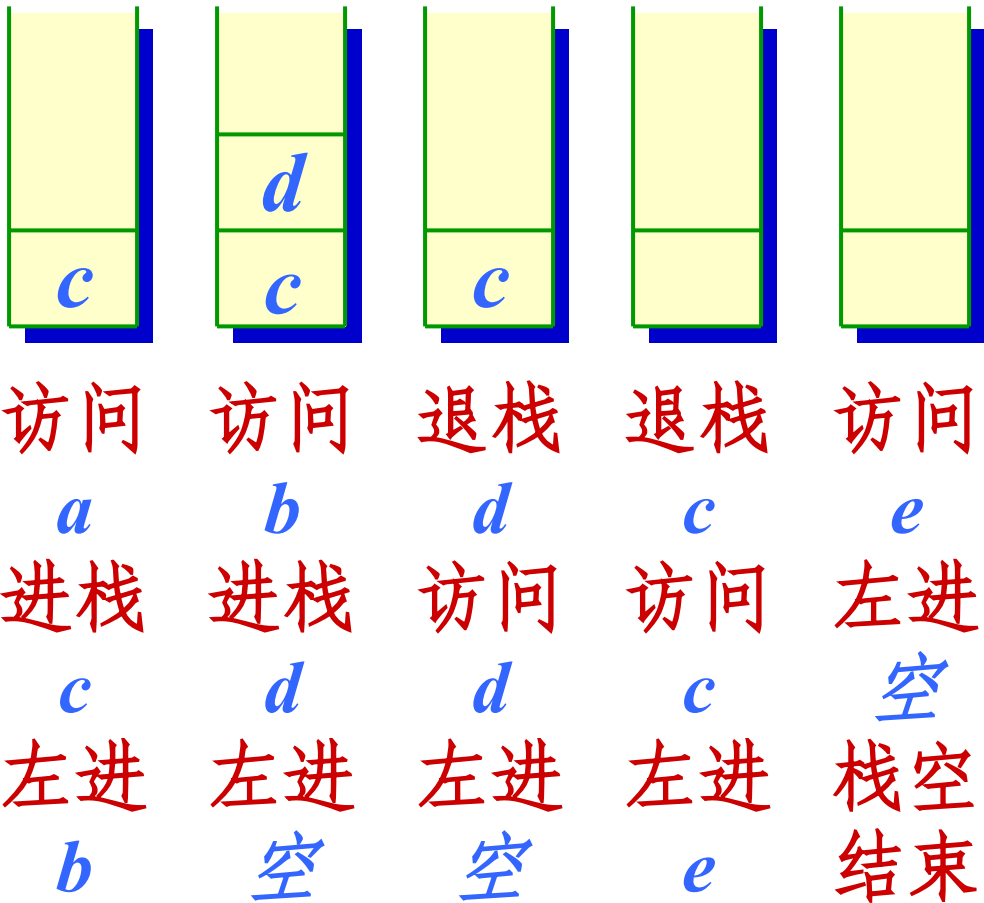
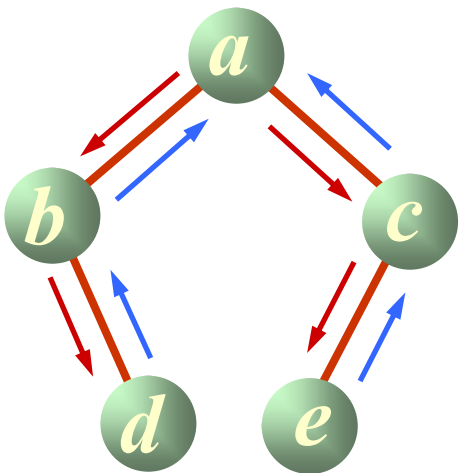
遍历结果

$- + a * b - c d / e f$



二叉树的前序遍历 (Preorder Traversal)

利用栈的前序遍历



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

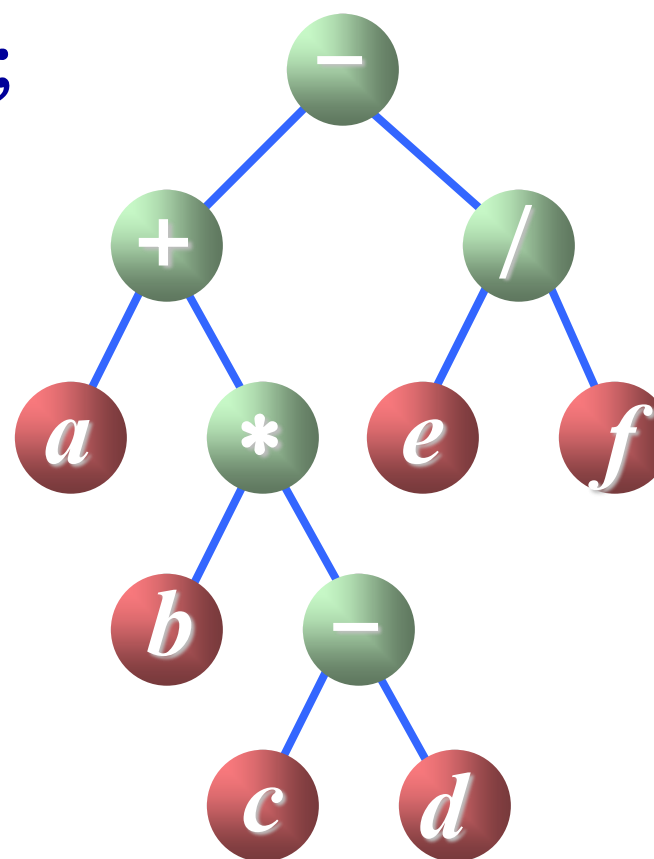
二叉树的后序遍历 (Postorder Traversal)

后序遍历二叉树算法的框架是：

- 若二叉树为空，则空操作；
- 否则
 - ◆ 后序遍历左子树 (L)；
 - ◆ 后序遍历右子树 (R)；
 - ◆ 访问根结点 (V)。

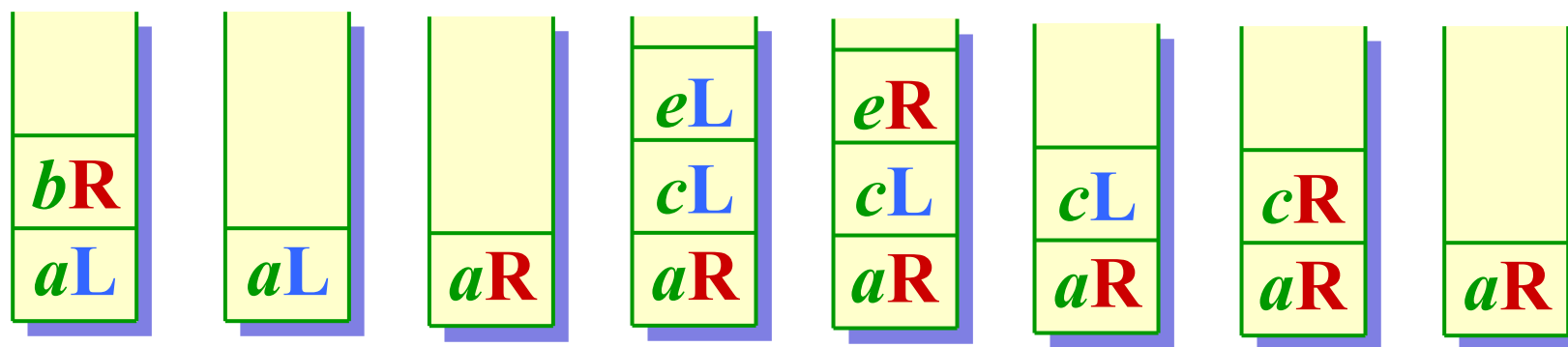
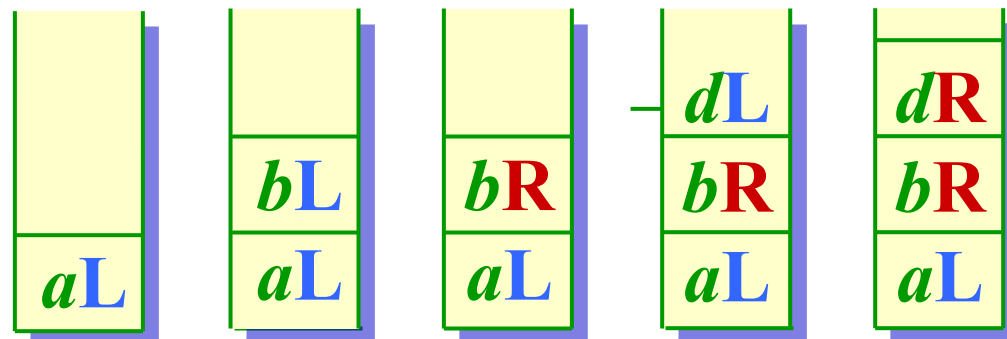
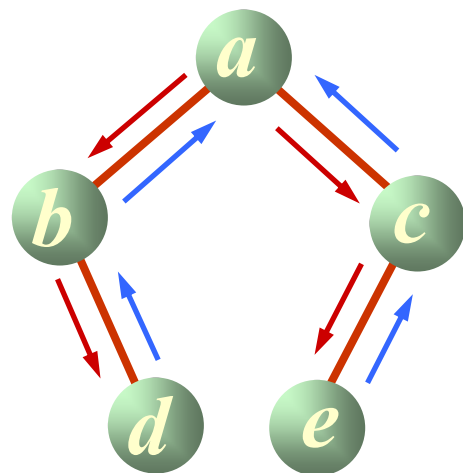
遍历结果

$a b c d - * + e f / -$



二叉树的后序遍历 (Postorder Traversal)

利用栈的后序遍历



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

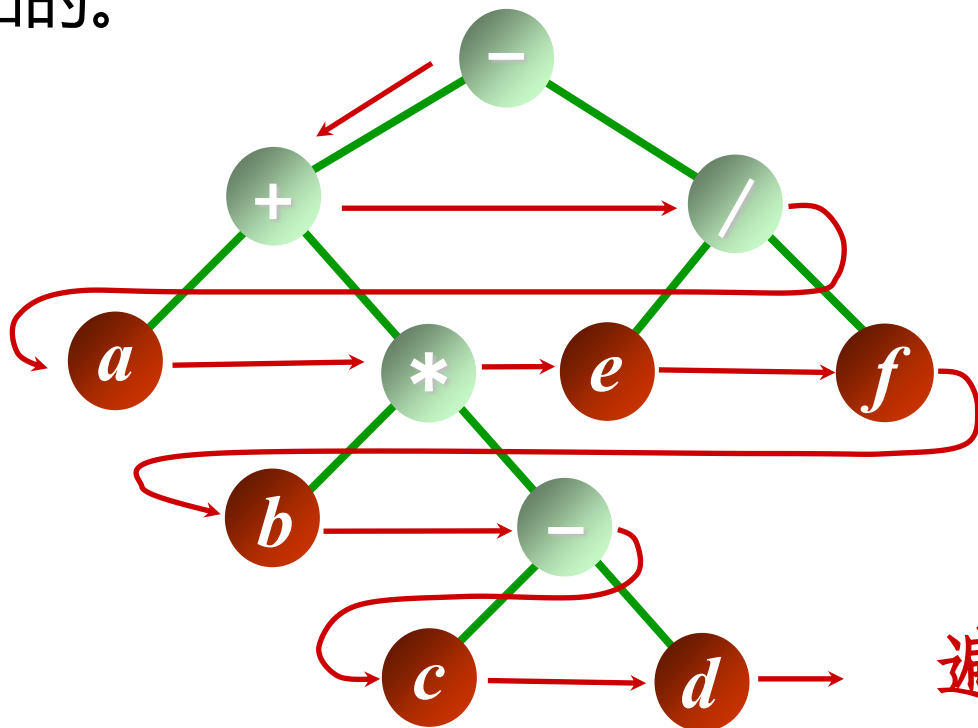
4.4 图结构

二、实例

4.5 应用举例

二叉树的层次遍历

- 层次序遍历二叉树就是从根结点开始，按层次逐层遍历，如图：
- 这种遍历需要使用一个先进先出的队列，在处理上一层时，将其下一层的结点直接进到队列（的队尾）。在上一层结点遍历完后，下一层结点正好处于队列的队头，可以继续访问它们。
- 算法是非递归的。



遍历顺序

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

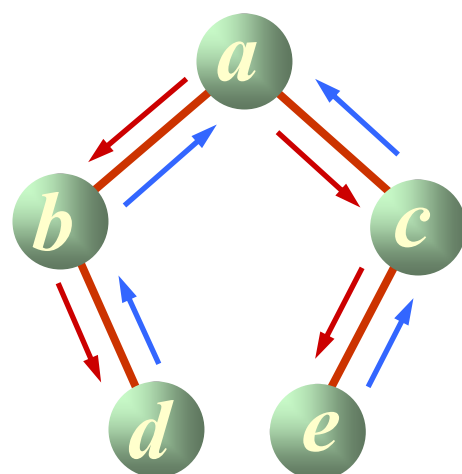
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

二叉树的层次遍历



Q

<i>a</i>				
----------	--	--	--	--

 访问*a*, 进队

Q

	<i>b</i>	<i>c</i>		
--	----------	----------	--	--

a 出队
访问*b*, 进队
访问*c*, 进队

Q

		<i>c</i>	<i>d</i>	
--	--	----------	----------	--

b 出队
访问*d*, 进队

Q

			<i>d</i>	<i>e</i>
--	--	--	----------	----------

c 出队
访问*e*, 进队

Q

				<i>e</i>
--	--	--	--	----------

d 出队

Q

--	--	--	--	--

e 出队

二叉树的遍历

总的来说:

- 前、中、后指的是根节点的访问顺序
 - 前序遍历就是先访问根节点
 - 中序遍历就是第二个访问根节点
 - 后序遍历就是最后访问根节点
- 左右子树的顺序通常固定为先左后右，除非明确指定是镜像顺序。

【【有挂！！】二叉树遍历秒解，前序遍历、中序遍历、后序遍历】

https://www.bilibili.com/video/BV1hw4m1q7DF/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

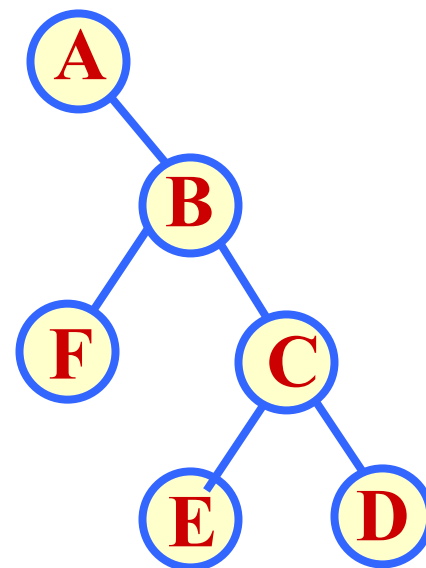
二叉树的遍历

例题：写出右图所示的二叉树对应的前序、中序、后序遍历序列。

前序：ABFCED

中序：AFBECD

后序：FEDCBA



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

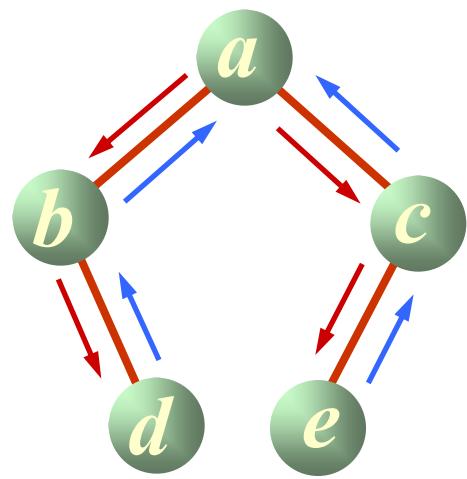
4.4 图结构

二、实例

4.5 应用举例

二叉树的建立过程

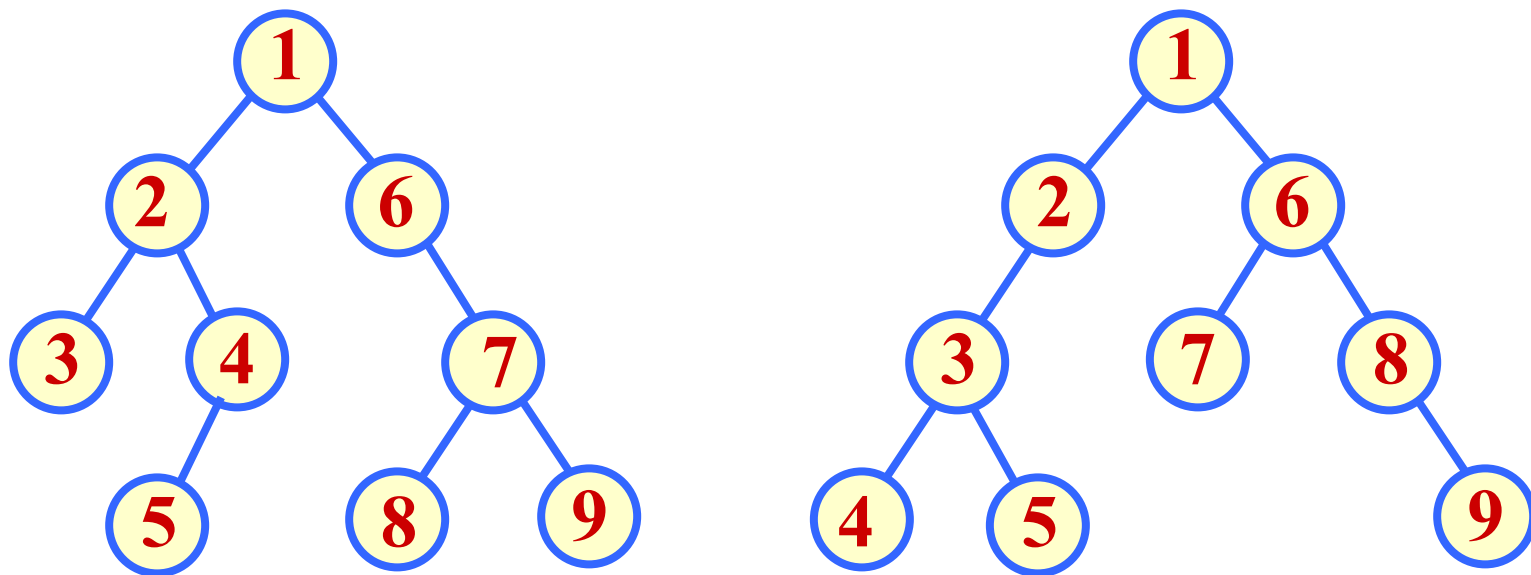
- 二叉树遍历的结果是将一个非线性结构中的数据通过访问排列到一个**线性序列**中。
- 前序序列: ***a b d c e*** 特点是第一个访问的***a***一定是树根, 只要左子树非空, 后面紧跟的***b***一定是根的左子女, ...
- 中序序列: ***b d a e c*** 特点是树根***a***把整个中序分成两部分,
a左侧子序列是根的**左子树**上的结点数据, 右侧子序列是根的右子树上的结点数据。



给出二叉树的遍历序列, 如何还原二叉树的结构?

二叉树的建立过程

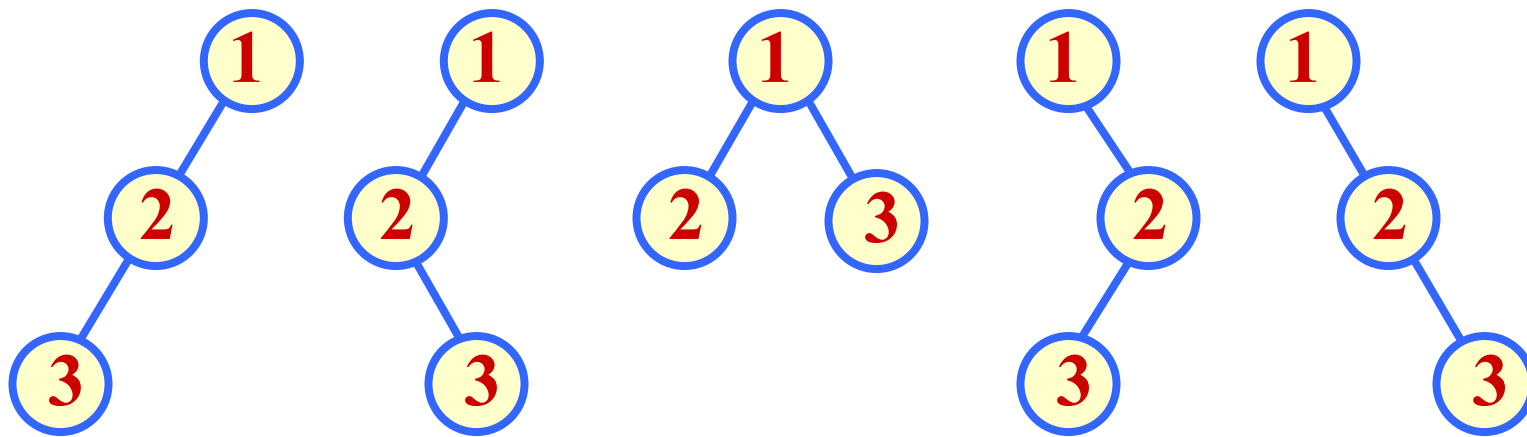
- 如果前序序列固定不变，给出不同的中序序列，可得到不同的二叉树。



- 固定前序排列，选择所有可能的中序排列，可能构造多少种不同的二叉树？

二叉树的建立过程

- 例如，有 3 个数据 { 1, 2, 3 }，可得 5 种不同的二叉树。它们的前序排列均为 123，中序序列可能是 321，231，213，132，123。

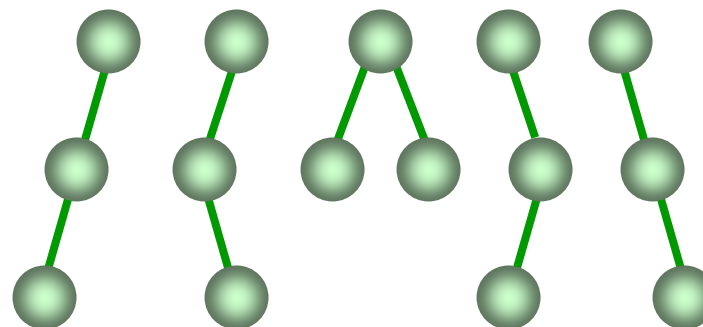
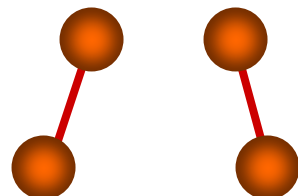


- 前序序列为 123，中序序列为 312 的二叉树不存在。

二叉树的建立过程

有0个, 1个, 2个, 3个结点的不同二叉树如下

Φ

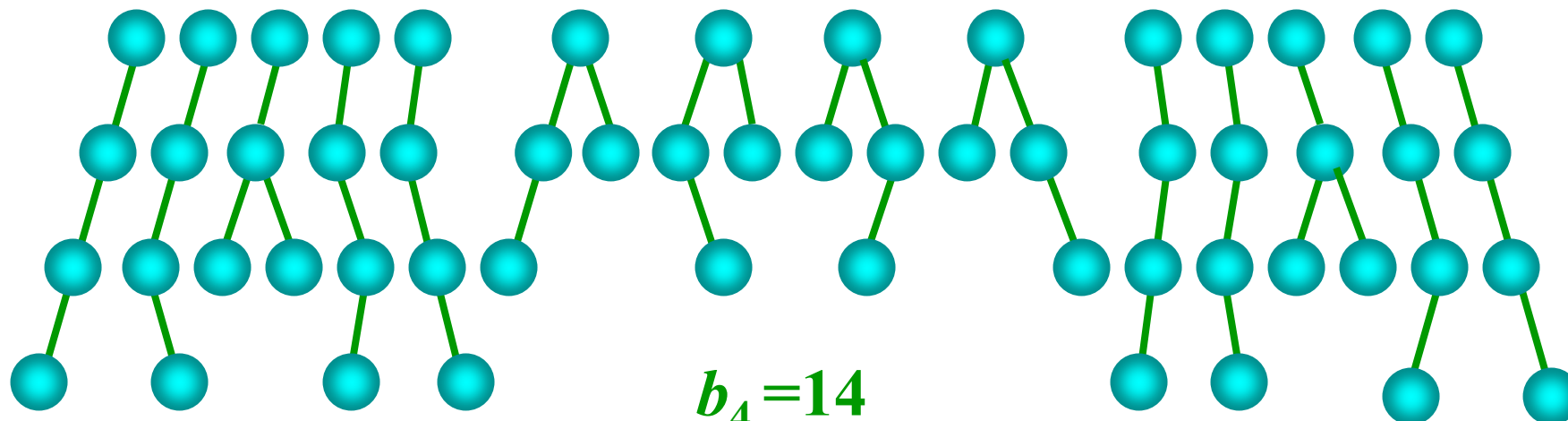


$b_0=1$

$b_1=1$

$b_2=2$

$b_3=5$



$b_4=14$

二叉树的建立过程

计算具有n个结点的不同二叉树的棵数

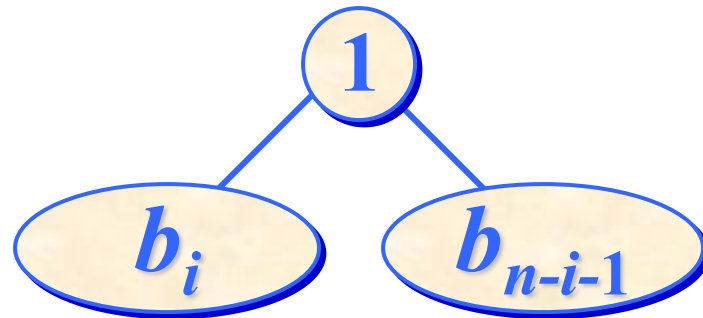
$$b_n = \sum_{i=0}^{n-1} b_i \cdot b_{n-i-1}$$

Catalan (卡特兰) 函数

$$b_n = \frac{1}{n+1} C_{2n}^n = \frac{1}{n+1} \frac{(2n)!}{n! \cdot n!}$$

$$b_3 = \frac{1}{3+1} C_6^3 = \frac{1}{4} \frac{6 \cdot 5 \cdot 4}{3 \cdot 2 \cdot 1} = 5$$

$$b_4 = \frac{1}{4+1} C_8^4 = \frac{1}{5} \frac{8 \cdot 7 \cdot 6 \cdot 5}{4 \cdot 3 \cdot 2 \cdot 1} = 14$$



二叉树的建立过程

- 指定一个序列无法建立唯一的二叉树，除非序列中明确标记出空节点的占位。
- 标记空节点的占位带来的问题与顺序表示一样，在很多极端情况下会带来冗余。
- 但是由二叉树的前序序列和中序序列可唯一地确定一棵二叉树。
- 由二叉树的后序序列和中序序列也可唯一地确定一棵二叉树。
- 由二叉树的前序序列和后序序列不可唯一地确定一棵二叉树。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

二叉树的建立过程

前序 + 中序:

- 前序遍历序列中的第一个数字为根节点，构造根节点；
- 找到根节点在中序遍历序列中的位置，中序中根节点左右两边分别为左子树和右子树，前序序列根节点后面为左子树+右子树；
- 递归处理左右子树，返回根节点，完成构造。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

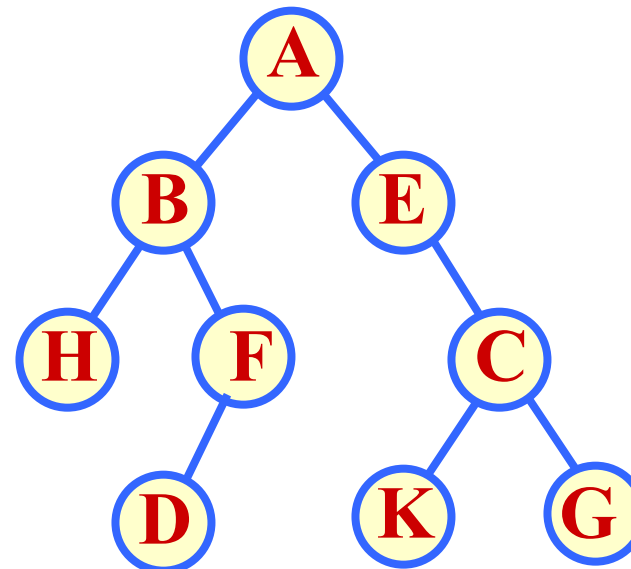
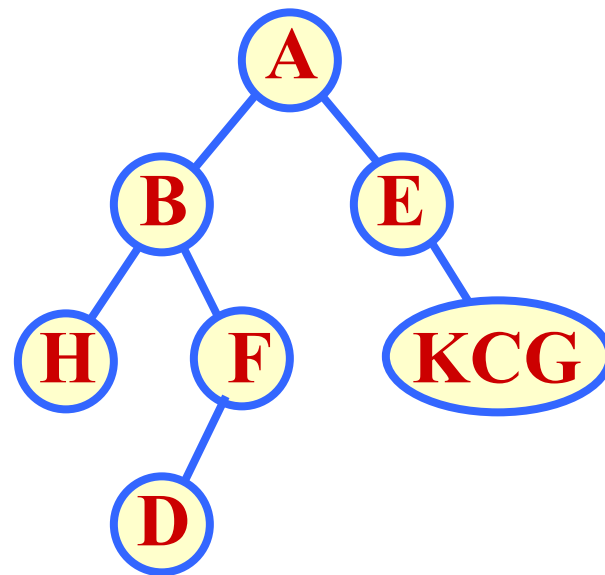
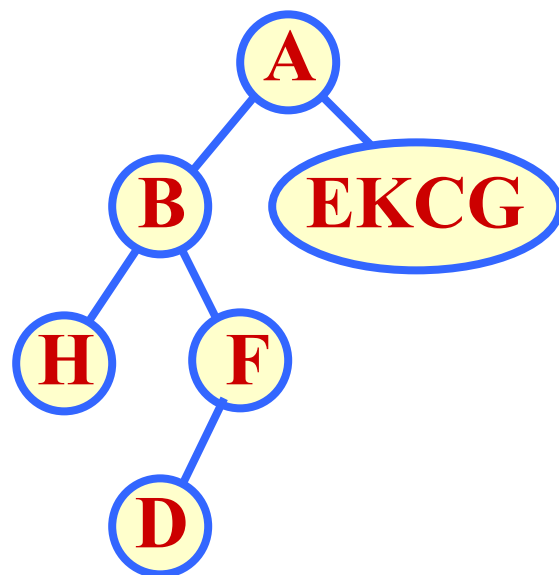
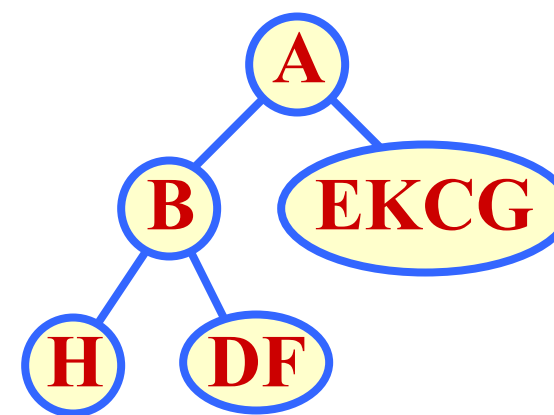
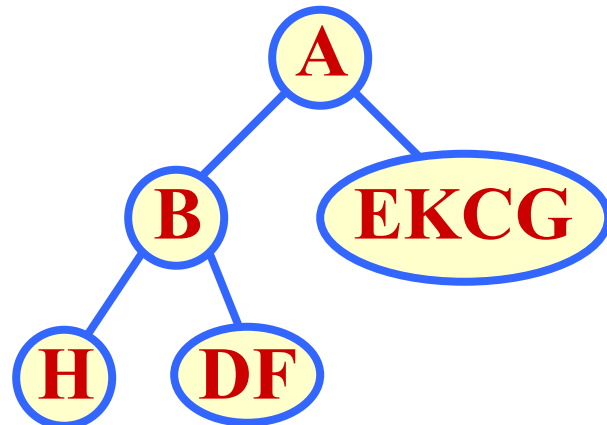
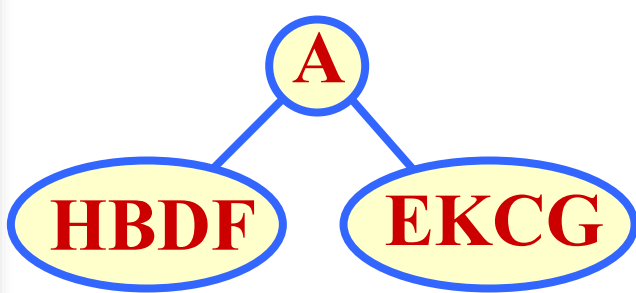
4.4 图结构

二、实例

4.5 应用举例

二叉树的建立过程

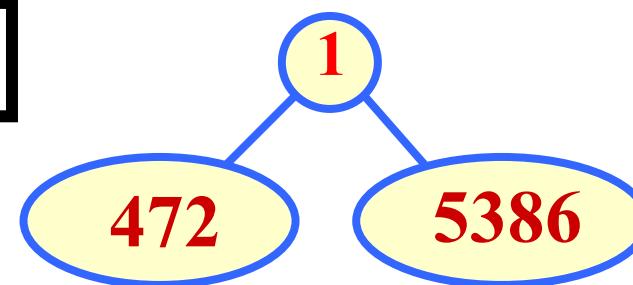
例：前序序列 { A B H F D E C K G } 中序序列 { H B D F A E K C G }



二叉树的建立过程

前序遍历序列为: $\{1, 2, 4, 7, 3, 5, 6, 8\}$

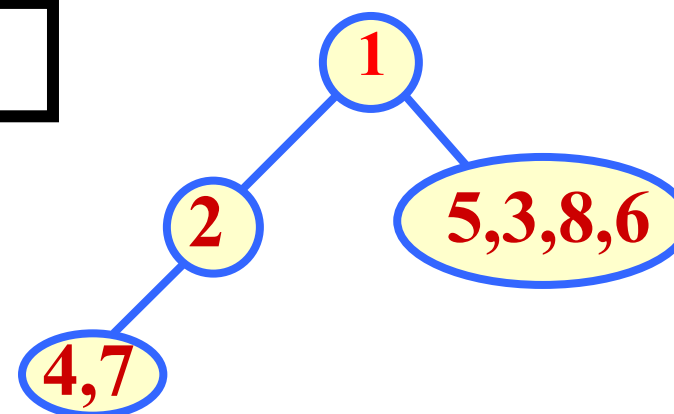
中序遍历序列为: $\{4, 7, 2, 1, 5, 3, 8, 6\}$



二叉树的建立过程

前序遍历序列为: $\{1, 2, 4, 7, 3, 5, 6, 8\}$

中序遍历序列为: $\{4, 7, 2, 1, 5, 3, 8, 6\}$



二叉树的建立过程

前序遍历序列为: $\{1, 2, 4, 7, 3, 5, 6, 8\}$

中序遍历序列为: $\{4, 7, 2, 1, 5, 3, 8, 6\}$



↑
ROOT1

↑
ROOT2

↑
左子树

└─
右子树

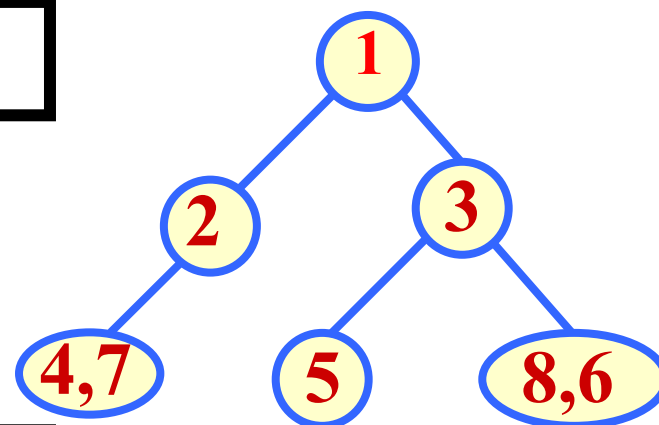


↑
ROOT1

↑
左子树

↑
ROOT2

└─
右子树



二叉树的建立过程

前序遍历序列为: $\{1, 2, 4, 7, 3, 5, 6, 8\}$

中序遍历序列为: $\{4, 7, 2, 1, 5, 3, 8, 6\}$



↑
ROOT1

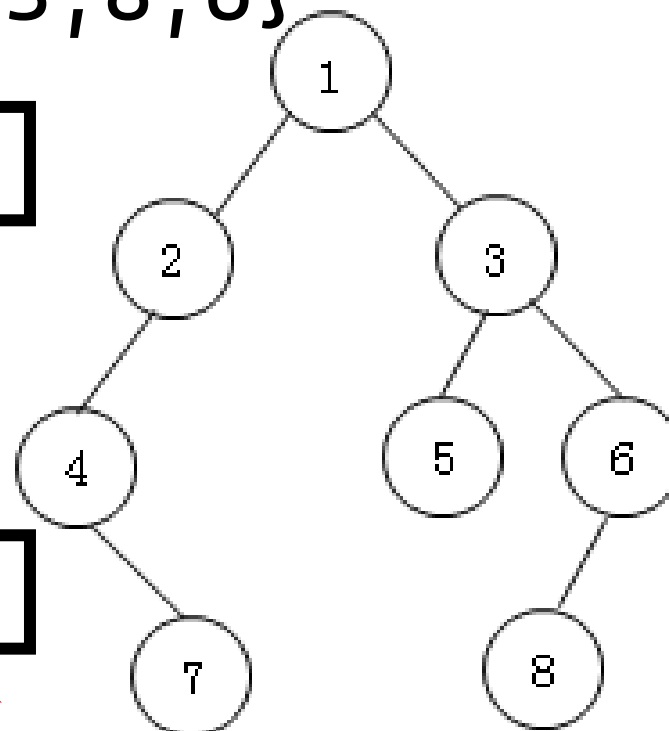
↑
ROOT2



↑
ROOT1

↑
右子树

↑
左子树



二叉树的建立过程

后序 + 中序

- 后序遍历序列中的最后一个数字为根节点，构造根节点；
- 找到根节点在中序遍历序列中的位置，中序中根节点左右两边分别为左子树和右子树，后序序列根节点前面为左子树 + 右子树；
- 递归处理左右子树，返回根节点，完成构造。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

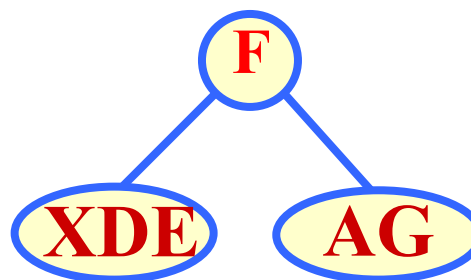
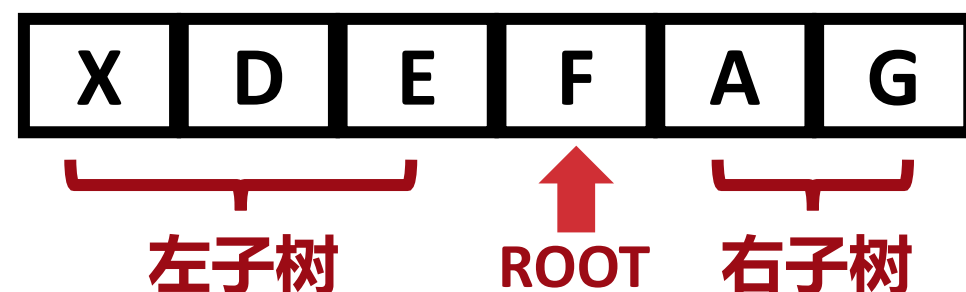
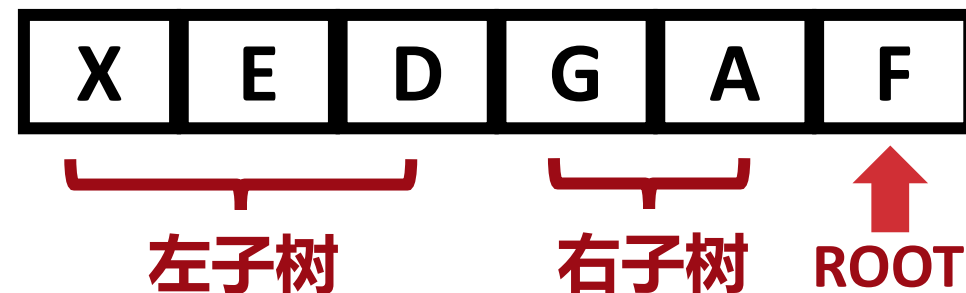
二、实例

4.5 应用举例

二叉树的建立过程

后序遍历序列为: {XEDGAF}

中序遍历序列为: {XDEFAG}



二叉树的建立过程

后序遍历序列为: {XEDGAF}

中序遍历序列为: {XDEFAG}



↑
ROOT



↑
ROOT



↑
ROOT

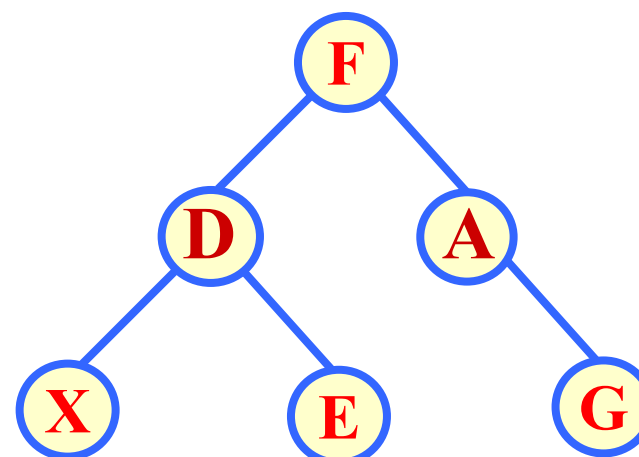
左子树

右子树



↑
ROOT

右子树

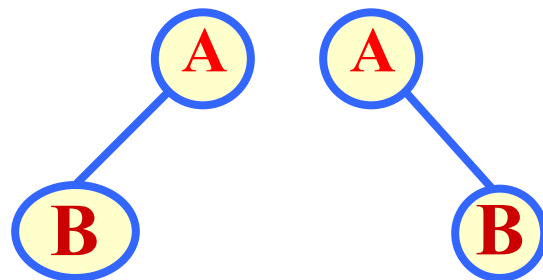


先序遍历序列为: {FDXEAG}

二叉树的建立过程

前序 + 后序

- 无法明确区分左子树和右子树的边界，尤其当某个节点只有一个子树时。
- 例如 前序: [A, B] 后序: [B, A]



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

二叉树的建立过程

例题：已知二叉树的前序遍历是ABHFDECKG，中序遍历序列是HBDFAEKCG，其后序遍历序列是（ ）

A、BHFDECKGA B、HDFBKGCEA

C、HFDBKCGEA D、ABEHFCDKG

由前序遍历可知，A是根节点，根据中序遍历可以划分得到左子树HBDF、右子树EKCG。对于左子树HBDF，根节点是B，左子树H、右子树DF；对于右子树，根节点是E，左子树空，右子树KCG。由此可以得到完整的二叉树结构。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

二叉树的建立过程

例题：已知二叉树的前序遍历是**ABHFDECKG**，
中序遍历序列是**HBDFAEKCG**，其后序遍历序列
是（ ）

A、 BHFDECKGA

B、 HDFBKGCEA

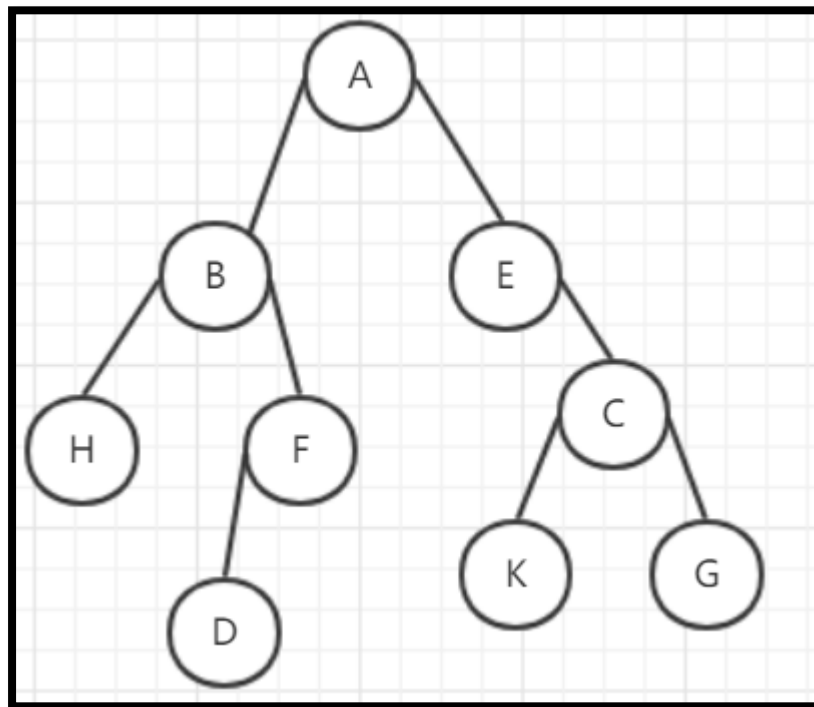
C、 HFDBKCGEA

D、 ABEHFCDKG

从树结构可以得到后序：

HDFBKGCEA

因此选**B**



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

二叉树的建立过程

例题：已知二叉树的前序遍历是`abdgcefh`，中序遍历序列是`dgbaechf`，其后序遍历序列是（ ）

A、`bdgcefha`

B、`gdbecfba`

C、`bdgaechf`

D、`gdbehfca`

从树结构可以得到后序选**D**

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

二叉树的应用

- 表达式求值：用二叉树表示算术或逻辑表达式（如 $a + b * c$ ），便于计算机解析和计算。
- 高效查找与排序：二叉搜索树（BST）支持快速查找、插入和删除操作，是许多有序数据结构（如数据库索引）的基础。
- 数据压缩：哈夫曼编码利用二叉树为不同字符分配长短不一的编码，实现无损压缩（如 ZIP、JPEG 等格式）。
- 等等.....

【数据结构合集 - 二叉搜索树(二叉排序树)(二叉查找树)】

https://www.bilibili.com/video/BV1uK421x7JJ/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

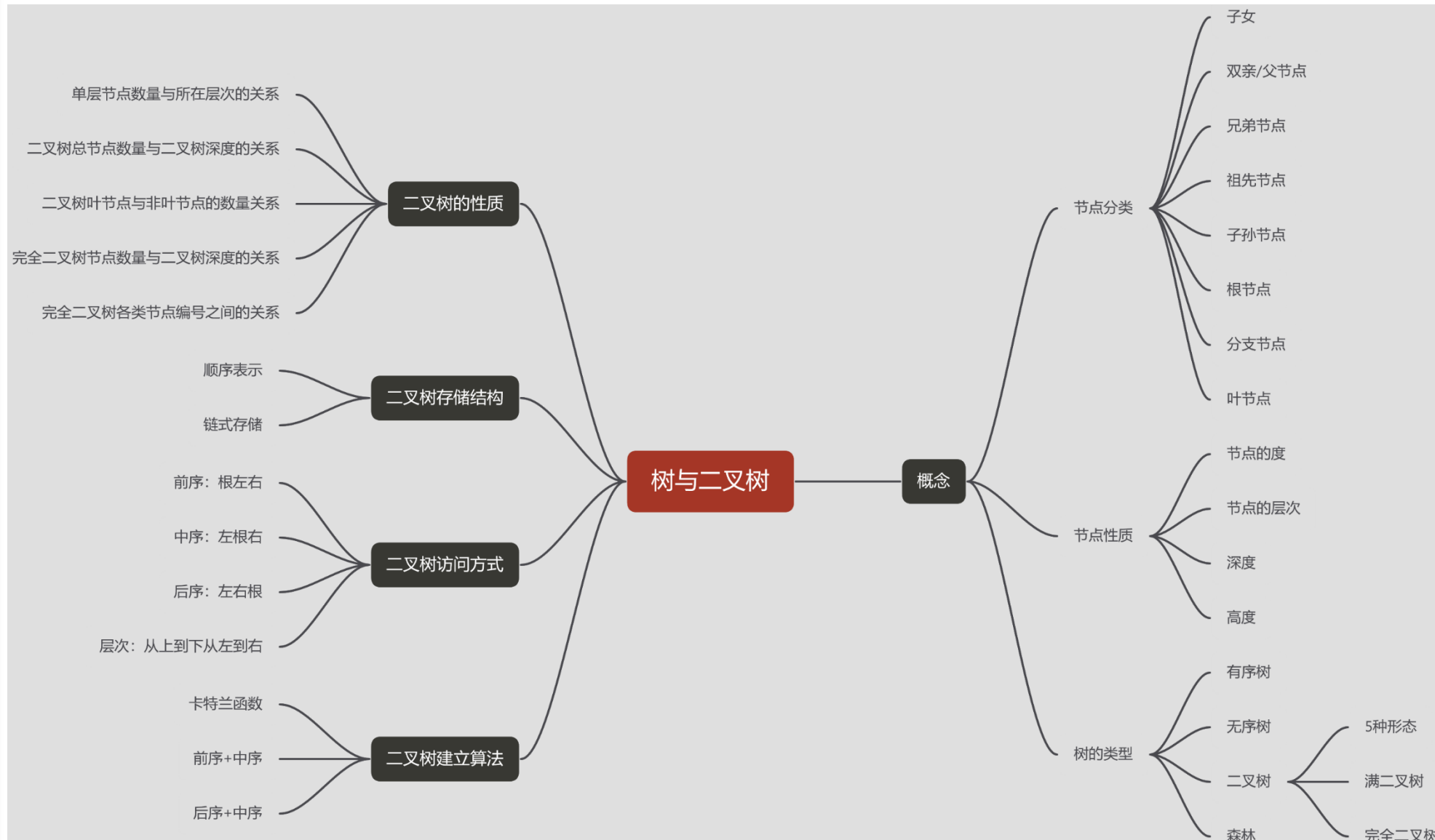
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

树的总结





4.4 图结构

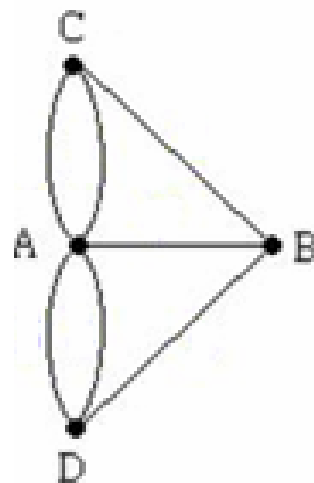
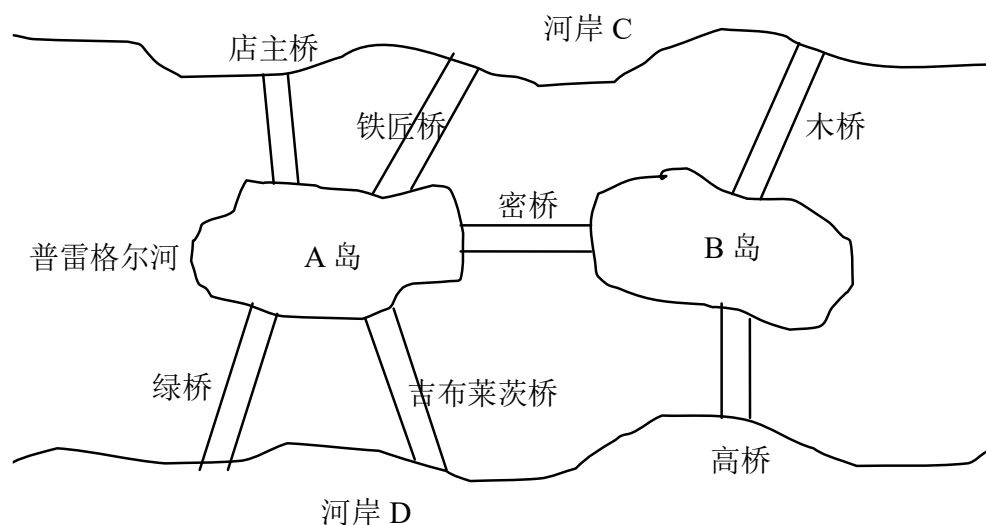
- 图的概念
- 图的存储
- 图的遍历
- 生成树和最小生成树

知识要点

任意关系，
图结构，
无向图，有向图，
边，弧，圈，
简单图，完全图，平面图，
子图，真子图，
支撑子图，
关联，邻接，
顶点的度，入度，出度，
路径，通路，回路，简单路径，
连通图，连通分量，强连通图，强
连通分量，
网，
邻接矩阵，
邻接表，逆邻接表，
图的遍历，
深度优先搜索，广度优先搜索，
生成树，
最小生成树

图论

- 哥尼斯堡七桥问题的数学抽象
 - 1736年，欧拉向圣彼得堡科学院递交了关于哥尼斯堡七桥问题的论文——“与位置几何相关的一个问题的解”。在论文中，欧拉将七桥问题抽象出来，把每一块陆地考虑成一个点，连接两块陆地的桥以线表示。并由此得到了如图一样的几何图形。



- 欧拉开创了数学的一个新的分支—图论 (Graph Theory)。图论的创立，为人们问题求解提供了一种新的数学理论和工具。

图的概念

- 图 (Graph) 是比线性表和树更为复杂的数据结构, 图可以表达数据对象之间的任意关系。
- **图定义** 图是由顶点集合(vertex)及顶点间的关系集合组成的一种数据结构:
$$\text{Graph} = (V, E)$$
其中 $V = \{x \mid x \in \text{某个数据对象}\}$
是顶点的有穷非空集合;
$$E = \{(x, y) \mid x, y \in V\}$$
或
$$E = \{ \langle x, y \rangle \mid x, y \in V \ \&\& \ \text{Path}(x, y) \}$$
是顶点之间关系的有穷集合, 也叫做边(edge)集合。
 $\text{Path}(x, y)$ 表示从 x 到 y 的一条单向通路, 它是有方向的。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

图的表示

- 图通常用图形来形象的描述，用圆圈表示**顶点**，连接两个顶点的线段表示**边**。

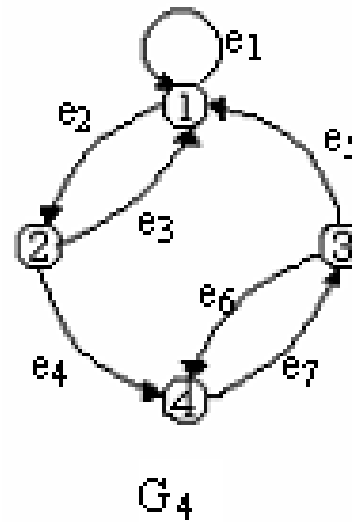
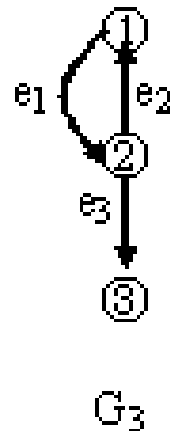
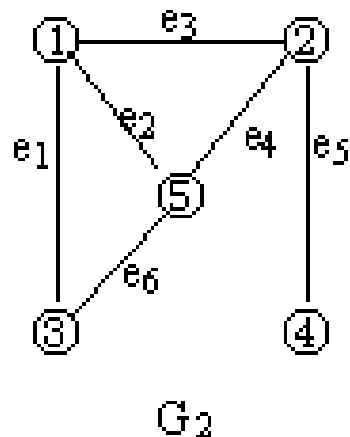
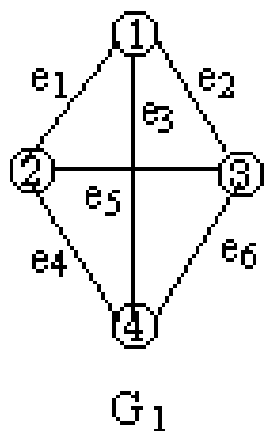
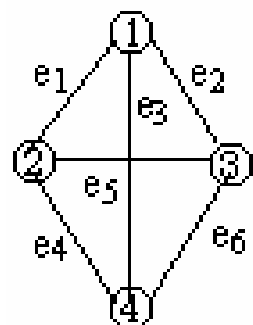


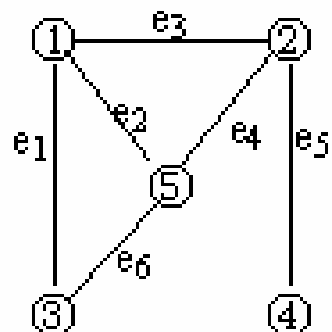
图 4-13 无向图与有向图示例

有向图和无向图

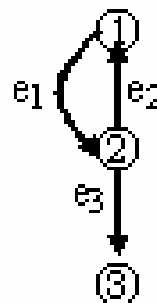
- 无向图，其边是顶点的无序对，即边没有方向性。若顶点 x 和顶点 y 之间有一条边，记作 (x,y)
- 有向图，其边是顶点的有序对，即边有方向性，又称为“弧” (Arc)，从顶点 x 到顶点 y 的有向边（弧）用 $\langle x,y \rangle$ 来表示



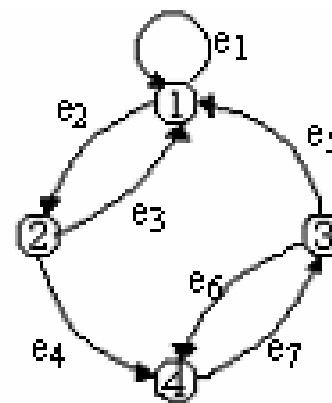
G_1



G_2



G_3



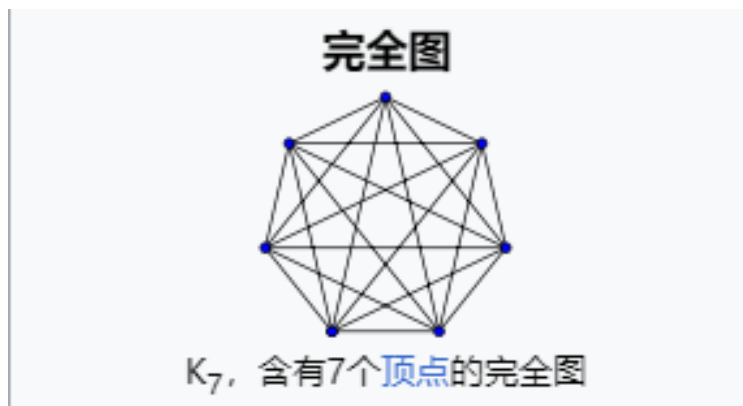
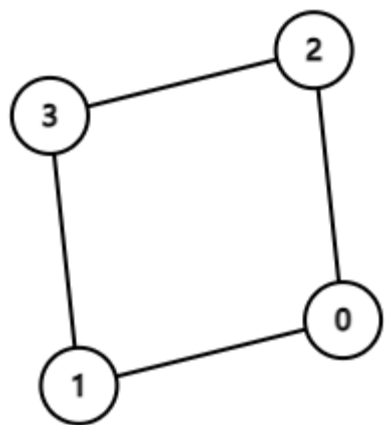
G_4

无向图

有向图

图相关概念--1

- 圈/环，若一条边（或弧）的两个顶点相同，则称其为“圈/环”
- 简单图，若图G不存在圈，也没有两边（或弧）连接同样两个顶点，则称这样的图为“简单图”。
- 完全图，对于简单图， n 个顶点的无向图最多有 $n(n-1)/2$ 条边，其中任意的两个顶点之间都有边相连，称为“完全图”



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

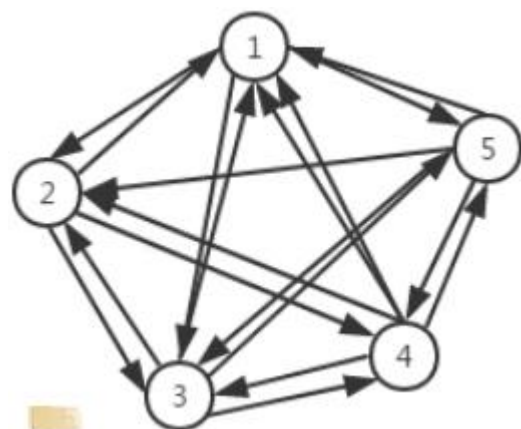
4.4 图结构

二、实例

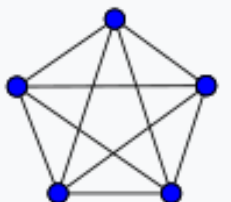
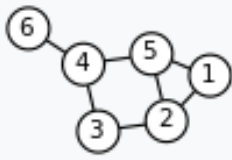
4.5 应用举例

有向完全图，对于有向图，具有 $n(n-1)$ 条弧的简单图称为“有向完全图”，相当于完全图的弧的数量 $\times 2$ 。

平面图，对于一个图，若存在一种画法，使其边仅在顶点处相交，则称其为“平面图”

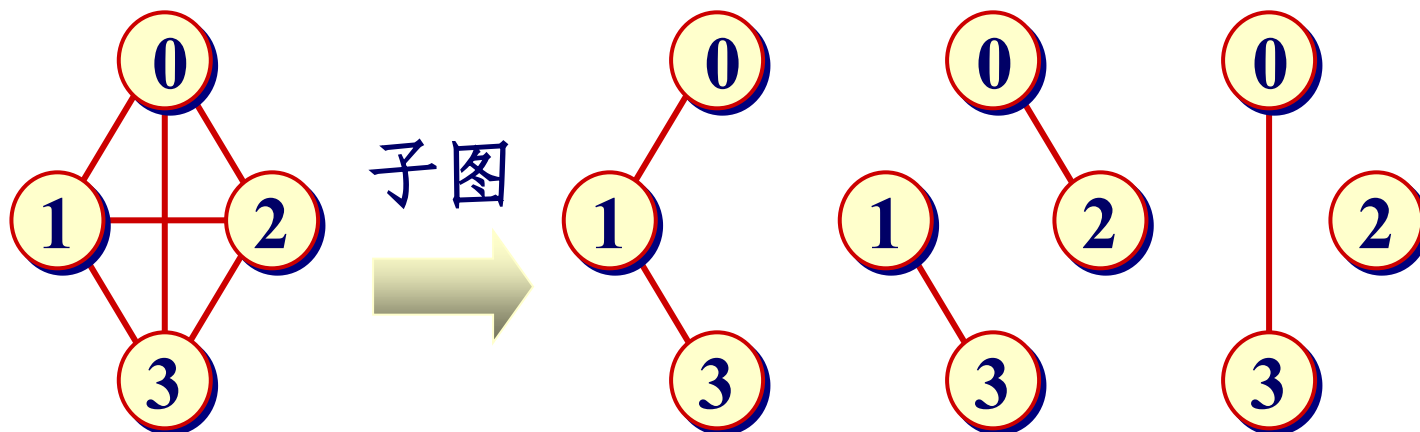


有向完全图

平面图	非平面图
 蝶形图，平面图的一种	 K_5不是平面图
 一个平面图	 $K_{3,3}$ (汤玛森图) 不是平面图

图相关概念--2

- 子图，若图 $G=\{V,E\}$ 和 $G'=\{V',E'\}$ 满足条件：
 - (1) $V'\subseteq V$;
 - (2) $E'\subseteq E$,
 则称 G' 为图 G 的“子图”



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

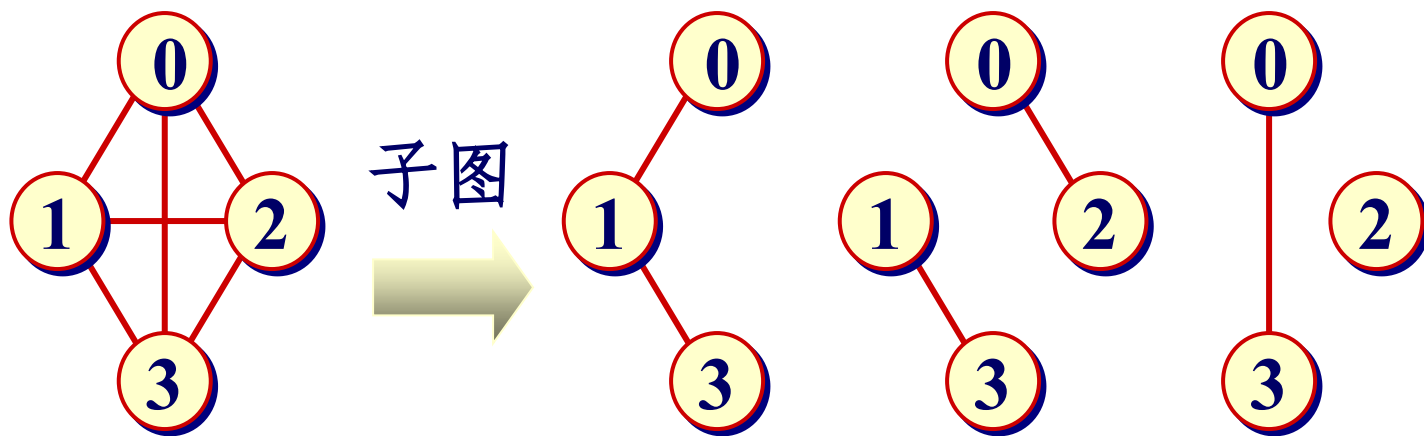
4.4 图结构

二、实例

4.5 应用举例

图相关概念--2

- 真子图，如果 $G' \subseteq G$ ，但 $G' \neq G$ ，则称 G' 为图 G 的“真子图”，记作 $G' \subset G$
- 支撑子图，若 $G' \subseteq G$ ，且 $V' = V$ ， $E' \subseteq E$ ，即两个图的顶点集相同，而 G' 的边集是 G 的边集的子集，则称 G' 是图 G 的“支撑子图”。



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

图相关概念--3

- 关联，顶点和边（或弧）之间的关系，一条边和它所连接的顶点是相关联的。
- 邻接，顶点和顶点之间、边和边之间的关系，和同一条边（或弧）相关联的两个顶点是邻接的，有共同顶点的两条边也是邻接的。
- 顶点 v 的度，指和顶点 v 相关联的边的条数，记作 $d(v)$
- 对于有向度，度又分为“入度”（Indegree）和“出度”（Outdegree）。顶点 v 的“入度”是指以 v 为终点（弧头）的弧的条数，记作 $ID(v)$ ； v 的“出度”是指以 v 为始点（弧尾）的弧的条数，记作 $OD(v)$

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

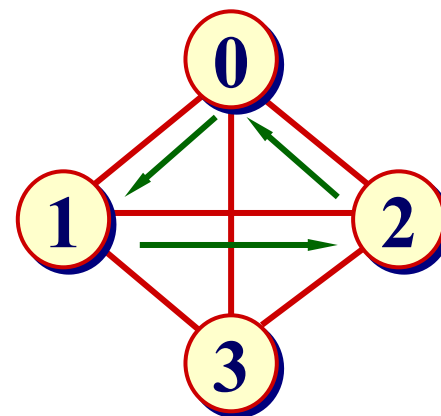
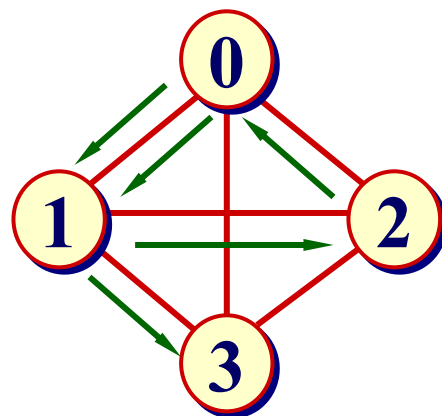
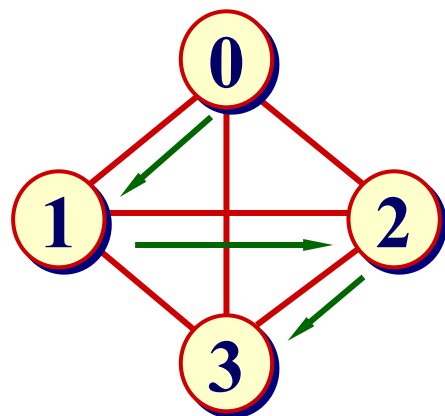
二、实例

4.5 应用举例

图相关概念--4

• 路径

- 从顶点 v 到顶点 v' 的一条路径是一个有穷非空序列 $w = (v = v_{i0}, v_{i1}, \dots, v_{ik}, v')$, 其中 $(v_{ij-1}, v_{ij}) \subseteq E$ 。
- 路径中边的数目 k 称为路径的“长度”。
- 起点和终点相同的路径称为“回路”或“环” (Cycle)。
- 序列中顶点不重复出现的路径称为“简单路径”。
- 顶点 v 到 v' 存在一条路径, 则称 v 到 v' 有一条“通路”。



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

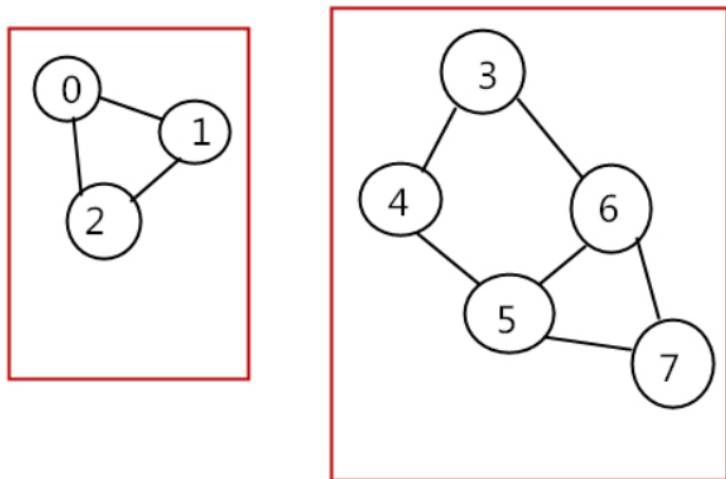
二、实例

4.5 应用举例

图相关概念--5

• 连通图

- 在无向图中，定点 v_i 和 v_j 之间存在一条通路（不需要直接连接），则成 v_i 和 v_j 之间是连通的。若图 G 中，任意两个不同的顶点 v_i 和 v_j 之间都是连通的，则称 G 为“连通图”
- 无向图的“连通分量”是它的极大连通子图，每个分量都是一个连通图。



左侧为非连通图，图中有两个极大连通子图（连通分量）。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

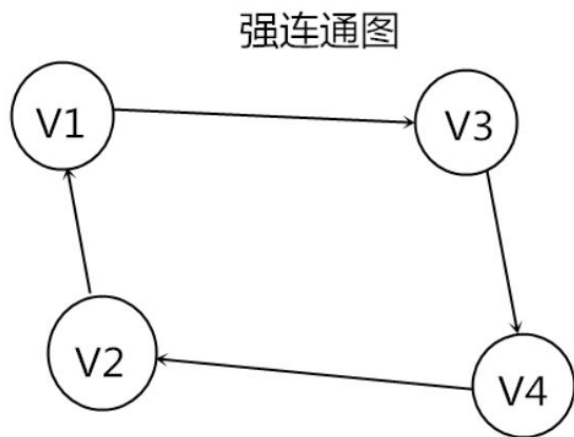
二、实例

4.5 应用举例

图相关概念--6

• 强连通图

- 若有向图G中，任意两个不同的顶点 v_i 和 v_j 之间都存在一条从 v_i 到 v_j 和一条从 v_j 到 v_i 通路，则称有向图G为“强连通图”。
- 有向图中，极大的强连通子图称为它的“强连通分量”。
- 所谓极大，是指再加入一个顶点或边，将不满足图的连通性。
一个图G的连通分量可以有多个



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

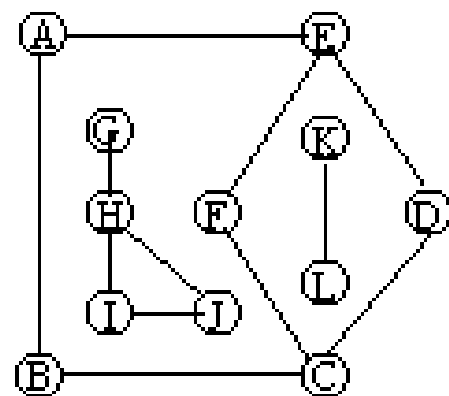
4.4 图结构

二、实例

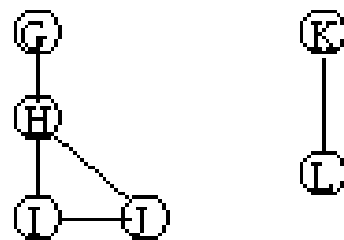
4.5 应用举例

举例

• 图和它的连通分量



(a) 图 G_5



(b) G_5 的部分连通分量

图 4-17 图和它的连通分量

图相关概念--6

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

• 网与图的权

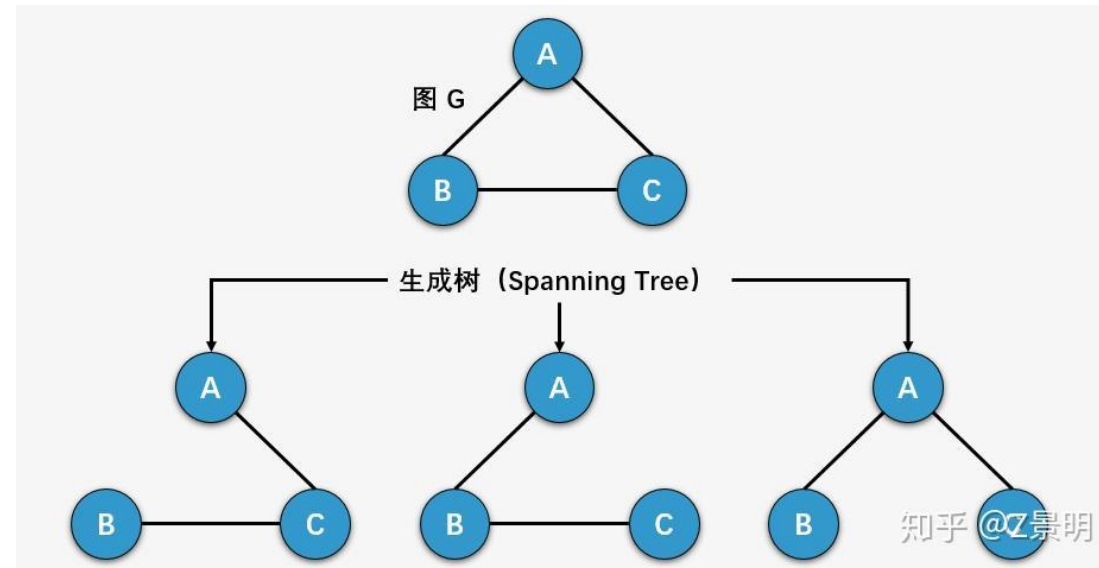
- 图中的边或弧往往给定一个权 (Weight) , 这种带权的图即是网

■ 生成树

一个连通图G的生成树是图G的极小连通子图 (之所以称为极小是因为此时如果删除一条边, 就无法构成生成树, 也就是说极小连通子图的每个边都是不可少的。), 它含有图中全部 n 个顶点, 但只有足以构成一棵树的 $n-1$ 条边。

【数据结构与算法, 图的数据结构, 动画讲解】

https://www.bilibili.com/video/BV1ET421673Y/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e



图的存储

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

- 存储问题

- 数据、数据之间的关系
- 静态存储（顺序存储）和动态存储（链式存储）
- 存储结构影响算法

- 图的存储

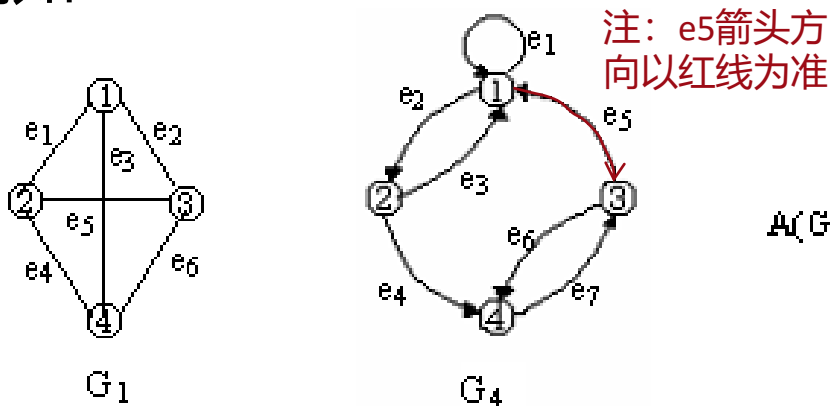
- 邻接矩阵
- 邻接表和逆邻接表
- 十字链表

邻接矩阵

- 在图的邻接矩阵表示中，有一个记录各个顶点信息的**顶点表**，还有一个表示各个顶点之间关系的**邻接矩阵**。
- 设图 $A = (V, E)$ 是一个有 n 个顶点的图，图的邻接矩阵是一个二维数组 $A.edge[n][n]$ ，定义：

$$A.Edge[i][j] = \begin{cases} 1, & \text{如果 } \langle i, j \rangle \in E \text{ 或者 } (i, j) \in E \\ 0, & \text{否则} \end{cases}$$

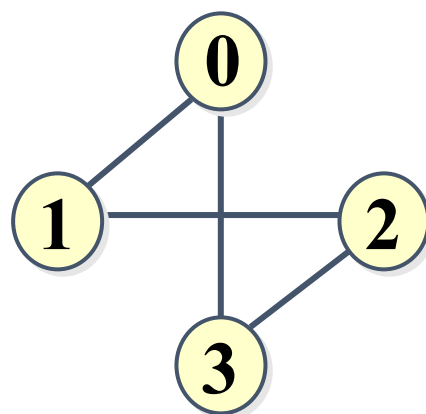
• 例如



$$A(G_1) = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

$$A(G_4) = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

邻接矩阵



$$\mathbf{A}.\text{edge} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$



$$\mathbf{A}.\text{edge} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

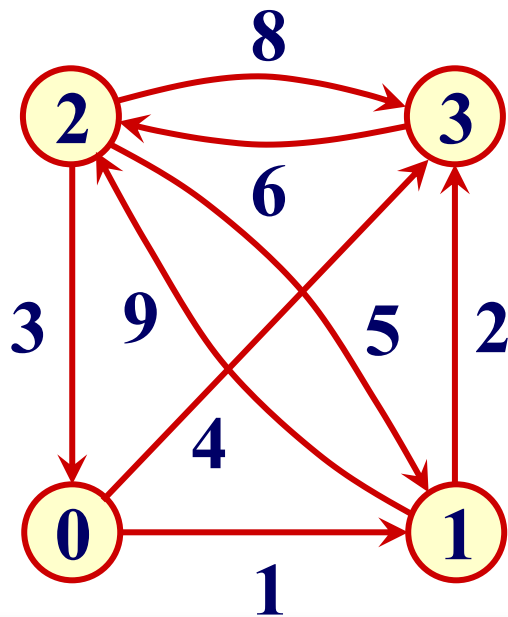
- 无向图的邻接矩阵是对称的；
- 有向图的邻接矩阵可能是不对称的。

邻接矩阵

- 在有向图中,统计第 i 行 1 的个数可得顶点 i 的出度,统计第 j 列 1 的个数可得顶点 j 的入度。
- 在无向图中,统计第 i 行 (列) 1 的个数可得顶点 i 的度。

网络的邻接矩阵

$$A.\text{edge}[i][j] = \begin{cases} W(i,j), & \text{若 } i \neq j \text{ 且 } \langle i,j \rangle \in E \text{ 或 } (i,j) \in E \\ \infty, & \text{若 } i \neq j \text{ 且 } \langle i,j \rangle \notin E \text{ 或 } (i,j) \notin E \\ 0, & \text{若 } i = j \end{cases}$$



$$A.\text{edge} = \begin{bmatrix} 0 & 1 & \infty & 4 \\ \infty & 0 & 9 & 2 \\ 3 & 5 & 0 & 8 \\ \infty & \infty & 6 & 0 \end{bmatrix}$$

邻接表

- 所谓“邻接表”（Adjacency list）就是对图中的每个顶点与它相关的所有边建立一个单链表，每个链表的头结点保存该顶点的有关信息，链内其它结点，保存与该顶点向关联的边的信息。
- 存储结构

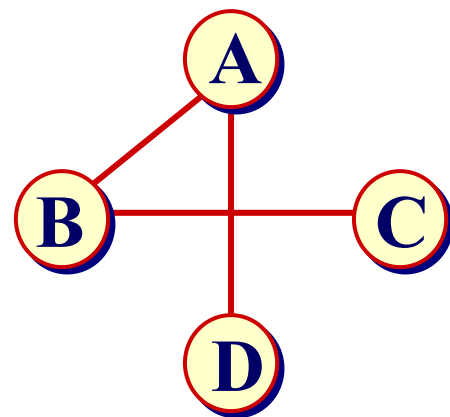
vexdata	firstarc
---------	----------

头结点

adjvex	arcinfo	nextarc
--------	---------	---------

链内结点

举例——无向图的邻接表



	data	adj	dest	link	dest	link
0	A		1		3	^
1	B		0		2	^
2	C		1	^		
3	D		0	^		

- 统计某顶点对应边链表中结点个数，可得该顶点的度。
- 某条边 (v_i, v_j) 在邻接表中有两个边结点，分别 在第 i 个顶点和第 j 个顶点对应的边链表中。

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

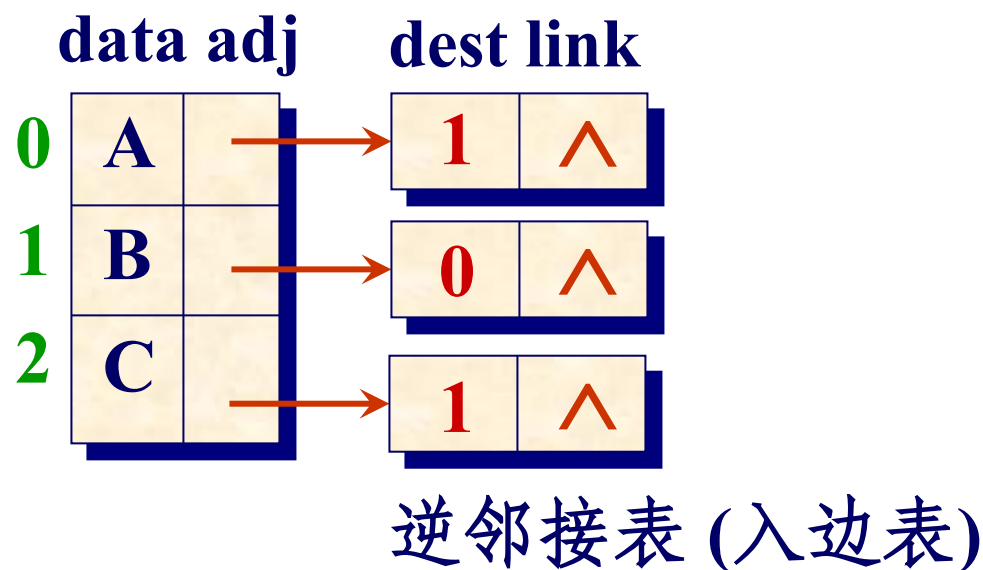
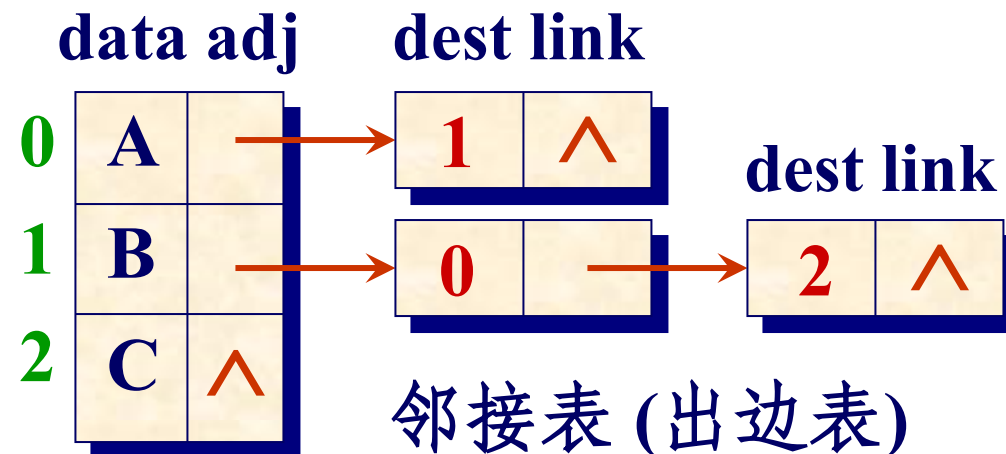
4.3 树形结构

4.4 图结构

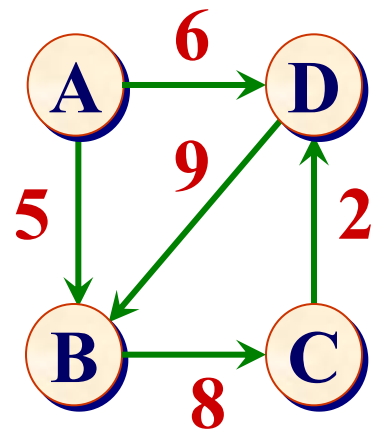
二、实例

4.5 应用举例

举例——有向图的邻接表



举例——带权图的邻接表



	data	adj	dest	cost	link
0	A	→	1	5	→ 3 6 ^
1	B	→	2	8	→ ^
2	C	→	3	2	→ ^
3	D	→	1	9	→ ^

【图的数据结构对比：邻接矩阵和邻接链表 数据结构与算法】
https://www.bilibili.com/video/BV1iHstefEaf/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

(顶点表) (出边表)

- 统计出边表中结点个数，得到该顶点的出度；
- 统计入边表中结点个数，得到该顶点的入度。

邻接矩阵和邻接表对比

对比项	邻接矩阵	邻接表
存储方式	二维数组（固定大小）	每个点对应一个列表（动态）
空间占用	总是 $O(V^2)$ ，与边数无关	$O(V+E)$ ，随边数变化
判断两点是否相连	快（直接查）	较慢（需遍历列表）
遍历所有邻居节点	慢（要扫整行）	快（直接看它的列表）
适合图的类型	稠密图（边多）	稀疏图（边少）
实现复杂度	简单	稍复杂（需管理多个列表）

【图的数据结构对比，邻接矩阵和邻接链表 数据结构与算法】
https://www.bilibili.com/video/BV1iHstefEaf/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

图的遍历

- 图的遍历（七桥问题）
 - 给定图 $G=(V,E)$ 后，一个最基本的问题就是对于顶点集中的任意顶点 v_0 ，从 v_0 出发访问图中的所有顶点，且每一个顶点只访问一次
 - 图的遍历算法是解决图的连通性问题、拓扑排序以及关键路经等算法的基础
- 深度优先搜索
- 广度优先搜索

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

生成树

- 图的生成树，当 G 为连通图时， G 的具有下列性质的子图 T 称为 G 的生成树（又称支撑树）
 - (1) T 包含 G 的所有顶点；
 - (2) T 为连通图；
 - (3) T 包含的边数最少。
- 一般情况下，一个连通图的生成树不是唯一的。若连通图 G 有 n 个顶点，由 $n-1$ 条边构成的包含 G 中所有顶点的连通子图都是 G 的生成树（连接 n 个顶点至少需要 $n-1$ 条边）。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

深度优先搜索

- 深度优先搜索 (Depth First Search, DFS) 递归算法
 - (1) 访问 v_0 , 对 v_0 作已访问标记。
 - (2) 找出与 v_0 邻接未被访问的顶点 w , 按深度优先搜索与 w 邻接的未作标记的顶点。
 - (3) 若与 v_0 邻接的顶点皆已作访问标记, 则返回; 否则, 找出与 v_0 邻接的未被访问的所有顶点 w , 按深度优先搜索与 w 邻接的未作标记的顶点。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

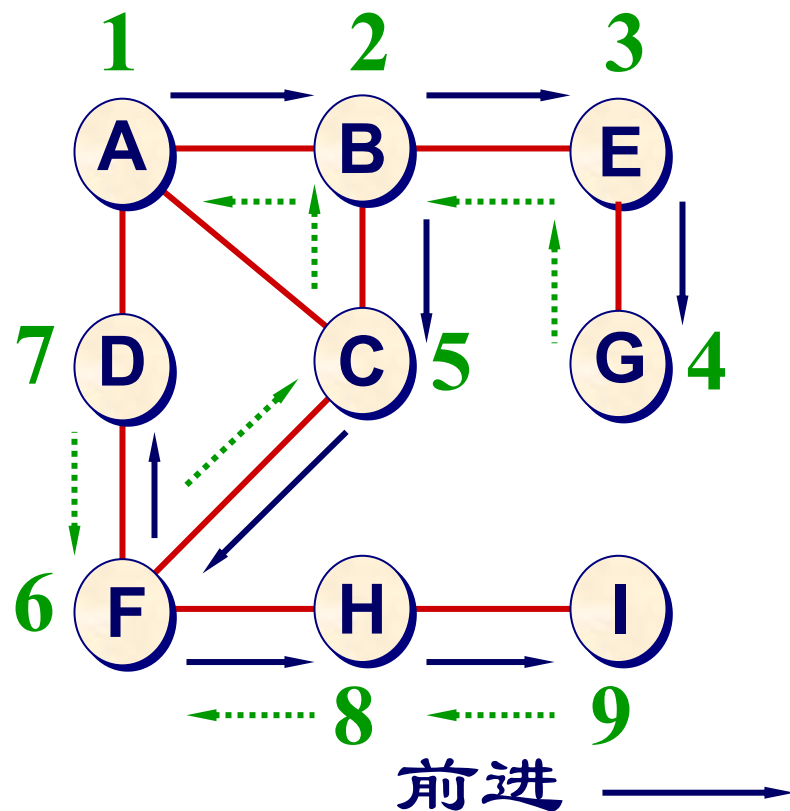
4.4 图结构

二、实例

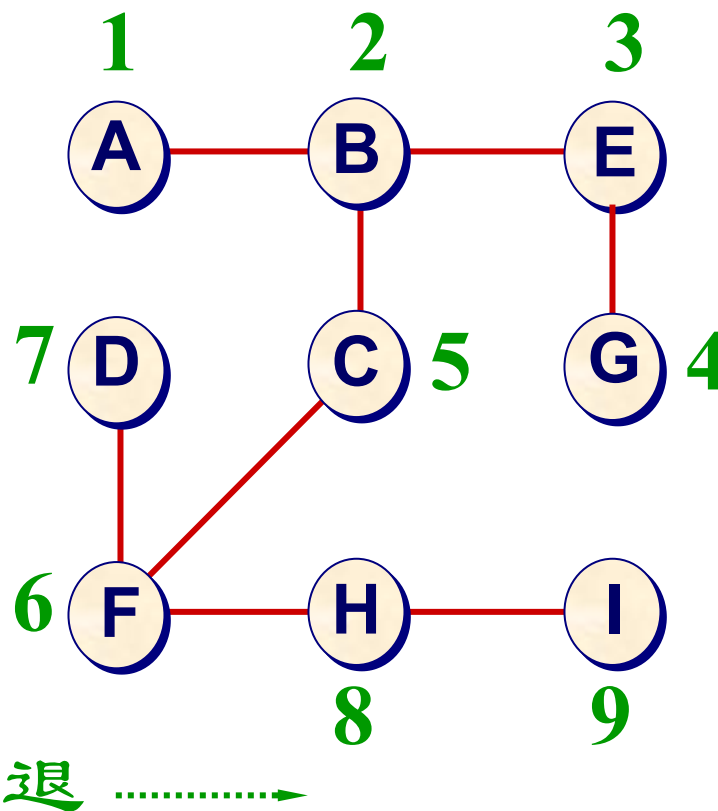
4.5 应用举例

深度优先搜索

深度优先搜索的示例



深度优先搜索过程



深度优先生成树

举例

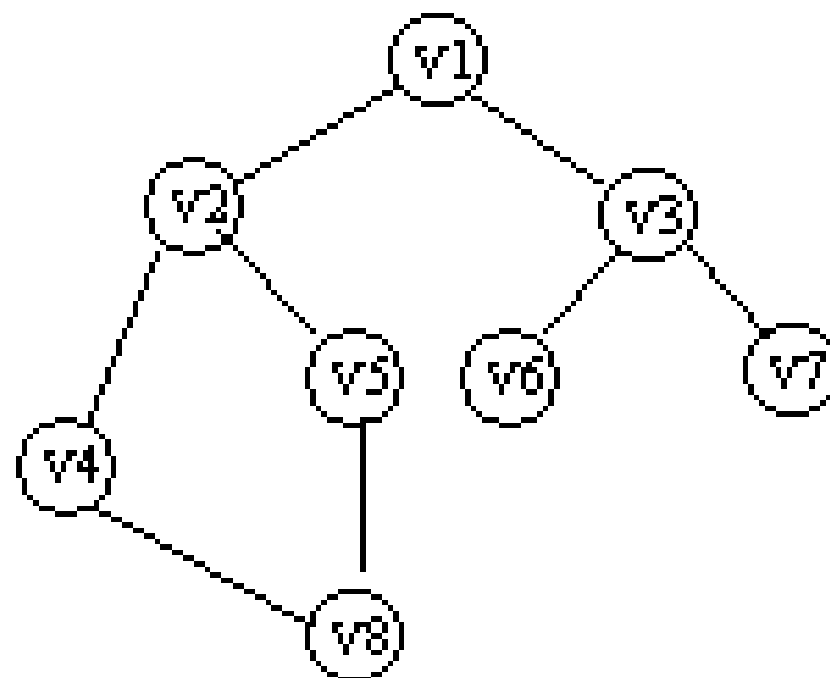


图 4-20 图 G_6

- 按照深度优先搜索的一种序列为：

$v_1, v_2, v_4, v_8, v_5, v_3, v_6, v_7$



广度优先搜索

- 图的广度优先搜索 (Breadth First Search, BFS) 类似于树的按层次遍历

- (1) 首先从顶点 v (初值为 v_0) 开始, 访问 v , 对 v 作已访问标记;
- (2) 按照字母或编号次序依次访问与 v 邻接且尚未访问的顶点;
- (3) 然后再顺序从这些顶点出发, 按同样原则访问邻接这些顶点的未被访问的所有顶点, 直到所有顶点都被访问过为止。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

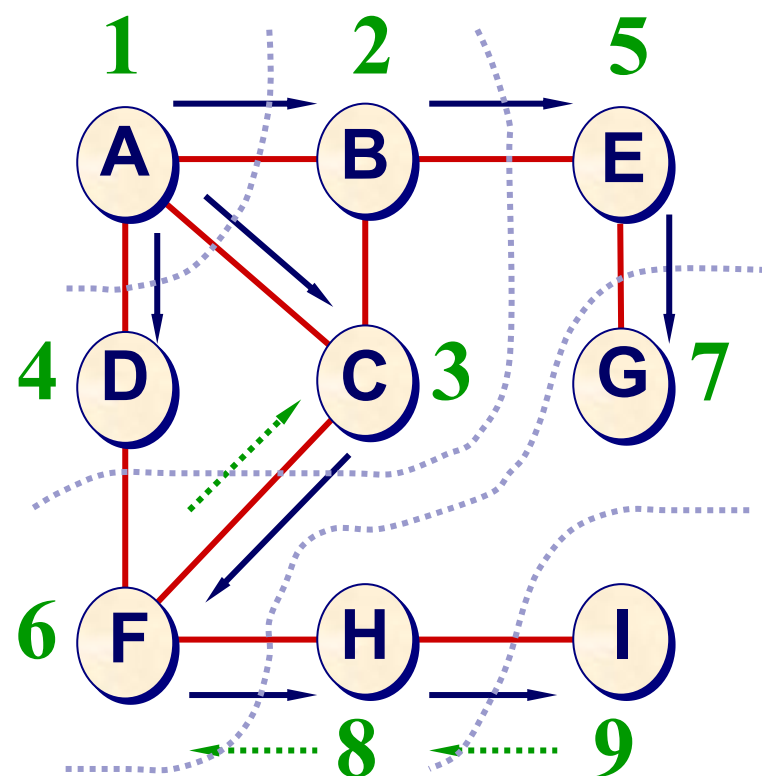
4.4 图结构

二、实例

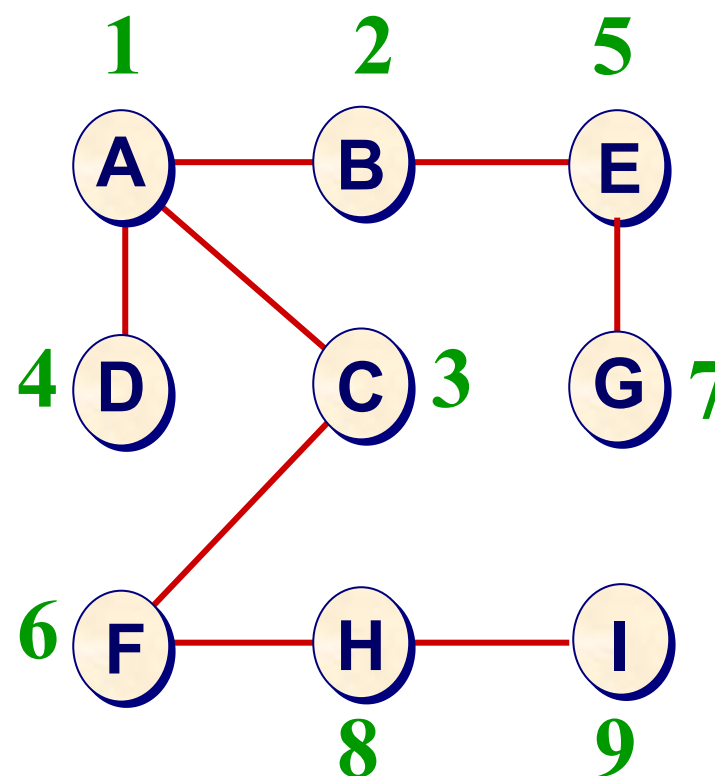
4.5 应用举例

广度优先搜索

■ 广度优先搜索的示例



广度优先搜索过程



广度优先生成树

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

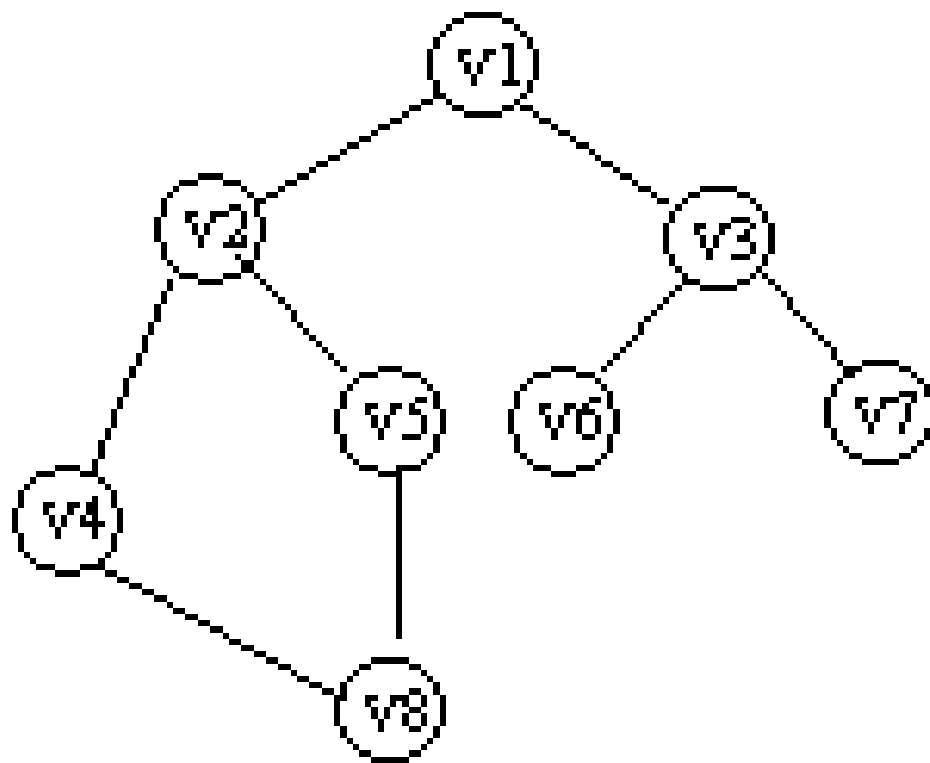
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

举例

图 4-20 图 G_6

- 广度优先搜索的一种序列为: $v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8$

最小代价生成树

- 图往往是赋权图，即图中的每条边（或弧）都对应一个数，这样的图又称网
- 在赋权图中，其生成树往往也是不唯一的，我们通常用生成树中各边的权之和来衡量生成树，称作生成树的代价。在 G 的所有生成树中，其代价最小的生成树称为最小代价生成树（Minimum Cost Spanning Tree），简称最小生成树（Minimum Spanning Tree, MST）
- 在 n 个城市之间建立通信网络，如何选择 $n-1$ 条线路，使得总的造价最低？这就是一个典型的图的最小生成树的例子

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

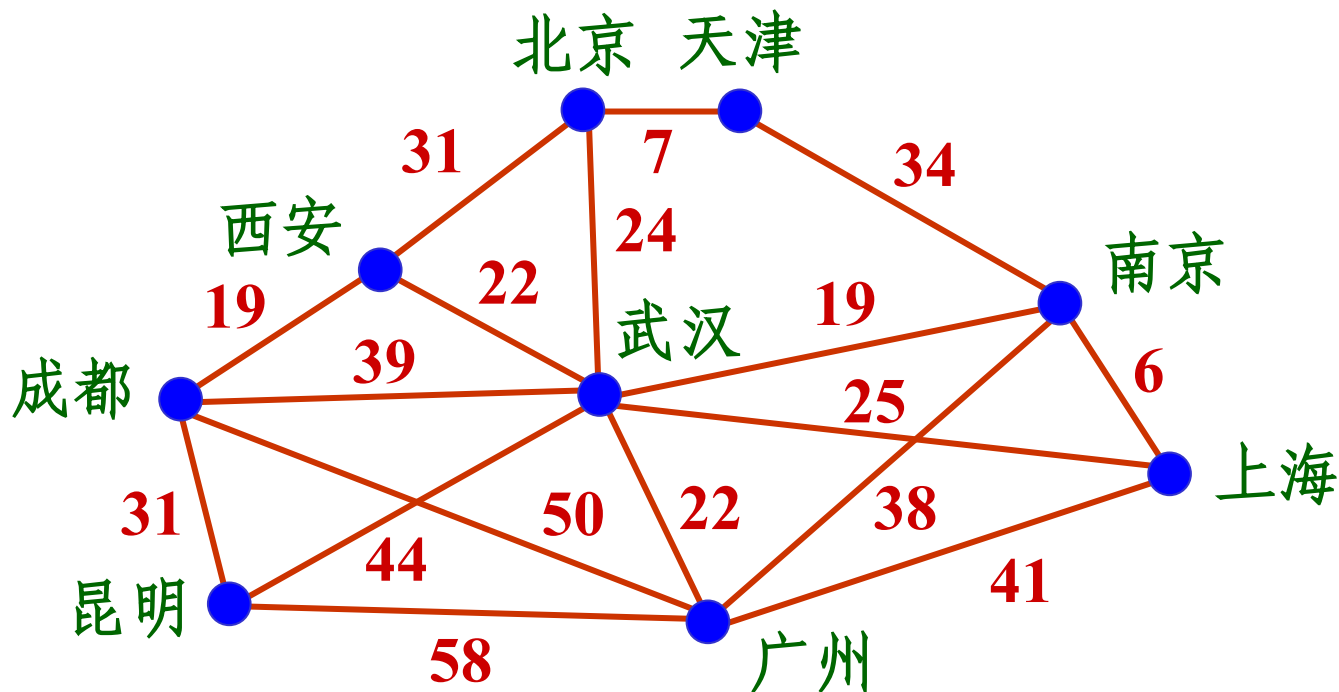
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

最小代价生成树



- 构造最小生成树的准则
 - ❖ 必须使用且仅使用该网络中的 $n-1$ 条边来联结网络中的 n 个顶点;
 - ❖ 不能使用产生回路的边;
 - ❖ 各边上的权值的总和达到最小。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

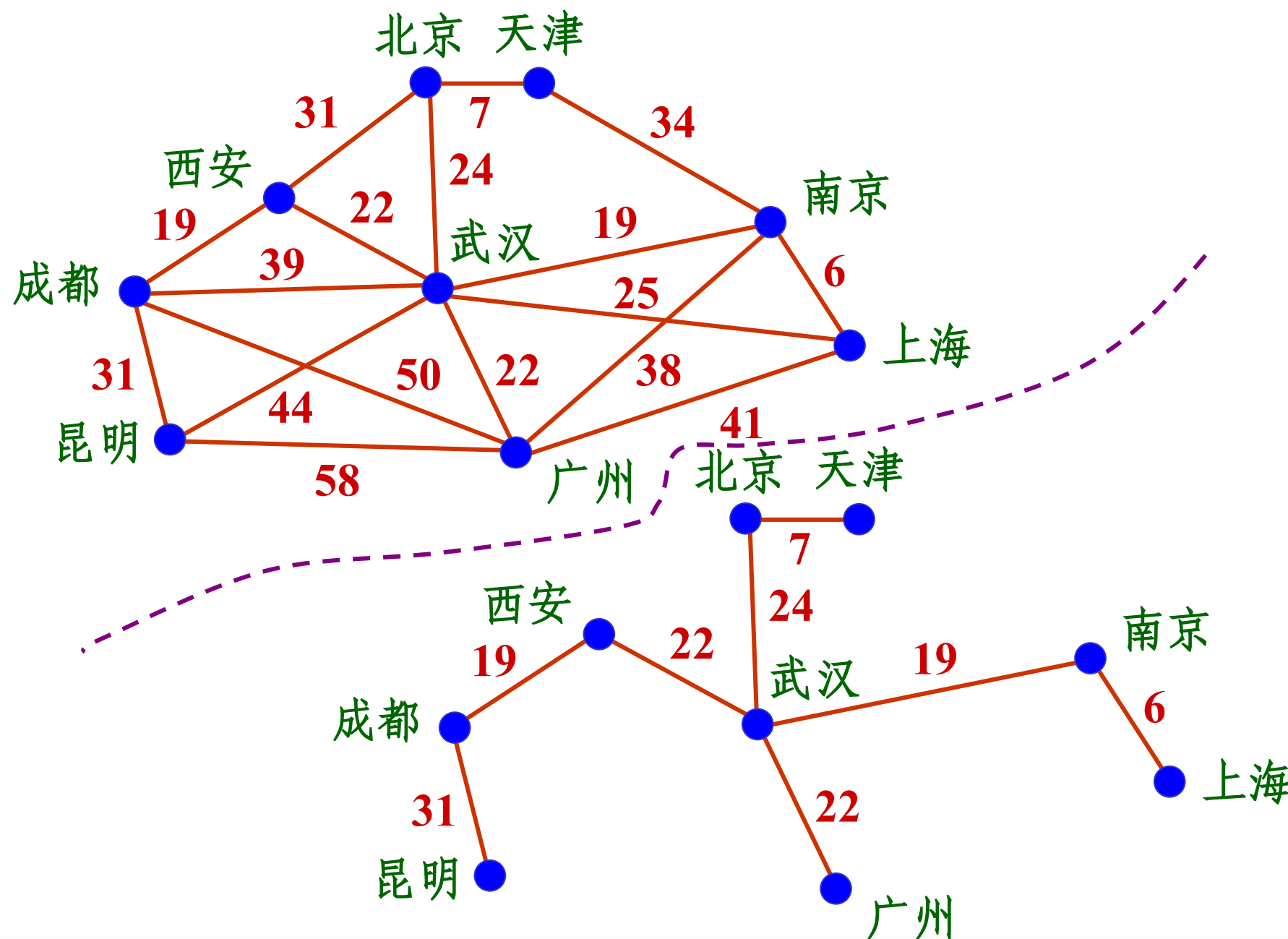
4.2 线性结构

4.3 树形结构

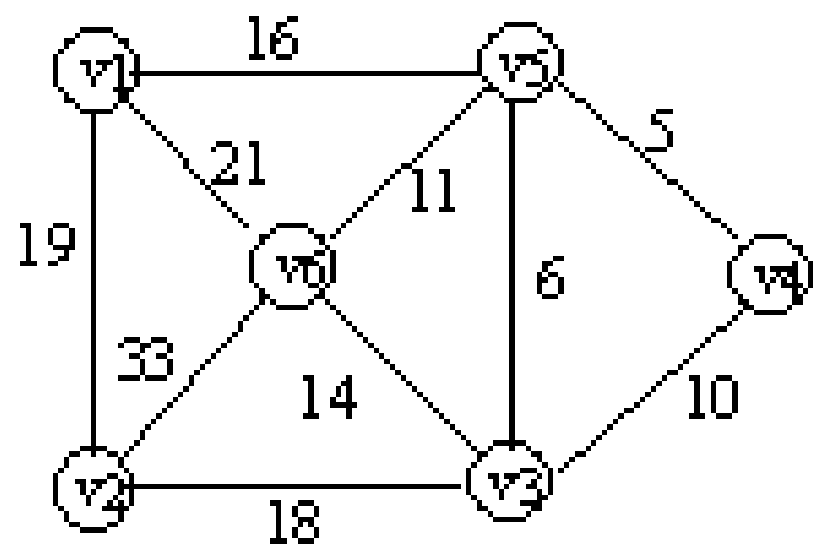
4.4 图结构

二、实例

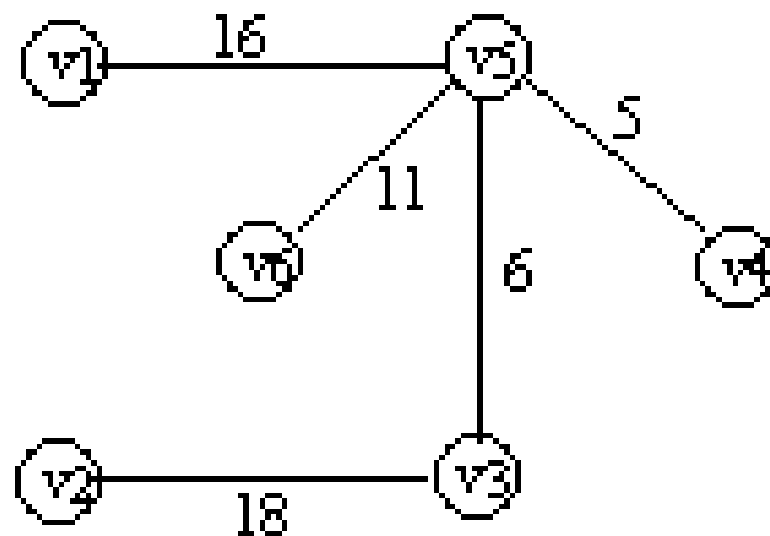
4.5 应用举例



最小生成树示例



(a) 图 G_7



(b) 图 G_7 的最小生成树

图 4-21 连通图和它的最小生成树示例

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

普里姆 (Prim) 算法

- 普里姆算法的基本思想:

从连通网络 $N = \{V, E\}$ 中的某一顶点 u_0 出发, 选择与它关联的具有最小权值的边 (u_0, v) , 将其顶点加入到生成树顶点集合 U 中。

以后每一步从一个顶点在集合 U 中, 而另一个顶点不在集合 U 中的各条边中选择权值最小的边 (u, v) , 把它的顶点加入到集合 U 中。如此继续下去, 直到网络中的所有顶点都加入到生成树顶点集合 U 中为止。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

普里姆 (Prim) 算法

选定构造最小生成树的出发顶点 u_0 ;

$V_{mst} = \{u_0\}$, $E_{mst} = \emptyset$;

while (V_{mst} 包含的顶点少于 n && E 不空) {

 从 E 中选一条边 (u, v) ,

$u \in V_{mst} \cap v \in V - V_{mst}$, 且具有最小代价(cost);

 令 $V_{mst} = V_{mst} \cup \{v\}$, $E_{mst} = E_{mst} \cup \{(u, v)\}$;

 将新选出的边从 E 中剔除: $E = E - \{(u, v)\}$;

}

if (V_{mst} 包含的顶点少于 n)

 cout << "不是最小生成树" << endl;

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

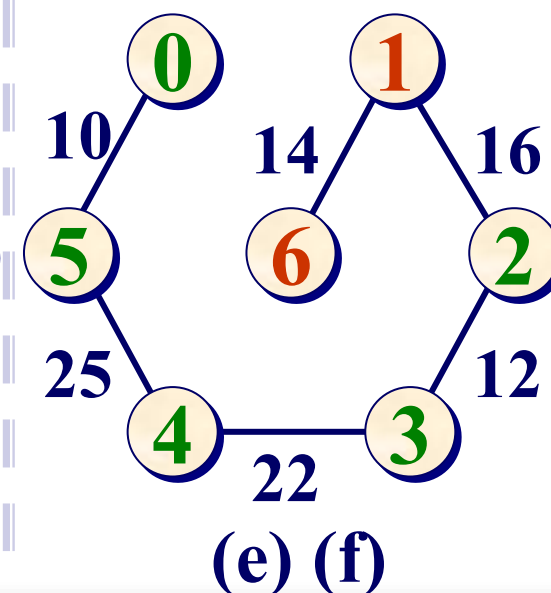
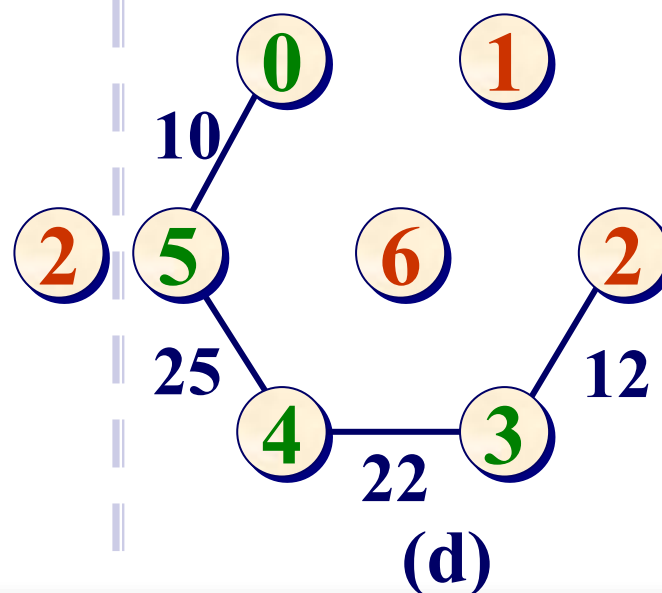
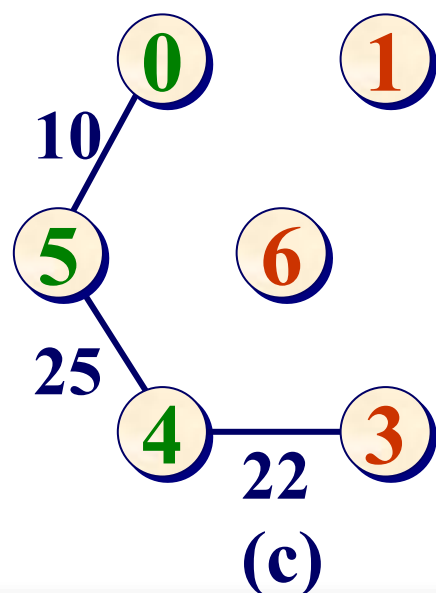
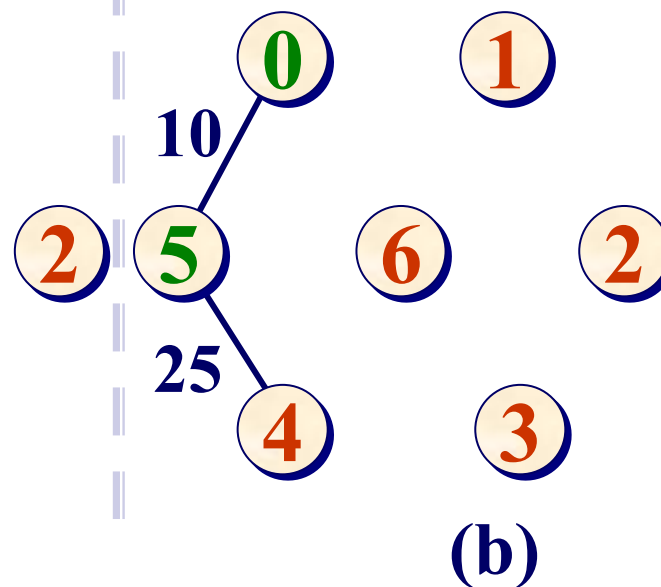
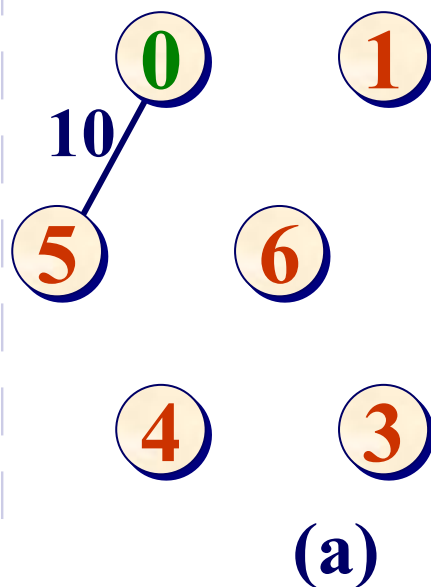
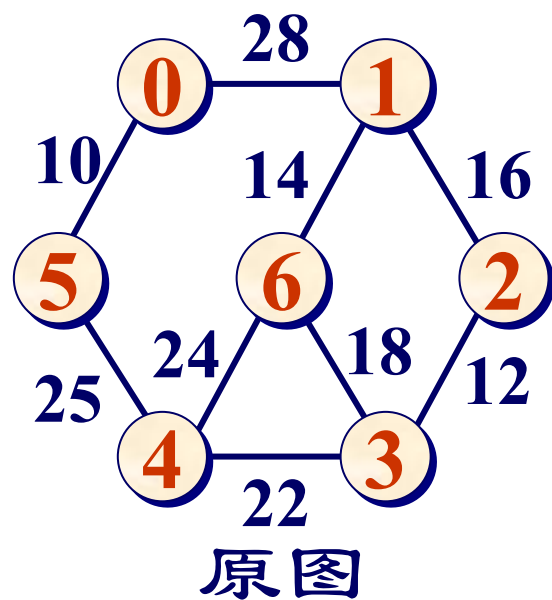
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

普里姆 (Prim) 算法



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

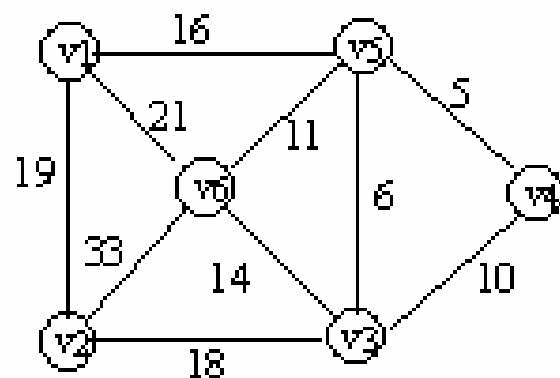
4.4 图结构

二、实例

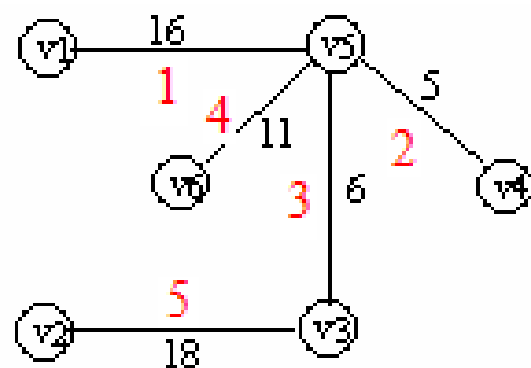
4.5 应用举例

举例

- 从顶点 v_1 开始, $U=\{v_1\}$
- 选择代价最小的边 (v_1, v_5) , 将 v_5 加入顶点集 $U=\{v_1, v_5\}$
- 选择代价最小边 (v_5, v_4) , 将 v_4 加入顶点集 $U=\{v_1, v_5, v_4\}$
- 选择代价最小边 (v_5, v_3) , 将 v_3 加入顶点集 $U=\{v_1, v_5, v_4, v_3\}$
- 选择代价最小边 (v_5, v_6) , 将 v_6 加入顶点集 $U=\{v_1, v_5, v_4, v_3, v_6\}$
- 选择代价最小边 (v_3, v_2) , 将 v_2 加入顶点集 $U=\{v_1, v_5, v_4, v_3, v_6, v_2\}$

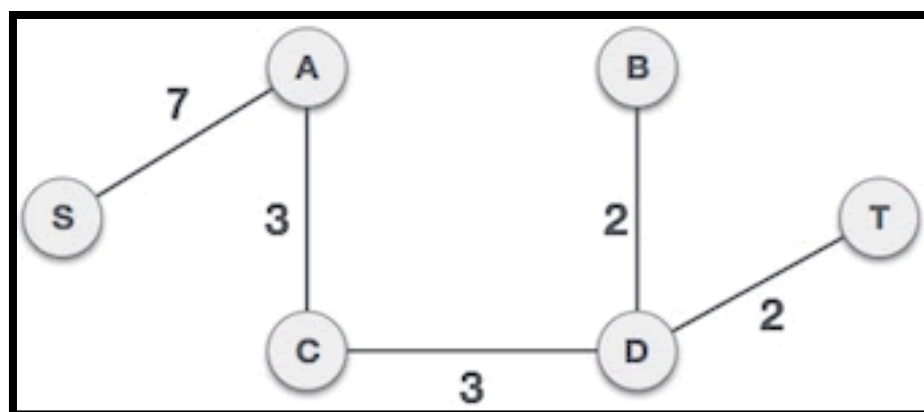
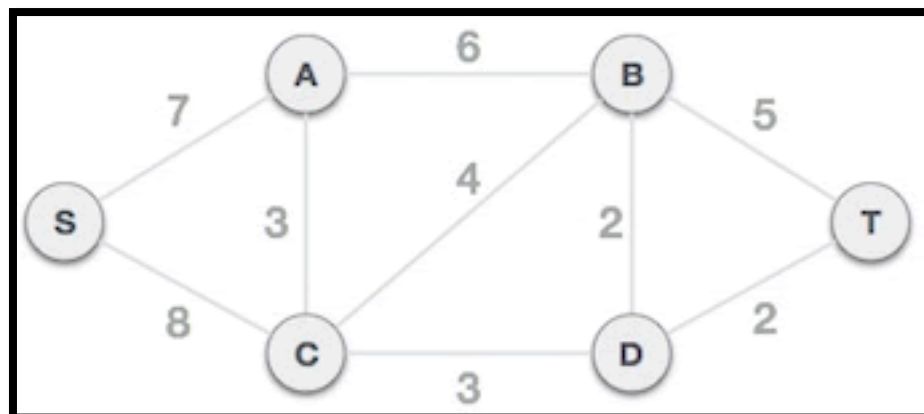


(a) 图 G_7



(b) 图 G_7 的最小生成树

例题



- 从顶点S开始, $U=\{S\}$
- 选择代价最小的边 (S,A) , 将A加入顶点集 $U=\{S,A\}$
- 选择代价最小边 (A,C) , 将C加入顶点集 $U=\{S,A,C\}$
- 选择代价最小边 (C,D) , 将D加入顶点集 $U=\{S,A,C,D\}$
- 选择代价最小边 (D,B) , 将B加入顶点集 $U=\{S,A,C,D,B\}$
- 选择代价最小边 (D,T) , 将T加入顶点集 $U=\{S,A,C,D,B,T\}$

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

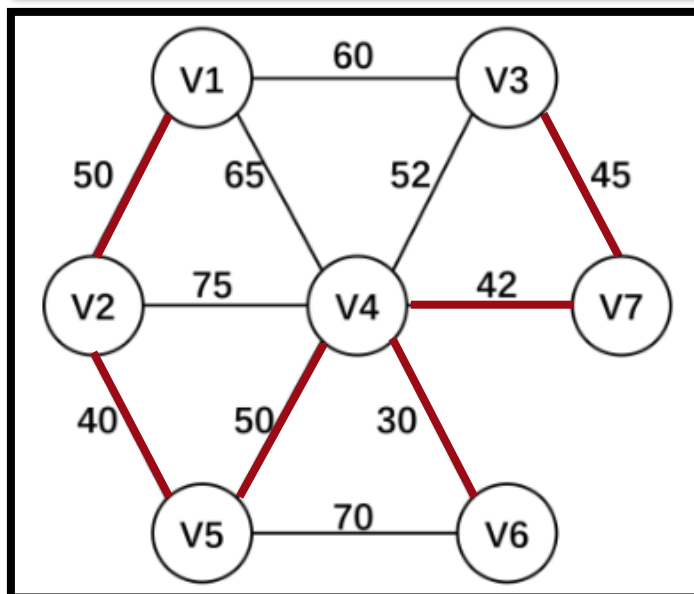
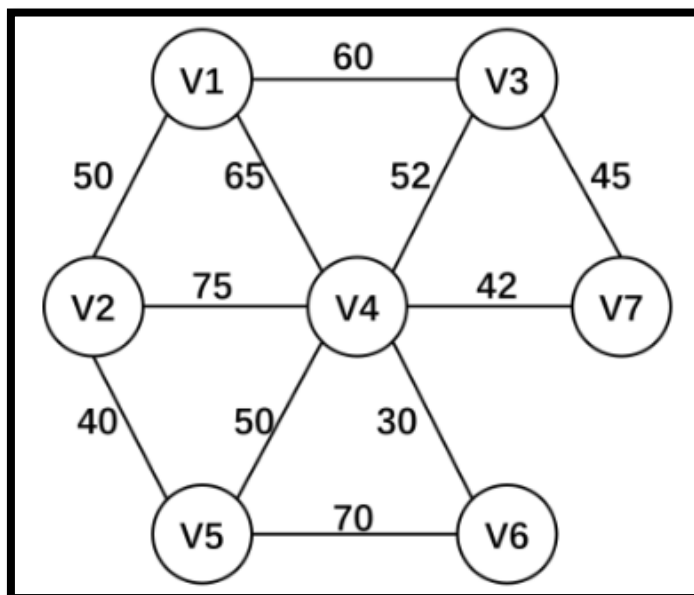
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

例题



- 从顶点 v_1 开始, $U=\{v_1\}$
- 选择代价最小的边 (v_1,v_2) , 将 v_2 加入顶点集 $U=\{v_1,v_2\}$
- 选择代价最小边 (v_2,v_5) , 将 v_5 加入顶点集 $U=\{v_1,v_2,v_5\}$
- 选择代价最小边 (v_5,v_4) , 将 v_4 加入顶点集 $U=\{v_1,v_2,v_5,v_4\}$
- 选择代价最小边 (v_4,v_6) , 将 v_6 加入顶点集 $U=\{v_1,v_2,v_5,v_4,v_6\}$
- 选择代价最小边 (v_4,v_7) , 将 v_7 加入顶点集 $U=\{v_1,v_2,v_5,v_4,v_6,v_7\}$
- 选择代价最小边 (v_7,v_3) , 将 v_3 加入顶点集 $U=\{v_1,v_2,v_5,v_4,v_6,v_7,v_3\}$

克鲁斯卡尔 (Kruskal) 算法

- 由美国计算机科学家约瑟夫·克鲁斯卡尔 (Joseph Bernard Kruskal) 于1956年发明
- 基本思想：对于具有 n 个顶点的图，将图中边按其权值由小到大的次序顺序选取，若选边后不形成回路，则保留作为一条边，若形成回路则除去该边。直到依次选够 $n-1$ 条边为止，即得到图的最小生成树。



克鲁斯卡尔 (Kruskal) 算法

```
 $T = \emptyset;$                                 //  $T$ 是最小生成树的边集合  
                                           //  $E$ 是带权无向图的边集合  
while (  $T$  包含的边少于  $n-1$  &&  $E$  不空) {  
    从  $E$  中选一条具有最小代价的边  $(v, w)$ ;  
    从  $E$  中删去  $(v, w)$ ;  
    如果  $(v, w)$  加到  $T$  中后不会产生回路, 则将  
         $(v, w)$  加入  $T$ ; 否则放弃  $(v, w)$ ;  
}  
if (  $T$  中包含的边少于  $n-1$  条)  
    cout << "不是最小生成树" << endl;
```

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

克鲁斯卡尔 (Kruskal) 算法

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

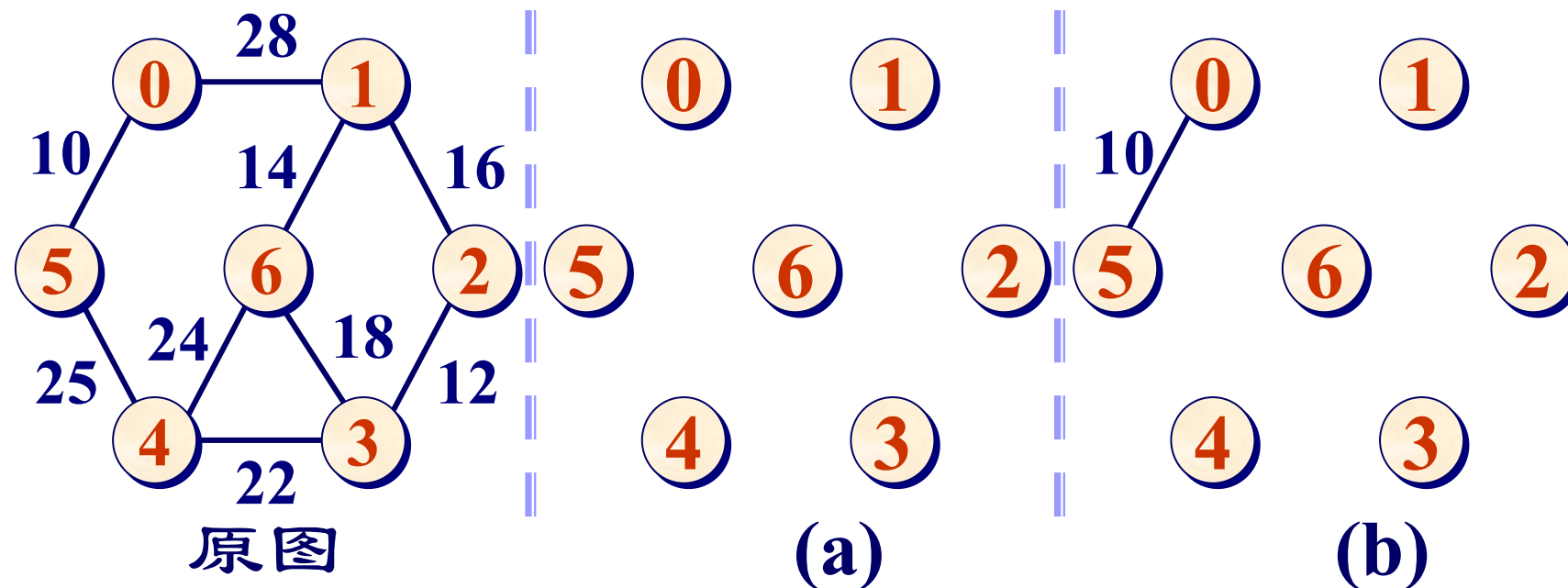
4.2 线性结构

4.3 树形结构

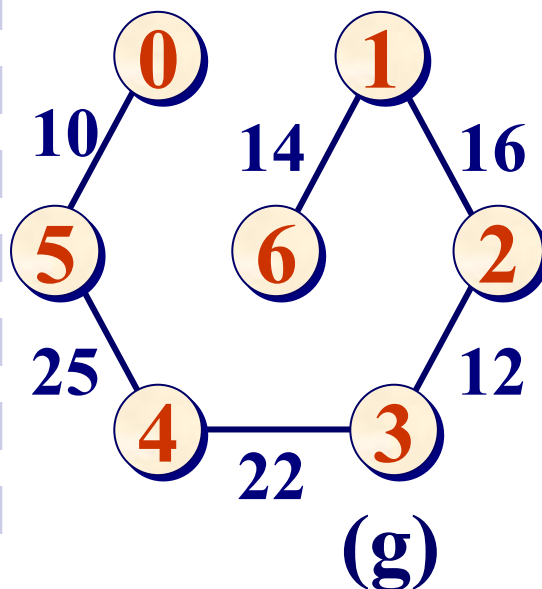
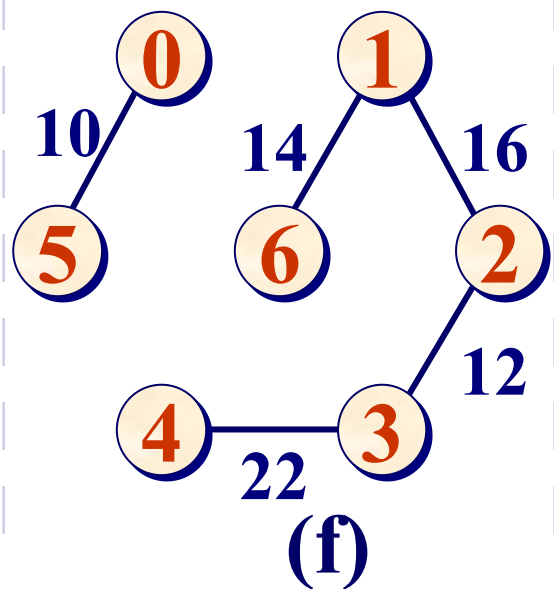
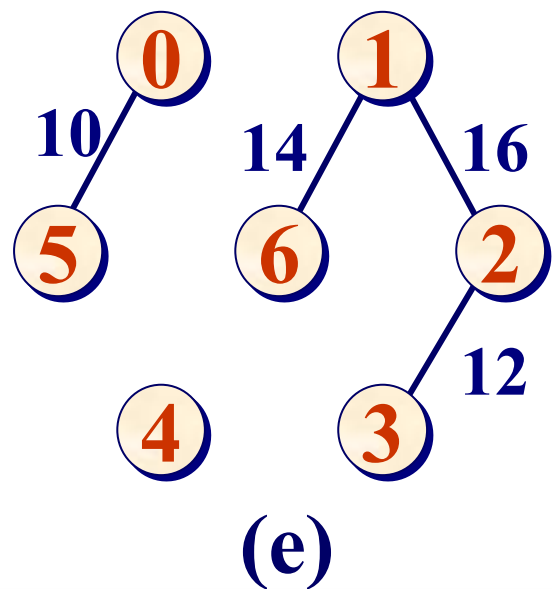
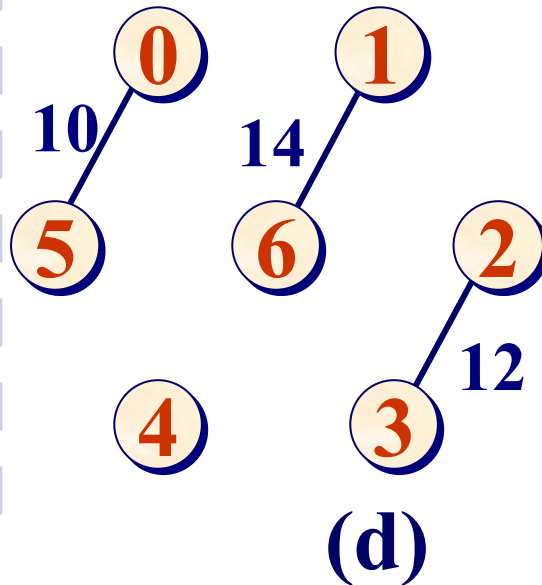
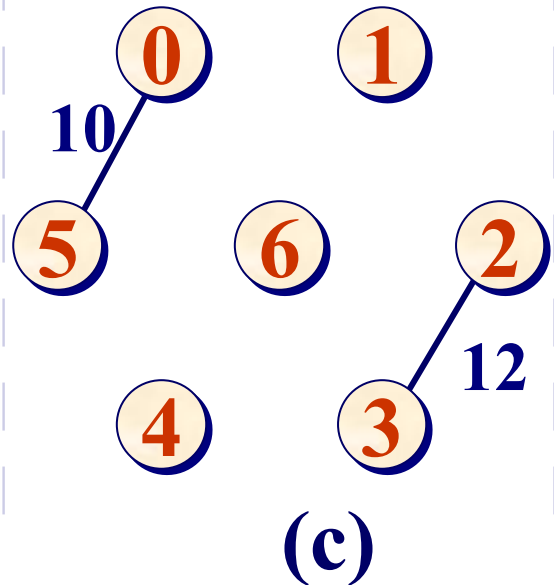
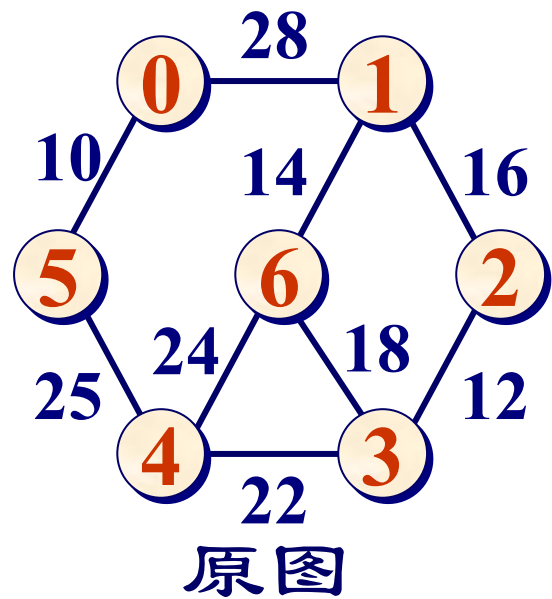
4.4 图结构

二、实例

4.5 应用举例



克鲁斯卡尔 (Kruskal) 算法



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

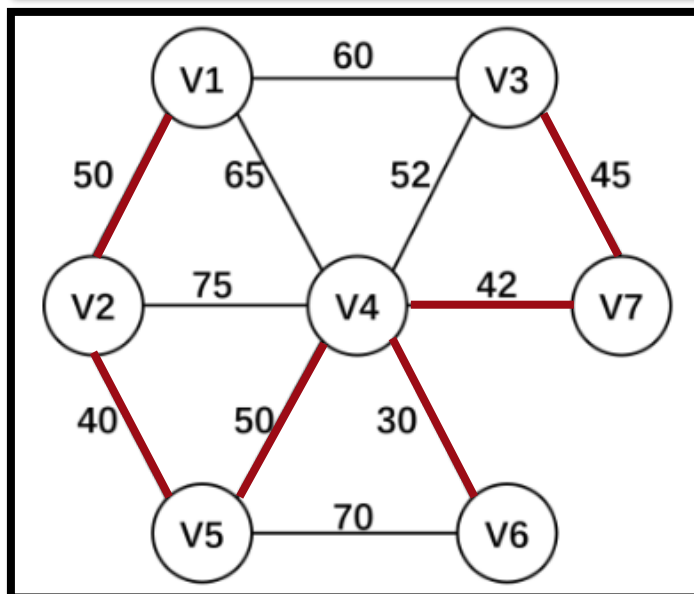
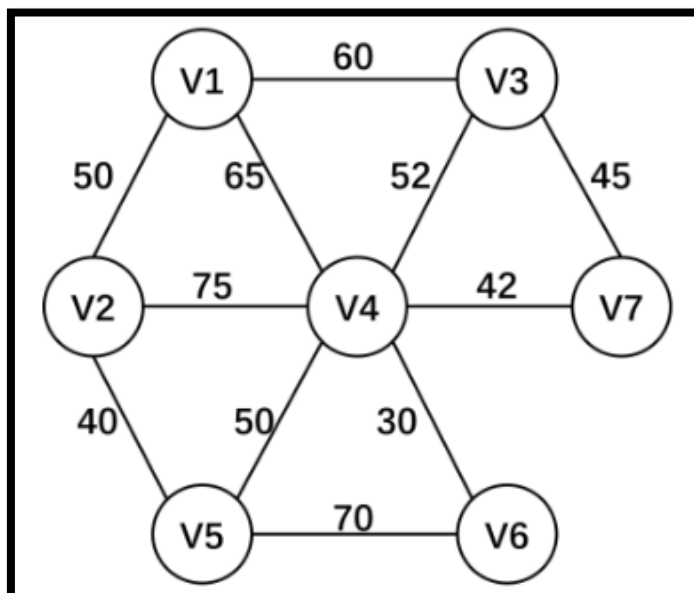
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

例题



- 选择最短边 $\{(V4,V6)\}$, 此时已连节点 $\{V4,V6\}$
- 从剩余边中选择不构成回路的最短边添加进来 $\{(V4,V6),(V2,V5)\}$, 此时已连节点 $\{V4,V6,V2,V5\}$
- $\{(V4,V6),(V2,V5),(V4,V7)\}$,此时已连节点 $\{V4,V6,V2,V5,V7\}$
- $\{(V4,V6),(V2,V5),(V4,V7),(V3,V7)\}$, 此时已连节点 $\{V4,V6,V2,V5,V7,V3\}$
- $\{(V4,V6),(V2,V5),(V4,V7),(V3,V7),(V1,V2),(V4,V5)\}$, 已经选够 $n-1$ 条边, 完成。



最小生成树演示

逐步演示: <https://visualgo.net/zh/mst>

视频动画:

<https://www.bilibili.com/video/BV1Te4y1Y7jR/>

<https://www.bilibili.com/video/BV1Eb41177d1/>

<https://www.bilibili.com/video/BV1wG411z79G/>

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例



4.5 应用举例

- 网络与最短通路问题
- 工程拓扑排序问题
- 工程关键路径问题

知识要点

最短路径，

Dijkstra算法，

AOV网络，

拓扑有序，

AOE网络，

关键路经

交通问题及其抽象

- 交通问题

- 顶点表示城市，边表示城市之间的道路，可以带权（表示道路的长度或沿道路行驶需要花费的时间、费用等）

- 问题

- 从城市A到城市B有通路吗？
- 若从城市A到城市B有多条通路，哪一条是最短的或者是最经济的呢？

- 抽象化

- 如果从图中某一顶点（称为源点）另一顶点（称为终点）的路径可能不止一条，如何找到一条路径使得沿此路径上各边上的权值总和达到最小
- 边上权值非负情形的单源最短路径问题——**Dijkstra算法**

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

单源最短路径问题

- **问题的提法：**给定一个带权有向图 D 与源点 v ，求从 v 到 D 中其他顶点的最短路径。
限定各边上的权值大于或等于0。
- 为求得这些最短路径，Dijkstra提出**按路径长度的递增次序，逐步产生最短路径的算法**。首先求出长度最短的一条最短路径，再参照它求出长度次短的一条最短路径，依次类推，直到从顶点 v 到其它各顶点的最短路径全部求出为止。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

Dijkstra算法

Step0: 设图有 n 个顶点, 对应的邻接矩阵为 $cost_{n \times n}$, 数组 $dist[1..n]$ 存储当前求得的起点 s 到各顶点的最短路径长度。集合 S 为已求得的最短路径的终点的集合, 初值为空。

求 $dist$ 的初值为: $dist[i] = w(s, v_i)$

Step1: 选择 v_j , 使得

$dist[v_j] = \min\{dist[i] \mid v_i \in V - S\}$

则, v_j 为从起点 s 出发的最短路径的终点。将 v_j 加入集合 S , 即:

$S = S \cup \{v_j\}$

Step2: 修改从顶点出发到集合 $V - S$ 中各顶点 v_k 的最短路径长度值 $dist[v_k]$

若 $dist[j] + w(v_j, v_k) < dist[k]$, 则修改 $dist[k]$ 为:

$dist[k] = dist[j] + w(v_j, v_k)$

Step3: 重复 Step1 和 Step2, 共 $n-1$ 次, 即可求得起点 s 到其余个顶点的最短路径

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

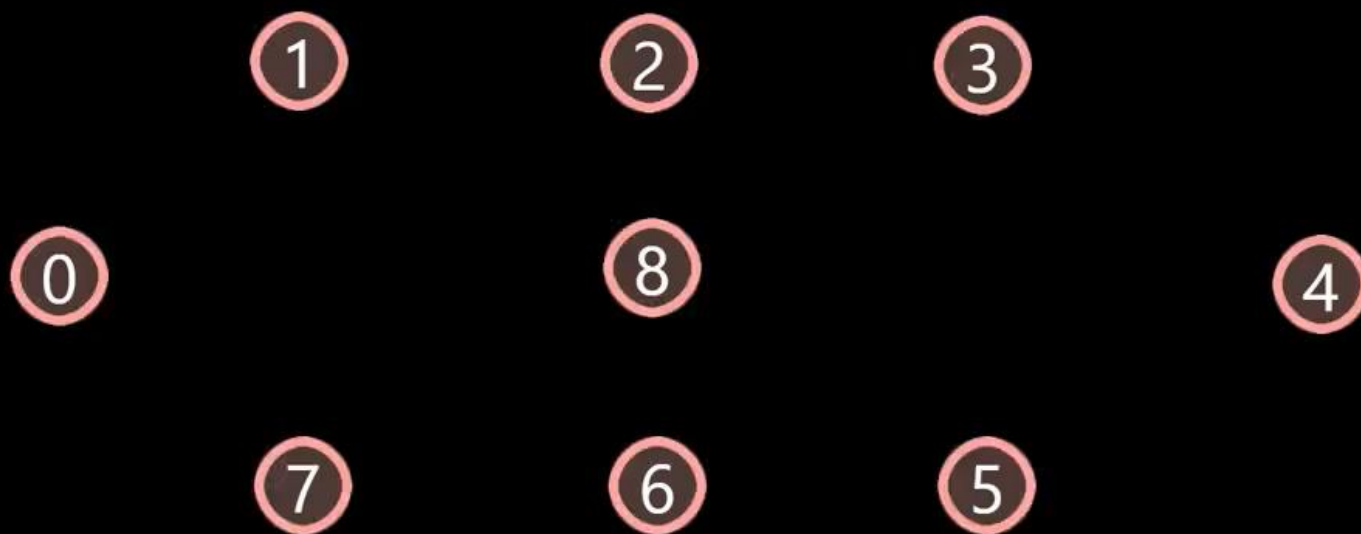
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

Dijkstra算法



这是一个由多个节点

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

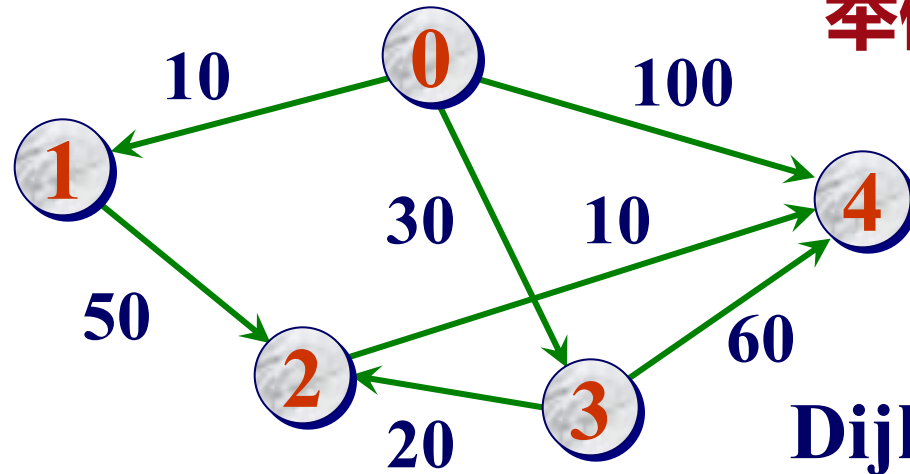
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

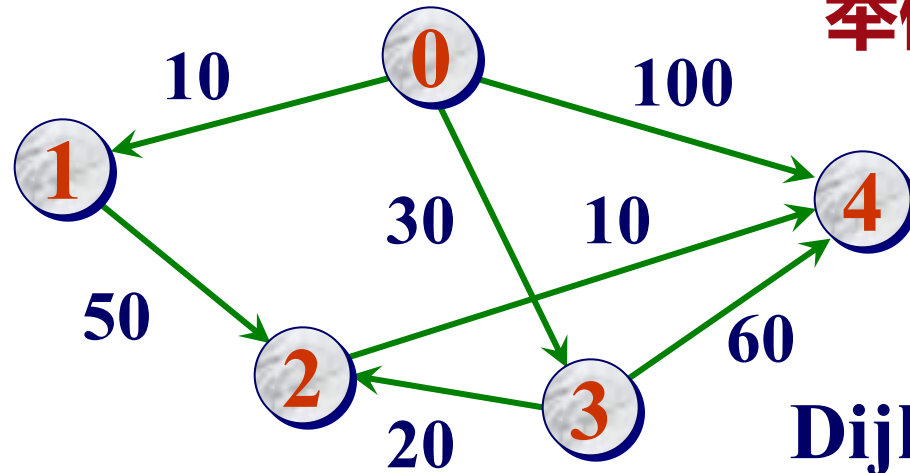
举例



Dijkstra逐步求解的过程

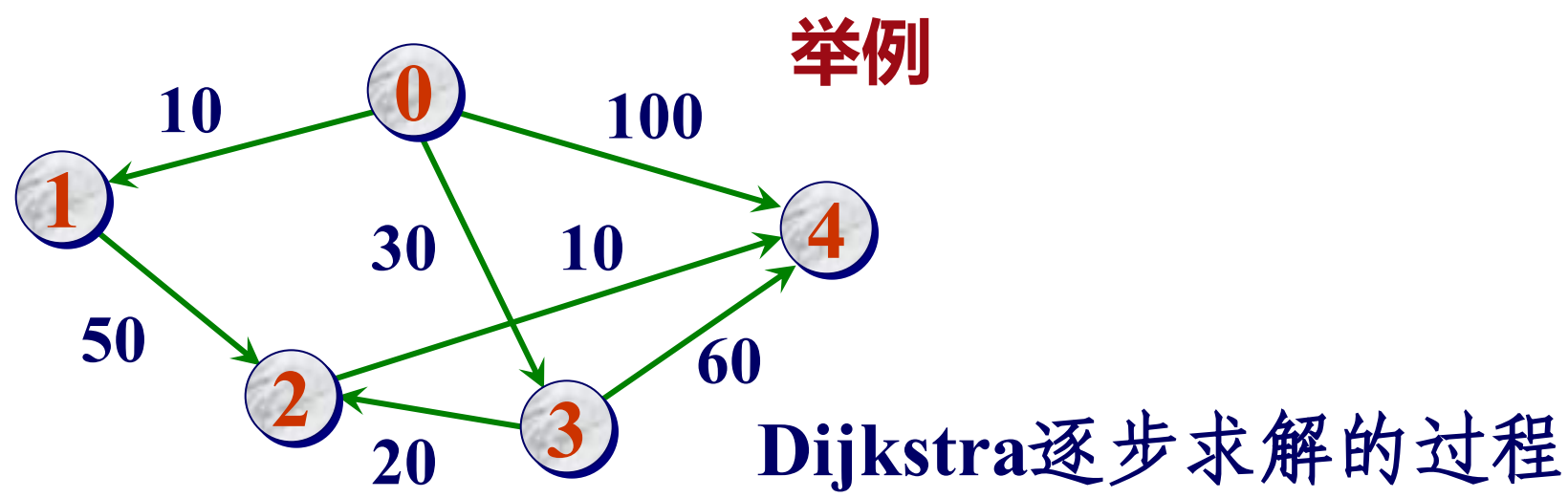
源点	终点	最短路径	路径长度
v_0	v_1	(v_0, v_1)	10 ← 最小
	v_2		∞
	v_3	(v_0, v_3)	30
	v_4	(v_0, v_4)	100

举例



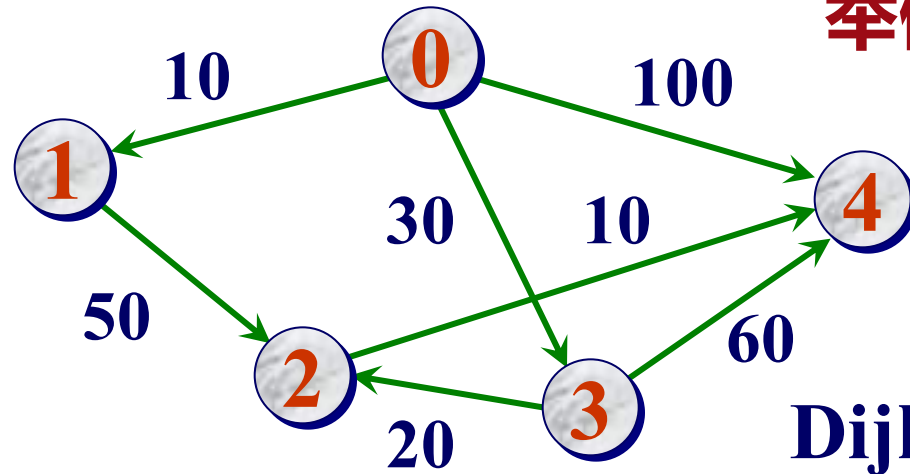
Dijkstra逐步求解的过程

源点	终点	最短路径	路径长度
v_0	v_1	(v_0, v_1)	10 ← 已标记
	v_2	(v_0, v_1, v_2) (v_0, v_3, v_2)	60 50
	v_3	(v_0, v_3)	30 ← 最小
	v_4	(v_0, v_4) (v_0, v_3, v_4)	100 90



源点	终点	最短路径	路径长度
v_0	v_1	(v_0, v_1)	10 ← 已标记
	v_2	(v_0, v_3, v_2)	50 ← 最小
	v_3	(v_0, v_3)	30 ← 已标记
	v_4	(v_0, v_3, v_4) (v_0, v_3, v_2, v_4)	90 60

举例



Dijkstra逐步求解的过程

源点	终点	最短路径	路径长度
v_0	v_1	(v_0, v_1)	10
	v_2	(v_0, v_1, v_2) (v_0, v_3, v_2)	$\infty, 60, 50$
	v_3	(v_0, v_3)	30
	v_4	(v_0, v_4) (v_0, v_3, v_4) (v_0, v_3, v_2, v_4)	100, 90, 60

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

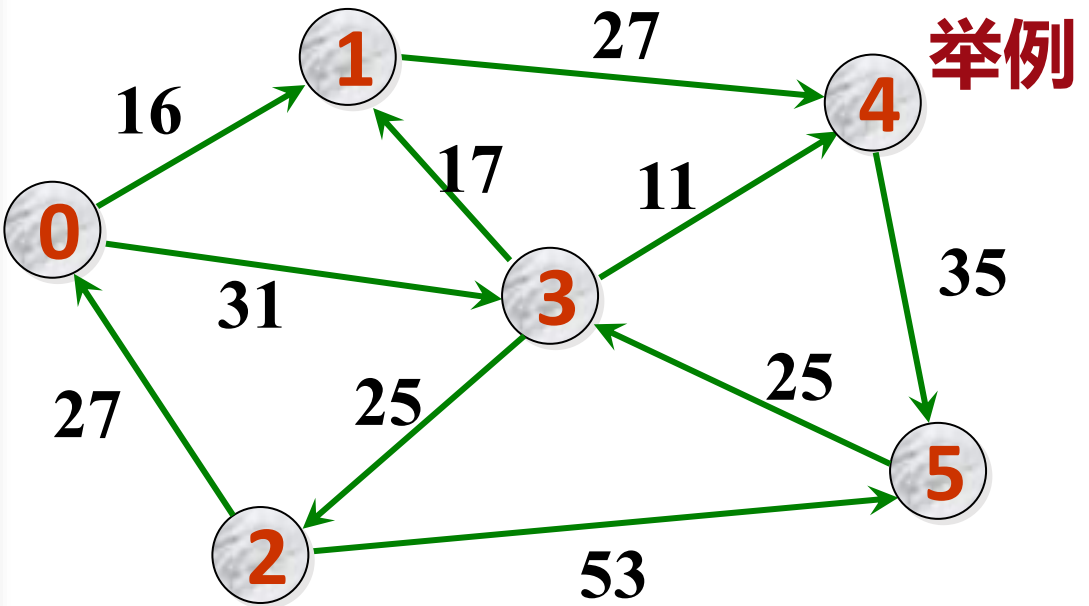
4.2 线性结构

4.3 树形结构

4.4 图结构

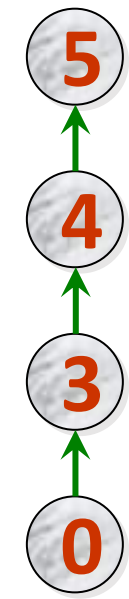
二、实例

4.5 应用举例



	v_0	v_1	v_2	v_3	v_4	v_5
v_0	0	16	∞	31	∞	∞
v_1	∞	0	∞	∞	27	∞
v_2	27	∞	0	∞	∞	53
v_3	∞	17	25	0	11	∞
v_4	∞	∞	∞	∞	0	35
v_5	∞	∞	∞	25	∞	0

轮次	v_0	v_1	v_2	v_3	v_4	v_5
1	0	16(v_0)	∞	31(v_0)	∞	∞
2	0	16(v_0)	∞	31(v_0)	43(v_1)	∞
3	0	16(v_0)	56(v_3)	31(v_0)	42(v_3)	∞
4	0	16(v_0)	56(v_3)	31(v_0)	42(v_3)	77(v_4)
5	0	16(v_0)	56(v_3)	31(v_0)	42(v_3)	77(v_4)



工程拓扑排序问题及其抽象

- 在工程项目或产品加工中，除了最简单的情况外，通常需要把一个复杂的工程分成若干个子工程，一个产品的加工分成若干加工工序。完成了这些子工程或加工工序则意味着整个工程的结束或产品制造的完成。而这些子工程或加工工序之间，通常受一定条件的约束，例如，某个子工程或加工工序的开始必须在另一些子工程或加工工序结束之后。
- 工程项目的施工或产品加工过程抽象为一个有向图，顶点表示各个子工程或加工工序，顶点之间的有向弧表示子工程或加工工序之间的约束，例如子工程之间的时间先后。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

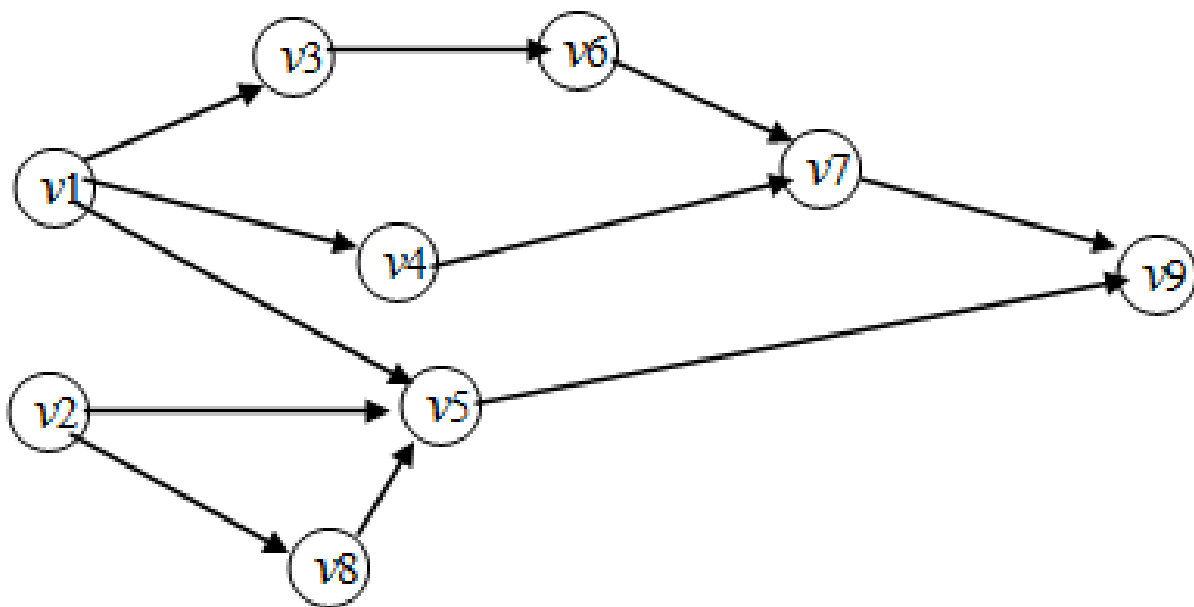
4.4 图结构

二、实例

4.5 应用举例

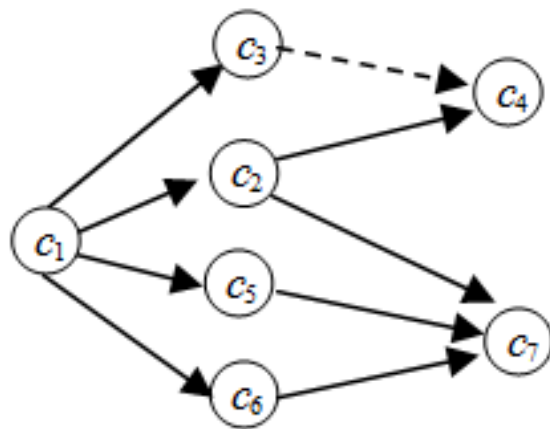
AOV网络

- 如果有向图为非赋权图，图中的顶点表示活动（Activity），表示一个工程的子工程或一个产品的加工工序，有向边表示活动之间的优先关系，这样的有向图称为AOV（activity on vertex）网络。



举例—课程编列

- 课程编列是指不同课程之间学习顺序的安排。课程之间学习顺序同样可以用AOV网结构来表达，其中顶点表示课程，顶点之间的有向弧表达课程之间的学习顺序。例如，如果存在一条有向弧 $\langle c_i, c_j \rangle$ ，则表明课程 c_i 是课程 c_j 的先导课程，换句话说，课程 c_j 是课程 c_i 的后续课程。
- 在高等学校计算机基础教学领域，计算机基础教学包括7门主要课程，其教学编列如图



c_1 :大学计算机基础,

c_2 :计算机程序设计

c_3 :计算机硬件基础,

c_4 :计算机软件基础

c_5 :数据库技术及应用,

c_6 :多媒体技术及应用

c_7 :计算机网络技术及应用

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

举例—课程编列

课程代号

课程名称

先修课程

C₁

高等数学

C₂

程序设计基础

C₃

离散数学

C₁, C₂

C₄

数据结构

C₃, C₂

C₅

高级语言程序设计

C₂

C₆

编译方法

C₅, C₄

C₇

操作系统

C₄, C₉

C₈

普通物理

C₁

C₉

计算机原理

C₈

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

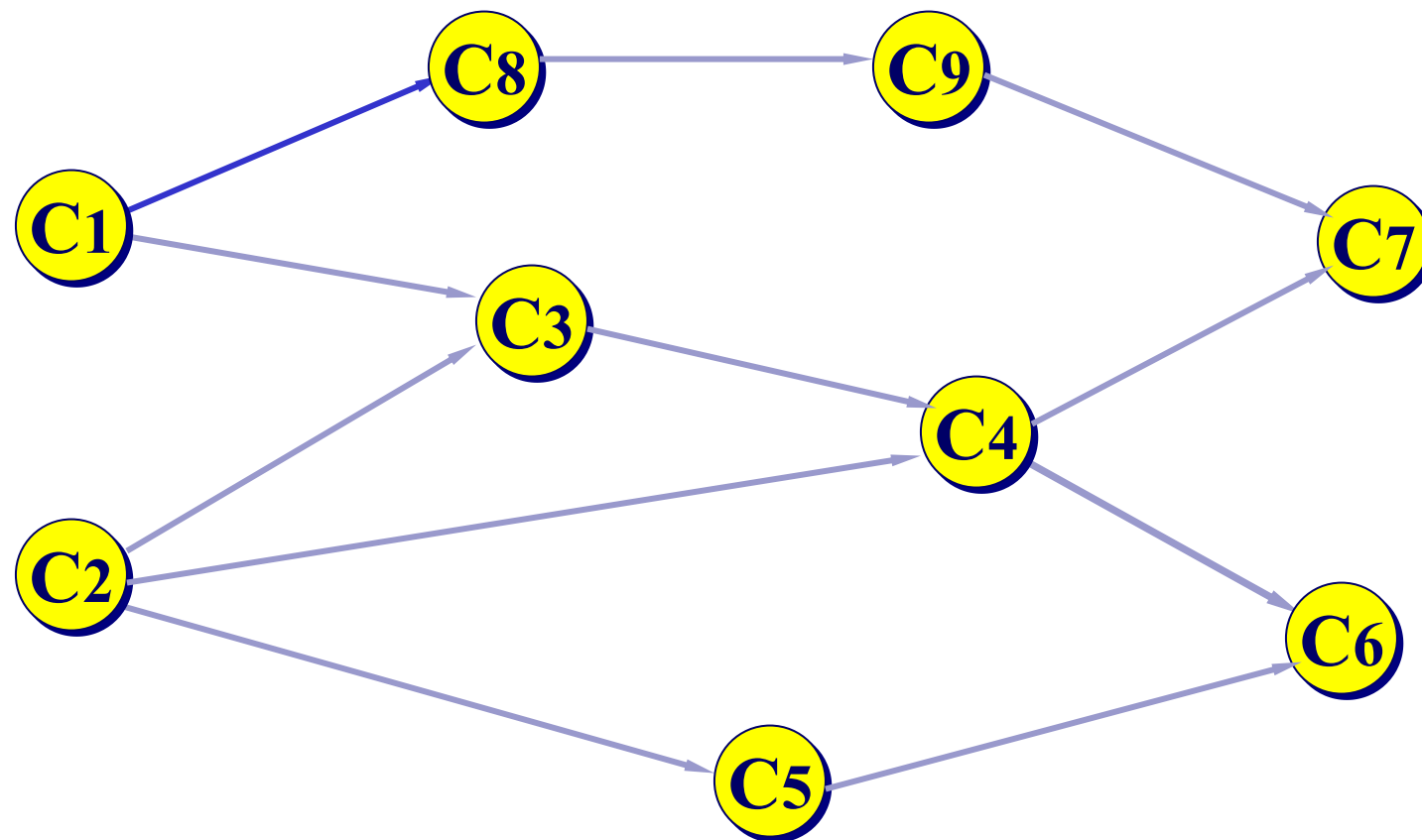
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

举例—课程编排



学生课程学习工程图

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

拓扑排序

- 在AOV网络中不能出现有向回路, 即有向环。如果出现了有向环, 则意味着某项活动应以自己作为先决条件。
- 因此, 对给定的AOV网络, 必须先判断它是否存在有向环。
- 检测有向环的一种方法是对AOV网络构造它的拓扑有序序列。
- 所谓拓扑排序, 就是产生这样的顶点序列: 若网络中, v_i 是 v_j 的前驱, 则在序列中, v_i 在 v_j 之前, 有这种特性的序列称为拓扑有序。
- 如果通过拓扑排序能将AOV网络的所有顶点都排入一个拓扑有序的序列中, 则该网络中必定不会出现有向环。
- 如果AOV网络中存在有向环, 此AOV网络所代表的工程是不可行的。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

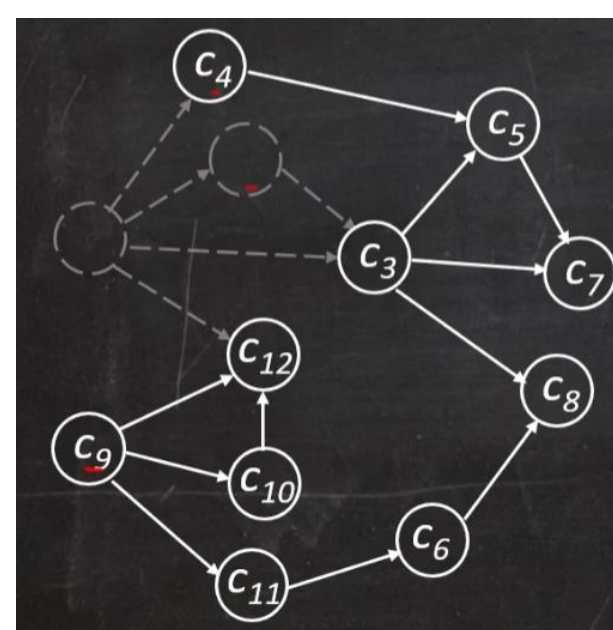
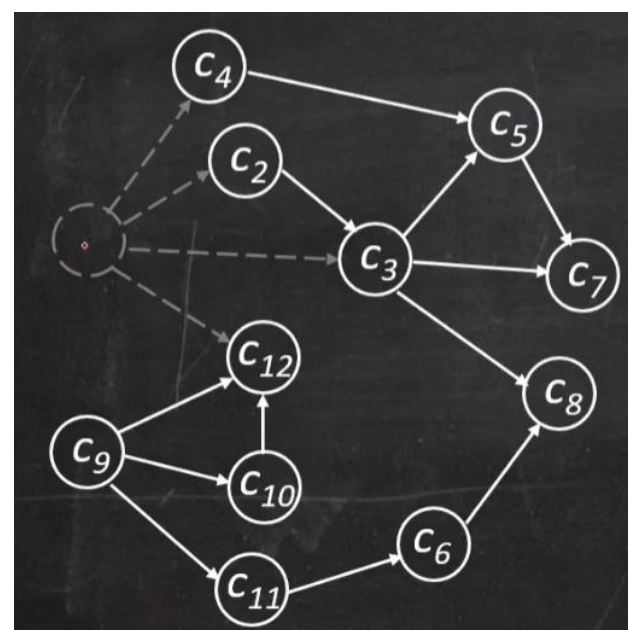
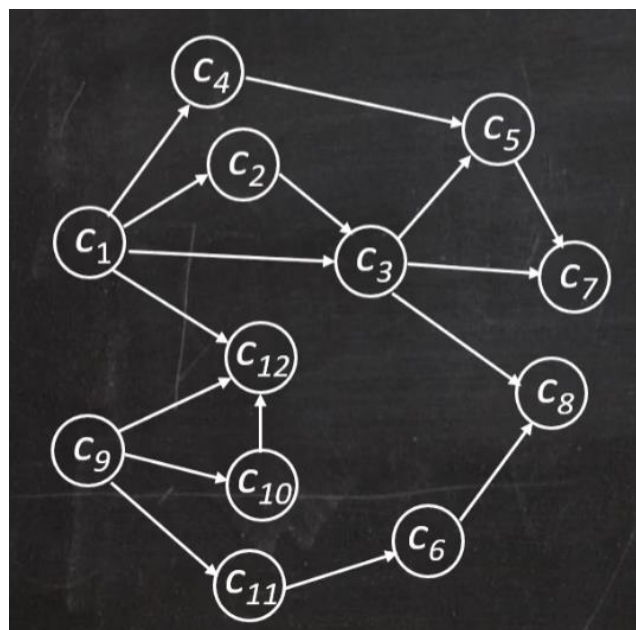
4.4 图结构

二、实例

4.5 应用举例

拓扑排序

1. 找一个没有前驱的顶点，以该顶点为开始的边全部删掉，并将该顶点插入序列；
2. 依次类推，重复以上步骤（当出现多个没有前驱的节点时，可以随机选取或按照索引大小进行step1）。



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

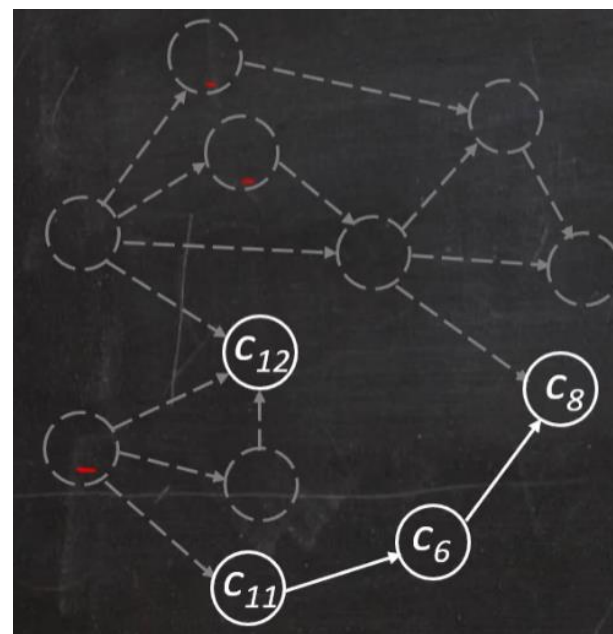
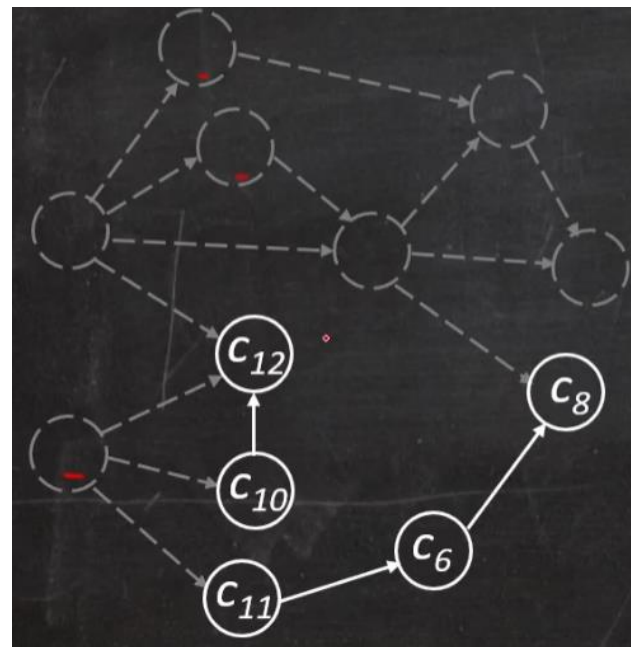
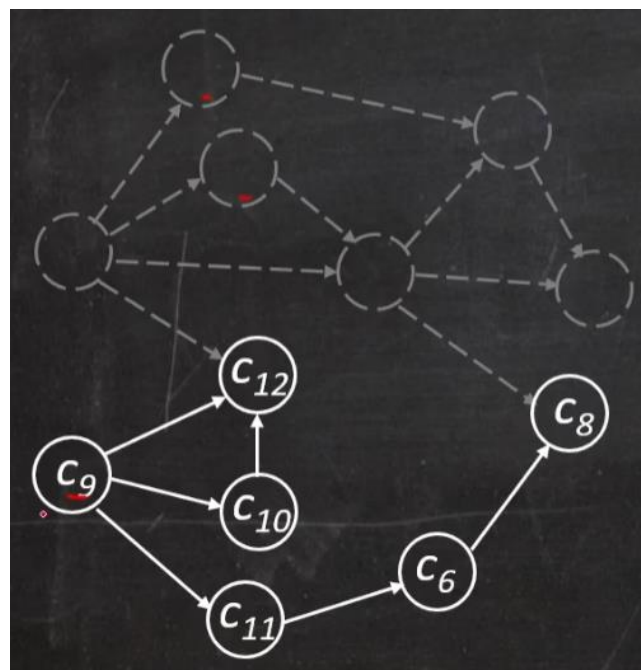
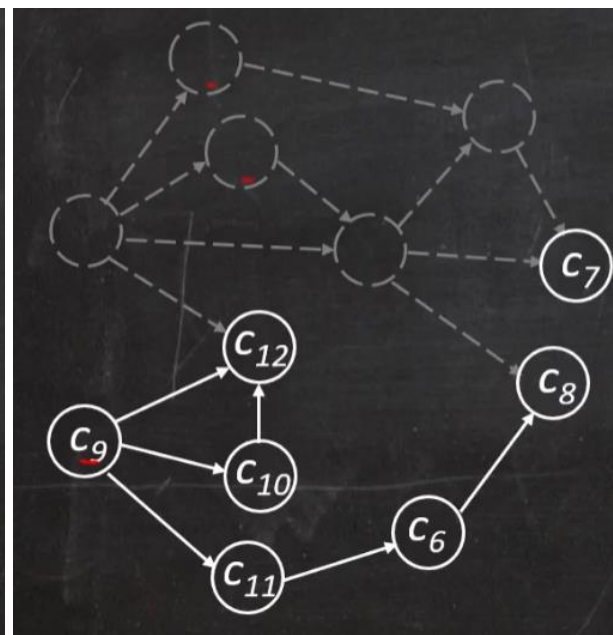
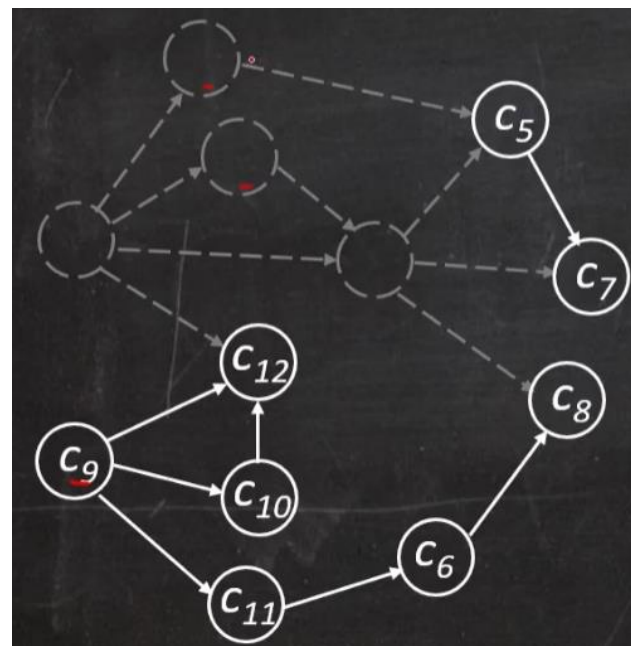
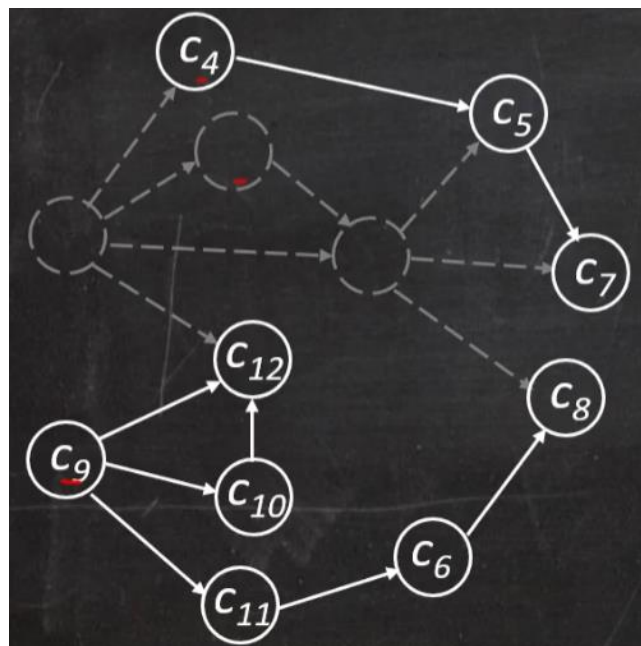
4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

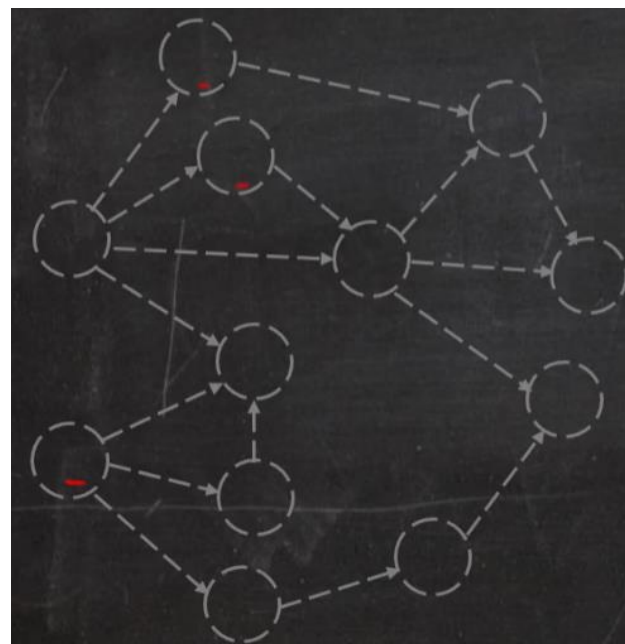
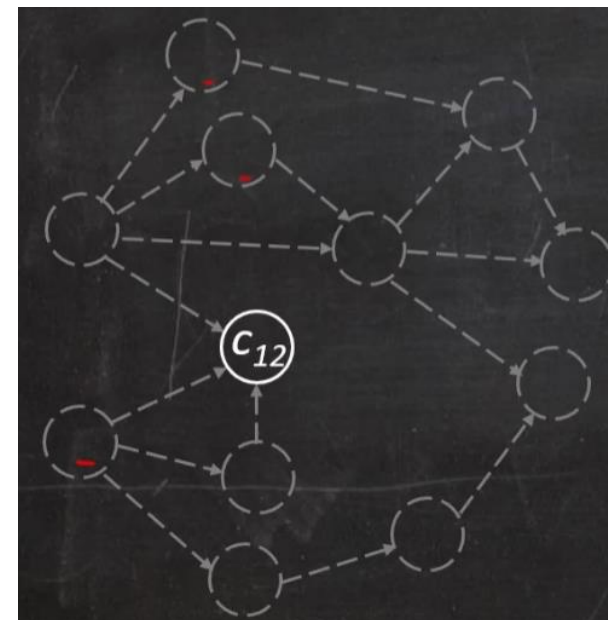
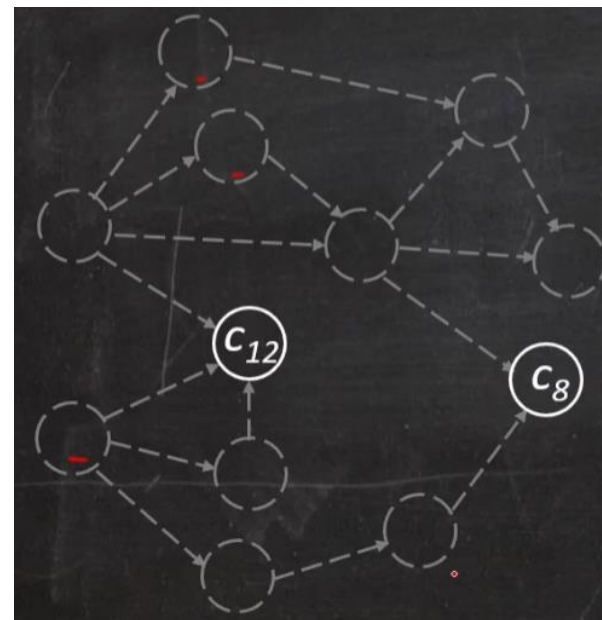
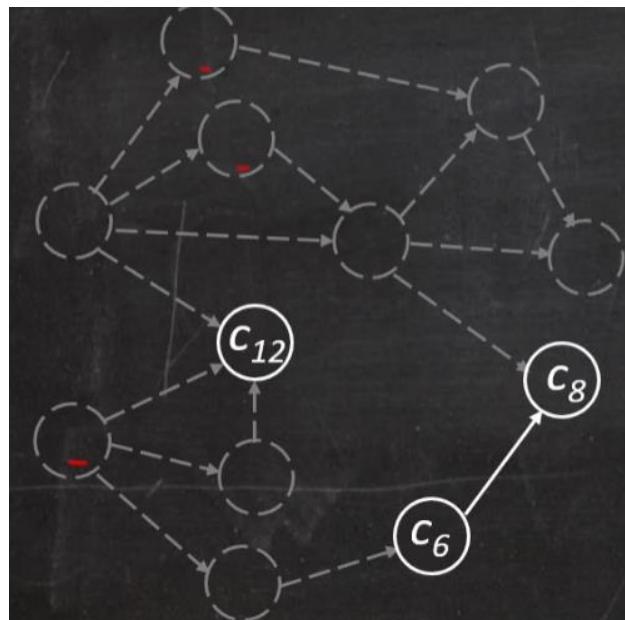
4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

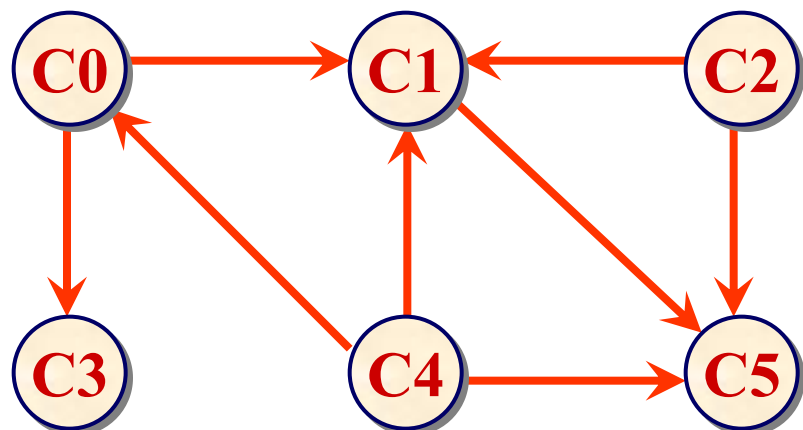


拓扑序列:

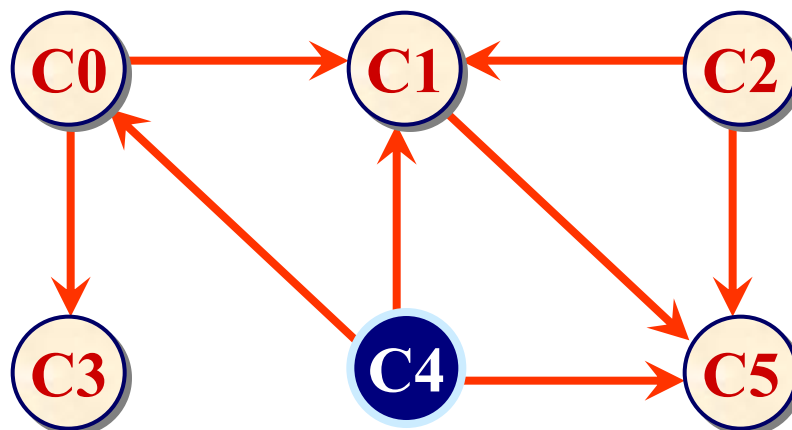
C_1 C_2 C_3 C_4 C_5 C_7 C_9 C_{10} C_{11} C_6 C_8 C_{12}

如果全部元素都可以按照该方法删除，则该图拓扑有序。

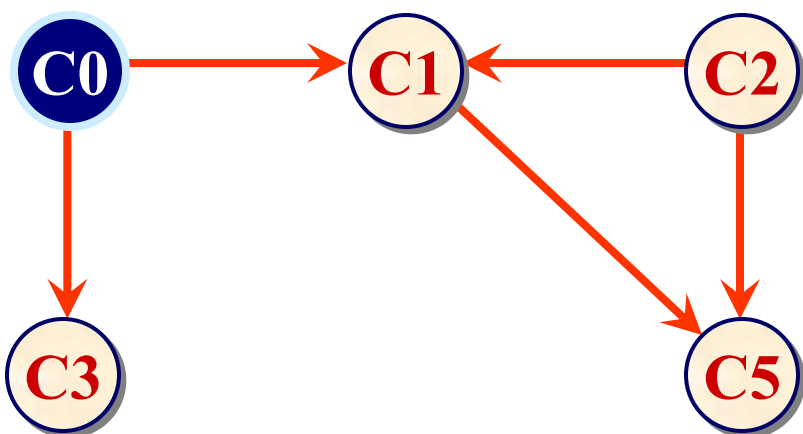
拓扑排序



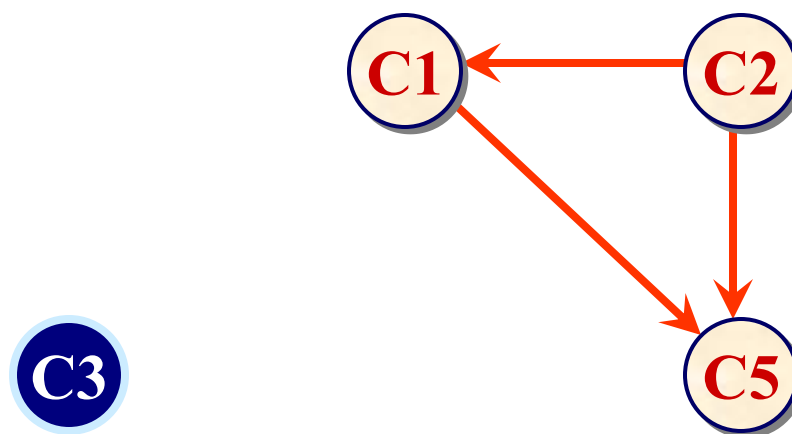
(a) 有向无环图



(b) 输出顶点 C4

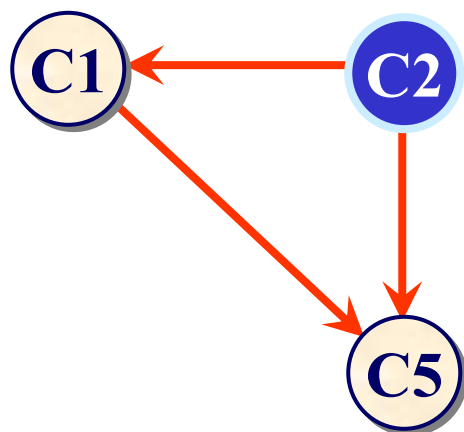


(c) 输出顶点 C0

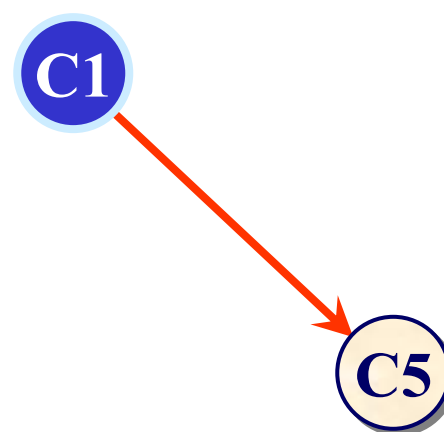


(d) 输出顶点 C3

拓扑排序



(e) 输出顶点 C2



(f) 输出顶点 C1



(g) 输出顶点 C5

(h) 拓扑排序完成

- 最后得到的拓扑有序序列为 $C_4, C_0, C_3, C_2, C_1, C_5$ 。它满足图中给出的所有前驱和后继关系，对于本来没有这种关系的顶点，如 C_4 和 C_2 ，也排出了先后次序关系。

工程关键路径问题及其抽象

- 工程问题
 - 项目和产品能否顺利实施
 - 工程的工期和产品的生产周期
- AOE (Activity On Edge) 网络
 - 图的顶点表示事件, 有向边表示活动, 权表示活动持续的时间
 - 为了描述问题的需要, 还可以增加持续时间为0的虚拟活动
- AOE网络在某些工程估算方面非常有用。例如, 可以使人们了解:
 - 完成整个工程至少需要多少时间(假设网络中没有环)?
 - 为缩短完成工程所需的时间, 应当加快哪些活动?

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

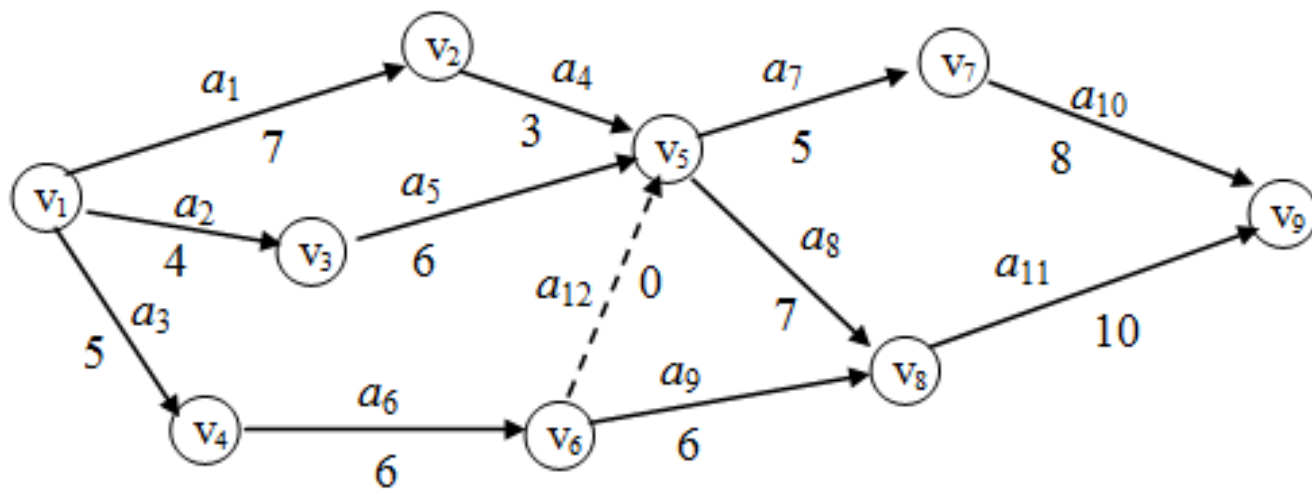
4.4 图结构

二、实例

4.5 应用举例

AOE网络与关键路径

- 从源点到各个顶点, 以至从源点到汇点的有向路径可能不止一条。这些路径的长度也可能不同。完成不同路径的活动所需的时间虽然不同, 但只有各条路径上所有活动都完成了, 整个工程才算完成。
- 因此, 完成整个工程所需的时间取决于从源点到汇点的最长路径长度, 即在这条路径上所有活动的持续时间之和。这条路径长度最长的路径就叫做**关键路径(Critical Path)**。



数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

事件与活动的最早和最迟发生时间

• 事件

- 最早发生时间 $Ve(j)$ ：从源点 V_0 到顶点 V_i 的最长路径长度。
- 最迟发生事件 $Vl(j)$ ：在保证汇点 V_{n-1} 在 $Ve[n-1]$ 时刻完成的前提下，事件 V_i 的允许的最迟开始时间。

• 活动

- 最早发生时间 $Ae(i)$ ：设活动 A_k 在边 $\langle V_i, V_j \rangle$ 上，则 $Ae[k]$ 是从源点 V_0 到顶点 V_i 的最长路径长度。因此， $Ae[k] = Ve[i]$ 。
- 最迟发生时间 $Al(i)$ ： $Al[k]$ 是在不会引起时间延误的前提下，该活动允许的最迟开始时间。 $Al[k] = Vl[j] - dur(\langle i, j \rangle)$ 。其中， $dur(\langle i, j \rangle)$ 是完成 ak 所需的时间。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

事件的最早和最迟发生时间

(1) 设起点的最早时间 $ve(0)=0$ ，从 v_0 开始沿**正向递推**，可依次求得各事件的最早发生时间，递推公式如下：

$$ve(j) = \max\{ve(i) + t(< i, j >) | < i, j > \in T, 2 \leq j \leq n\},$$

其中 T 是所有以 j 为头的边的集合（多路径选最大值）。

(2) 最后一个事件的最早发生时间与最迟发生时间一致，从 $vl(n) = ve(n)$ 起，**反向递推**，可求各事件的最迟发生时间，递推公式如下：

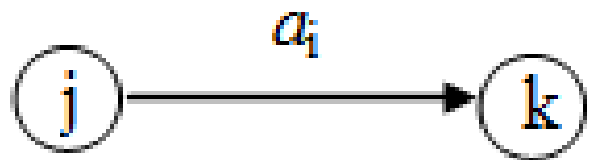
$$vl(i) = \min\{vl(j) - t(< i, j >) | < i, j > \in S, 1 \leq i \leq n - 1\}$$

其中 S 是所有以 i 为尾的边的集合。（多路径选最小值）

概括： ve 是从**起点到该点所有路径中最长的值**；

vl 是**终点的 ve 值减去该点到终点所有路径中的最长的值**。

活动的最早和最迟发生时间



(a) 活动 a_i

- 根据AOE网络中活动和事件的约束关系：

$$ae(i) = ve(j)$$

- 活动 a_i 的最早发生时间是节点 j 的最早发生时间

$$al(i) = vl(k) - t(< j, k >)$$

- 顶点 k 的最迟发生时间- $< j - k >$ 的活动时间即为活动 a_i 的最迟发生时间

- 所有 $ae(i) = al(i)$ 的活动所构成的路径就是**关键路径**。

概括：

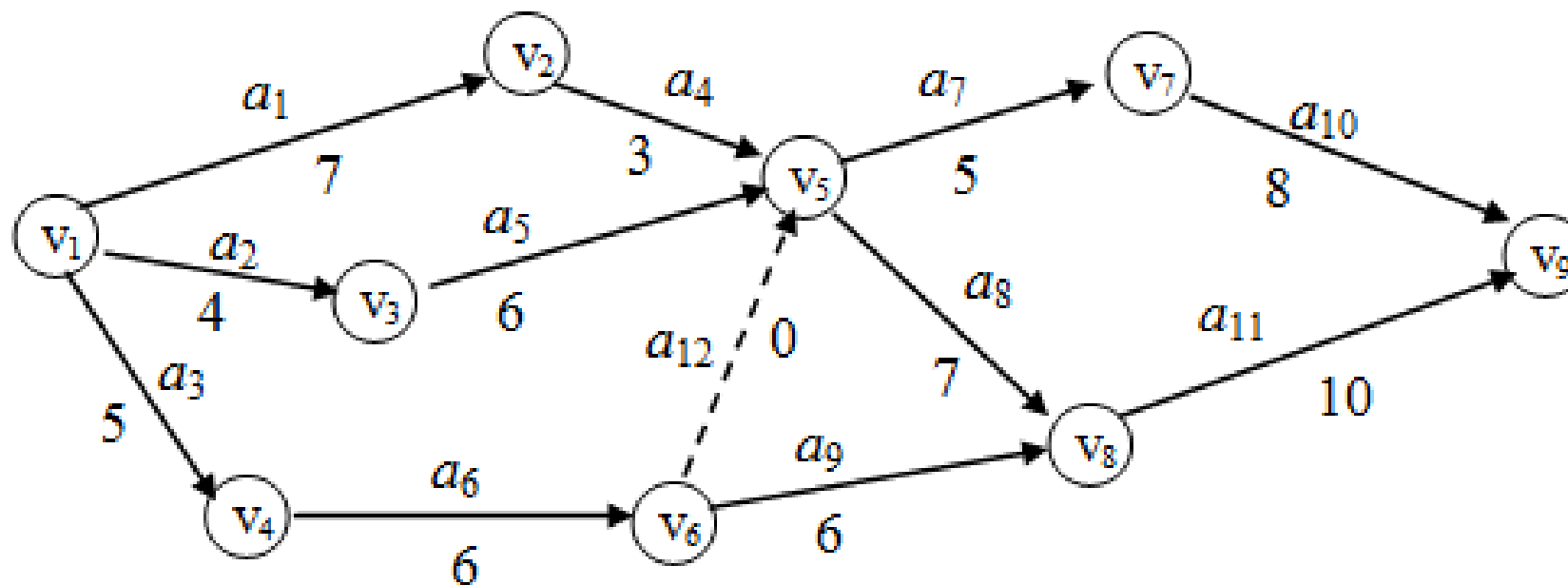
ae是**该边起点对应的ve值**；

al是该**边终点的vl值**减去**该边的值**。

举例

ve 是从起点到该点所有路径中最长的值;
 vl 是终点的 ve 值减去该点到终点所有路径中的最长的值。

• AOE网络



顶点	v1	v2	v3	v4	v5	v6	v7	v8	v9
ve	0	7	4	5	11	11	16	18	28
vl	0	8	5	5	11	11	20	18	28

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

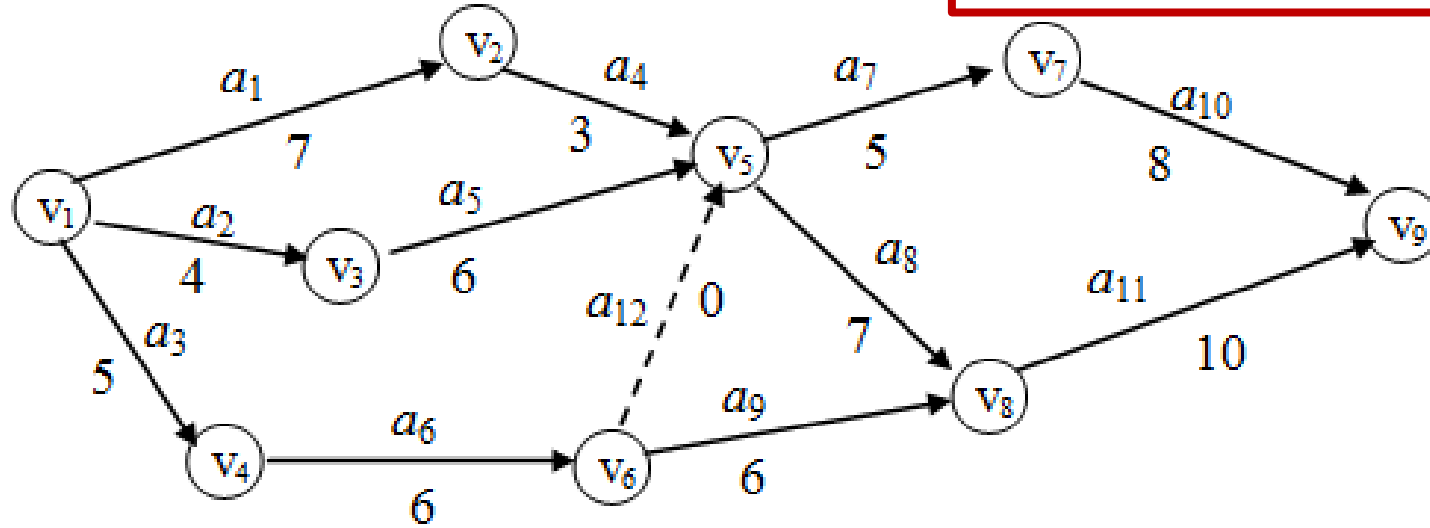
4.4 图结构

二、实例

4.5 应用举例

举例

ae是该边起点对应的ve值;
al是该边终点的vl值减去该边的值。



事件	v1		v2	v3	v4	v5		v6	v7		v8	v9
ve	0		7	4	5	11		11	16		18	28
vl	0		8	5	5	11		11	20		18	28
活动	a1	a2	a3	a4	a5	a6	a7	a8	a9	a10	a11	a12
ae	0	0	0	7	4	5	11	11	11	16	18	11
al	1	1	0	8	5	5	15	11	12	20	18	11

关键路径

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

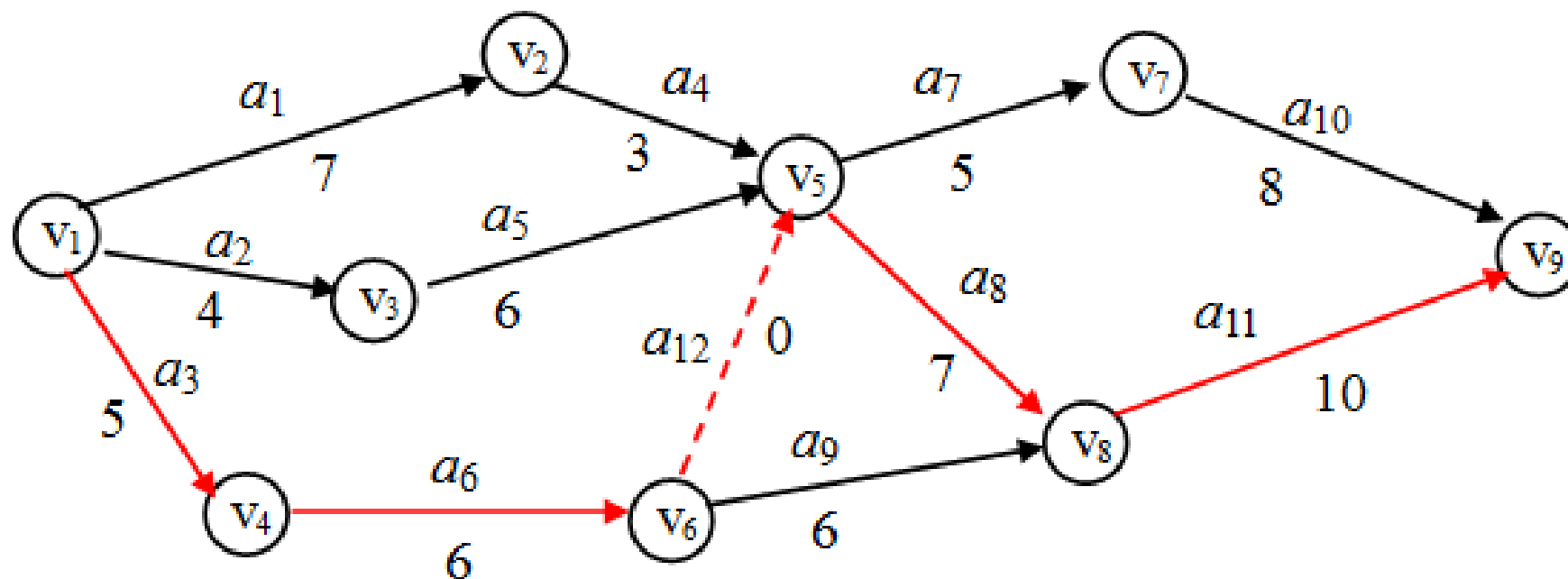
4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例





说明--1

- 在求顶点 v_5 的最早发生时间 $ve(5)$ 时，由于存在虚拟活动 a_{12} (对应弧 $\langle v_6, v_5 \rangle$)，因此，要考察 v_2 、 v_3 和 v_6 三个顶点。虚拟活动虽然时间为0，但它改变着工程的启动顺序，因此，在上述AOE网络中，长度最长的路径是 $(v_1, v_4, v_6, v_5, v_8, v_9)$ ，而不是 $(v_1, v_4, v_6, v_8, v_9)$ 。
- 活动的最早和最迟发生时间如果不相同，说明该活动的启动时间可以适当推迟，而不影响整个工程的进度。

说明--2

- 在关键路径上，各活动的最早开始时间和最迟开始时间相等，没有时间余量。
- 通常情况下，缩短关键活动的工期，可以缩短整个工程的工期。但这不是绝对的，因为改变某个活动的权值，将直接影响整个AOE网络中各个事件的最早和最迟发生时间的计算，可能使的关键路径发生变化。如果活动不再是关键路径上的活动，缩短该活动的时间无法缩短整个工程的工期。

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

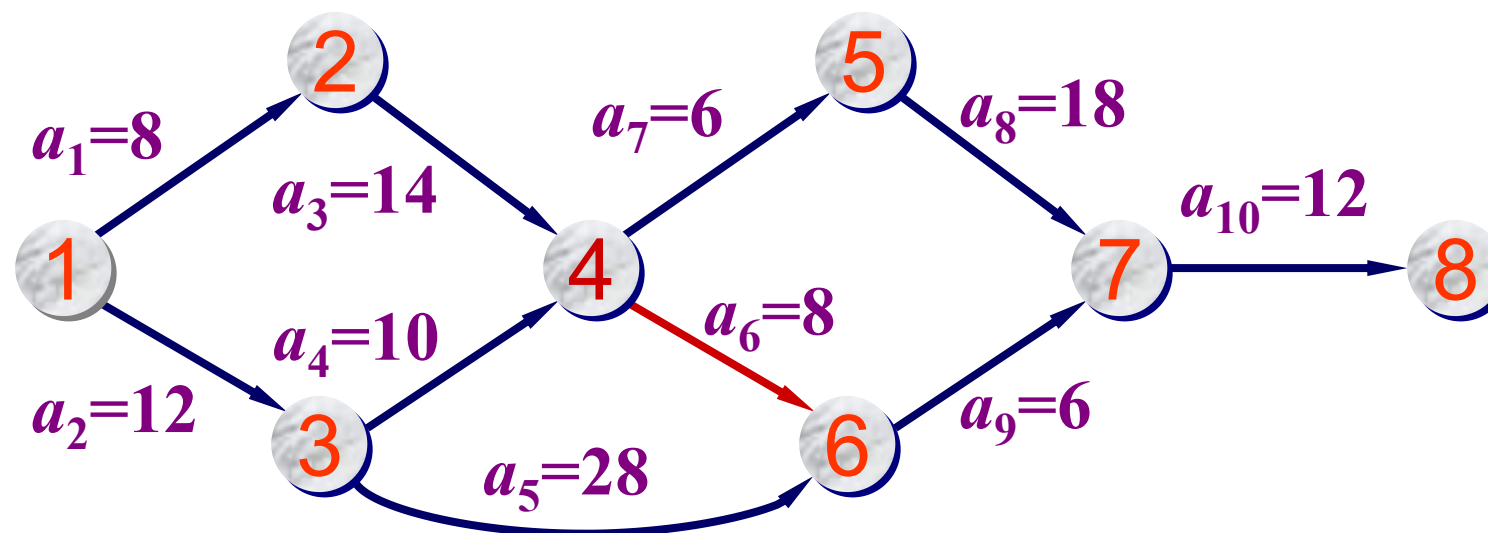
4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

例题



	1	2	3	4	5	6	7	8
V_e	0	8	12	22	28	40	46	58
V_l	0	8	12	22	28	40	46	58

	1	2	3	4	5	6	7	8	9	10
A_e	0	0	8	12	12	22	22	28	40	46
A_l	0	0	8	12	12	32	22	28	40	46

AOV网和AOE网的应用

- **AOV网（拓扑排序）适合任务依赖性分析**，帮助规划任务顺序。
 - 处理工程项目中的模块依赖关系
 - 规划合理的课程安排
 - 制造流水线优化
- **AOE网（关键路径）适合项目管理和资源优化**，确保重点任务得到优先处理。
 - 项目时间管理
 - 活动策划时间控制
 - 物流与供应链管理

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

数据与数据结构

一、数据结构

4.1 数、数据及数据结构

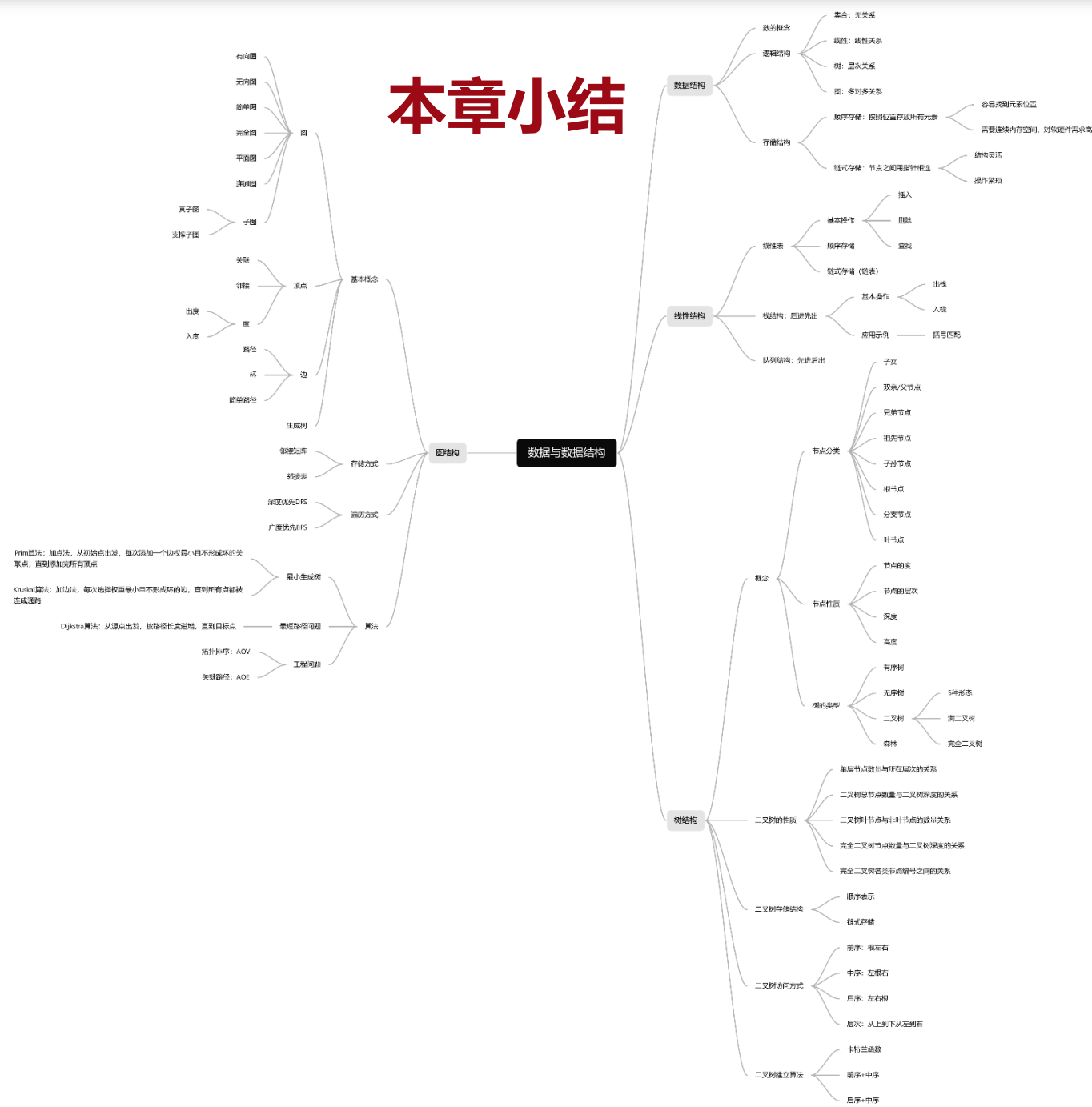
4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例





数据与数据结构

一、数据结构

4.1 数、数据及数据结构

4.2 线性结构

4.3 树形结构

4.4 图结构

二、实例

4.5 应用举例

谢谢大家